密　　级：_____

加密论文编号：_____

论文题目：AI 大语言模型应用数据安全风险监控平台设计与实现

学　　号：M202340082

研 究 生：胡程钰

专 业 名 称：软件工程

2025 年 12 月 25 日

115.25.60.50

AI 大语言模型应用数据安全风险监控平台设计与实现

Design and Implementation of a Monitoring Platform for Data Security Risks in AI Large Language Model Applications

研究生：胡程钰

指导教师：周贤伟 教授

北京科技大学数理学院

北京 100083，中国

Master Degree Candidate：Chengyu Hu

Supervisor：Dr. Xianwei Zhou

School of Mathematics And Physics

University of Science and Technology Beijing

30 Xueyuan Road，Haidian District

Beijing 100083，P.R.CHINA

115.25.60.50

中图分类号：	学校代码：10008
U D C：	密　　级：公开

北京科技大学硕士学位论文

论文题目：AI 大语言模型应用数据安全风险监控平台设计与实现

研究生：胡程钰

指 导 教 师：周贤伟 单位：北京科技大学 职称：教授

指导小组成员：　　单位：　　职称：

　　单位：　　职称：

论文提交日期：2025 年 12 月 25 日

学位授予单位：北 京 科 技 大 学

摘 要

随着 ChatGPT 等大语言模型成为员工提升效率的利器，一个隐蔽的风险也随之浮现，员工在有意或无意的情况下，极易将客户资料、未公开的研究数据等敏感信息输入至第三方 LLM 应用，这些行为使企业不仅面临知识产权流失的直接风险，更可能触发严厉的数据合规审查及处罚。因此，构建有效的 LLM 应用数据安全风险监控机制，已成为企业安全管理中的关键课题。

现有针对 LLM 应用数据安全风险的研究工作多集中于模型安全性评估、数据脱敏技术或权限控制等方面，缺乏针对企业内部员工使用行为的有效监控手段。为此，本文设计并实现了 AI 大语言模型应用数据安全风险监控平台，聚焦员工使用 LLM 应用过程中的数据泄露风险。

该平台通过中间人代理技术采集终端流量，并基于小语言模型（SLM）的语义分析和泛化能力自动识别 LLM 应用流量特征；通过集成预设规则与数据分类分级策略，实现对敏感数据外泄的实时监测。此外，系统引入 RAG 检索增强生成技术为告警信息提供智能分析与处置建议，辅助安全人员高效响应。

测试结果表明，本平台能够有效识别员工使用的 LLM 应用并实时监控数据泄露行为，从而提升企业数据安全管控能力。本研究成果为企业构建 LLM 应用数据安全监控体系提供了可行的技术路径与实践参考。

关键词：大语言模型；数据安全监控；流量分析；信息泄露；企业内部风险

Design and Implementation of a Monitoring Platform for Data Security Risks in AI Large Language Model Applications

ABSTRACT

While large language models (LLMs) like ChatGPT have become powerful tools for enhancing employee productivity, a hidden risk has emerged: employees, whether intentionally or inadvertently, can easily input sensitive information such as customer data or unpublished research into third-party LLM applications. These actions expose enterprises not only to the direct risk of intellectual property loss but also to severe data compliance reviews and potential penalties. As a result, establishing an effective monitoring mechanism for data security risks associated with LLM applications has become a critical issue in corporate security management.

Existing research on LLM data security risks has primarily focused on model security evaluation, data desensitization techniques, and access control, while lacking effective monitoring mechanisms for employee usage behavior within enterprises. To address this gap, this paper designs and implements an AI-powered data security risk monitoring platform for LLM applications, with a specific focus on mitigating data leakage risks during employee usage.

The platform employs a man-in-the-middle proxy to collect terminal traffic and utilizes a small language model (SLM) for semantic and generalized analysis to automatically identify traffic characteristics of LLM applications. By integrating predefined rules and data classification and grading strategies, it enables real-time monitoring of sensitive data leakage. Furthermore, the system incorporates Retrieval-Augmented Generation (RAG) technology to provide intelligent analysis of alerts and remediation recommendations, thereby assisting security personnel in responding more efficiently.

Test results demonstrate that the platform effectively identifies LLM

applications used by employees and monitors data leakage in real time, thereby enhancing the enterprise's data security control capabilities. This research offers a practical technical pathway and actionable insights for organizations to build a comprehensive data security monitoring framework for LLM applications.

Key Words: Large Language Models; Data Security Monitoring; Traffic Analysis; Information Leakage; Internal Corporate Risk

目 录

摘 要 I
ABSTRACT III
1 引言 1
1.1 研究背景与意义 1
1.1.1 大语言模型国内外发展现状 2
1.1.2 企业大模型应用数据安全风险发展现状 3
1.1.3 企业大模型应用数据安全风险监控的必要性 4
1.2 监控平台设计与实现的研究目标与方法 5
1.2.1 研究目标 5
1.2.2 研究方法 6
1.3 创新点 7
1.3.1 基于 RAG 技术的告警智能分析方法 7
1.3.2 基于 SLM 的 LLM 应用流量特征自动化发现 8
2 监控平台关键技术与理论基础 10
2.1 中间人代理技术 10
2.2 ElasticSearch 框架 11
2.3 Flask 及 FastAPI 框架 12
2.4 ElastAlert 告警技术 13
2.5 RAG 检索增强生成技术 13
3 监控平台需求分析 15
3.1 监控平台需求目标 15
3.2 监控平台用户及其需求调研 15
3.3 监控平台功能需求分析 16
3.3.1 终端流量采集模块 16
3.3.2 特征管理及自发现模块 18
3.3.3 监控规则管理模块 19
3.3.4 智能分析模块 20
3.3.5 数据存储管理模块 21
4 监控平台设计与算法研究 23
4.1 监控平台架构设计 23

4.1.1 监控平台整体架构 23
4.2 监控平台数据库设计 26
4.2.1 设计原则 26
4.2.2 数据库模型设计 27
4.2.3 数据存储与管理 28
4.3 LLM 应用流量特征自发现功能的算法研究 29
4.3.1 LLM 应用流量特征发现问题定义 29
4.3.2 形式化数学建模 30
4.3.3 算法正确性与收敛性 35
4.3.4 算法复杂度分析 37
5 监控平台实现 38
5.1 监控平台实现概述 38
5.1.1 技术栈和工具选择 38
5.1.2 开发平台与环境搭建 43
5.2 终端流量采集模块 44
5.3 基于 RAG 技术的告警分析 49
5.3.1 知识库构建与向量化存储 49
5.3.2 告警触发与智能检索 50
5.4 大模型应用流量特征发现模块 53
5.4.1 LLM 应用流量特征自发现算法 53
6 监控平台测试 60
6.1 监控平台测试计划与策略 60
6.2 监控平台测试结果与分析 64
7 结论与展望 67
7.1 研究结论 67
7.2 后续工作展望 68
参考文献 69
附录 A 流量特征自发现算法代码 75
致　谢 81
作者简历及在学研究成果 83
独创性说明 84
关于论文使用授权的说明 85
学位论文数据集 86

1 引言

1.1 研究背景与意义

大语言模型是指在海量无标注文本数据上进行预训练得到的大型预训练语言模型，例如 GPT-3，PaLM 和 LLaMA。目前大语言模型所需要具有的最小参数规模还没有一个明确的参考标准，但是大语言模型通常是指参数规模达到百亿、千亿甚至万亿的模型；也有部分工作认为经过大规模数据预训练（显著多于传统预训练模型如 BERT 所需要的训练数据）的数十亿参数级别的模型也可以称之为大语言模型（如 LLaMA-2 7B），通过大规模未标记的数据训练，模式可以捕获自然语言中的复杂模式，并完成自然语言处理[1]、翻译[2]、文学创作[3]、情感分析、对话[4]等诸多自然语言处理任务。

当前市场内对大语言模型在音视频创作、教育培训、代码开发等领域内应用都有诸多的探索和实践性研究。随着人工智能技术的飞速发展，AI 大语言模型应用已成为企业及员工提升效率、优化决策的重要工具。这些大模型具有处理海量数据的能力，能够在短时间内完成复杂的数据分析任务，为企业提供精准的市场预测、客户洞察和业务优化建议。但是，在提供这些智能分析服务的同时，AI 语言大模型的广泛应用也暴露出了潜在的数据安全风险[5]。企业员工在利用 AI 语言大模型应用进行日常工作时，可能会由于故意、操作失误、安全意识不足或应用漏洞，无意或有意将敏感信息，如客户资料、企业机密、市场战略等，输入到大模型应用中，被其用于训练和使用。这种敏感信息的泄露不仅会直接影响到企业的商业利益，可能导致客户信任度下降、市场竞争优势丧失，还可能违反数据保护法规，如中国的《个人信息保护法》、《数据安全法》以及《生成式人工智能服务管理暂行办法》[6]，欧盟的通用数据保护条例（GDPR）或加州消费者隐私法案（CCPA），从而引发法律诉讼、高额罚款，甚至影响企业的市场准入资格[7]。

此外，数据泄露事件导致的企业形象受损，想要修复其代价往往高昂且周期漫长。因此，如何监控及感知员工在使用大语言模型应用时的行为，防止敏感数据泄露，已成为企业信息安全领域一个亟待解决的关键问题。这不仅关乎企业的即时利益，更是对企业长期可持续发展的直接威胁。为

应对此挑战，本文旨在设计并实现一套面向企业场景的大语言模型应用数据安全风险监控平台（以下简称“监控平台”），通过对员工使用 LLM 应用的行为进行监测，为企业监控员工是否合规使用 LLM 大语言模型应用提供技术支撑。

1.1.1 大语言模型国内外发展现状

语言模型的发展大致可分为四个阶段，包括统计语言模型、神经语言模型、预训练语言模型、大语言模型[8]，其中 20 世纪 90 年代兴起的统计语言模型[9]基于统计学习方法使用马尔可夫假设建立预测模型，其对于上下文的刻画能力较弱，2013 年，神经语言模型[10]随着图灵奖获得者 Yoshua Bengio 引入分布式词表示，构建聚合上下文特征的目标词预测函数进入了发展快车道，针对自然语言的处理能力也进一步提升。

2017 年谷歌提出了基于自注意机制的 Transformer 模型，通过自注意机制建模长程序列关系[11]，支持 GPU 或者 TPU 进行并行加速训练，并提出了预训练语言模型 BERT[12]，同年 OpenAI 也迅速采用 Transformer 架构，将其用于 GPT-1 的训练，以 ELMo、BERT、GPT-1 为代表的语言模型确立了“预训练-微调”这一任务求解范式，预训练阶段主要是通过大规模无标注文本建立模型的基础能力，微调阶段这使用标注数据对模型进行特定任务的适配，从而更好解决自然语言处理任务。

研究人员发现，通过增加模型参数或数据规模的扩张可以带来模型性能的提升，这种现象被称为“扩展法则”（Scaling Law）[13]。2020 年 5 月，OpenAI 发布了包含 1750 亿参数的 GPT-3[14]，这种大规模的预训练语言模型在解决复杂任务时表现了与小型预训练模型（例如 15 亿参数的 GPT-2）不同的能力，GPT-3 可以通过上下文学习的方式利用少样本数据解决下游任务，而 GPT-2 则不具备此种能力，这种大模型具有但小模型不具有的能力被称为“涌现能力”，为了区分这一能力上的差异，学术界将这些大型预训练语言模型命名为“大语言模型”（Large Language Model, LLM）[15]。

2022 年 11 月 30 日，OpenAI 基于 GPT-3 发布了一款人工智能对话聊天机器人 ChatGPT，其出色的自然语言生成能力引起了全世界的广泛关注。

2021 年 3 月，中国第一个 AI 大语言模型“悟道 1.0”由北京智源人工智能研究院发布[16]。2023 年 3 月，百度正式发布“文心一言”AI 大语言模型应用[17]，成为中国第一个类 ChatGPT 产品，随后我国的 AI 大语言模型应用也进入发展的快车道，百花齐放[18]。现在已有许多商用或免费开放的

大语言模型助手供公众使用，诸如阿里的通义千问，月之暗面推出的 Kimi 等大语言模型应用助手，也包括基于开源的 LLaMA、Baichuan、ChatGLM 等进行开发的第三方大语言模型应用。

1.1.2 企业大模型应用数据安全风险发展现状

随着使用的用户不断增多，随之而来频发的数据安全事件也引发了大家对大模型使用安全性的思考。大模型的数据安全风险[19]，主要有三大数据来源风险，包括数据获取侵权、数据内容违法、吸收虚假信息[20]。同时大语言模型训练所需的一部分训练数据来源于用户的被迫让与[21]。

例如，ChatGPT 使用过程中需要用户输入数据并通过算法模型对输入数据进行处理，才能有结果输出，在用户数据输入、数据处理、数据输出阶段均可能发生数据安全事件，例如根据 ChatGPT 使用条款的归档，ChatGPT 对任何用户输入的输入及输出信息有广泛的使用权来改善 ChatGPT 应用[22]，不同于公开的社交平台，大语言模型很有可能会收集到大量高度敏感的隐私信息乃至保密信息[23]。

2023 年初，据硅谷媒体报道，亚马逊的公司律师称，他们在 ChatGPT 生成的内容中发现了与公司机密“非常相似”的文本，可能是由于一些亚马逊员工在使用 ChatGPT 生成代码和文本时输入了公司内部数据，该律师担心输入的信息可能被用作 ChatGPT 迭代的训练数据[24]。

同年，DeepMind 研究人员发表论文称其发现 ChatGPT 新漏洞，通过让 ChatGPT 重复一个词，在重复一定次数后模型就会偏离正常的聊天式生成，开始输出与训练数据更接近的文本，泄露模型的训练数据，且模型训练时间越长，其泄露的数据越多[25]。

考虑到信息泄漏和合规风险[26]，同年摩根大通开始禁止员工使用 ChatGPT，随后花旗集团和高盛集团也针对 ChatGPT 进行了禁用，微软和亚马逊就曾因担心泄漏机密信息而禁止公司员工向 ChatGPT 分享敏感信息，埃森哲也因类似原因警告员工不要将客户信息透露给 ChatGPT[27]。

2023 年 4 月，三星员工在使用 ChatGPT 处理工作时，无意间泄漏了公司的绝密数据。员工在使用过程中输入了机密数据，包括新程序的源代码本体、与硬件相关的内部会议记录等数据，由于 ChatGPT 会保留用户输入数据以进行进一步训练自身，因此三星的商业机密已被 ChatGPT 的开发商 OpenAI 所掌握[28]。

这些事件无不提示着企业需要审视自身内部更加安全可控的使用大语

言模型应用，根据 Gartner 发布的《2025 年数据防泄漏（DLL）市场指南》，有效管理数据外泄风险仍是企业面临的关键挑战之一，各类组织正基于多重动因积极部署数据防泄漏解决方案。值得注意的是，越来越多的机构对利用 DLP 技术控制机器对敏感数据的访问表现出显著兴趣[29]。

生成式人工智能（GenAI）的广泛应用与企业数据规模的持续扩张，使得数据外泄风险进一步加剧。但是，传统 DLP 方案普遍缺乏对 AI 交互场景的感知能力，既无法有效识别用户与大型语言模型（LLM）工具之间的通信行为，也难以对应用层的数据流进行精准监控。该问题的核心在于，现代数据防泄漏已超越简单的关键字匹配，转而要求系统具备对数据语义和业务上下文的深度理解能力。该指南进一步明确，下一代数据防泄漏工具应支持对静态敏感数据的识别，并具备跨通道数据流动的监测能力，尤其需覆盖生成式 AI 场景下的数据传输行为。

1.1.3 企业大模型应用数据安全风险监控的必要性

生成式 AI 工具的爆发式普及，意外地打开了企业数据安全的"潘多拉魔盒"。员工无意识地将客户数据粘贴到 AI 聊天窗口，或使用 AI 工具处理敏感文档，都可能造成数据外泄。越来越多的案例，促使企业开始思考除了相信大模型会按照其声明的保护用户输入的数据以外，是否还有其他更加主动的方式去管控员工使用 AI 大语言模型应用中的数据安全问题。

并且，随着其功能不断强化，国内越来越多的公司、组织和个人开始依赖大模型来处理日常事务。目前针对大模型技术最常见的使用就是知识库场景，从 2023 年至 2024 年，包括智谱清言在内的 AI 大语言模型应用都加入了检索增强生成（Retrieval-Augmented Generation,RAG）等技术[30]，供用户上传文档、数据，通过引入外部知识库的信息来辅助大语言模型生成更准确且符合上下文的答案。这一功能在方便用户的同时也带来了新的数据安全问题，用户上传的资料文件会作为大模型的上下文进行输入，如果在过程中上传了公司的敏感信息和资料，那么在回答时也会将文档中的信息召回作为参考输出，造成数据泄露。

国外频发的数据安全事件也引发了国内对于大模型安全性的思考，部分大型企业选择自建大模型，但对于大部分企业来说成本太大，且实际应用效果还有待市场检验，那么除了禁止员工使用大模型这种方式之外，是否有其他缓解措施能够帮助企业监控或阻止用户向大模型应用输入敏感数

据也是企业正在寻找的解决方案。通过构建监控系统，并结合企业内部数据分类分级管控要求，可以实时监测分析员工的大语言模型使用情况，帮助管理者进行全面的分析和评估，同时为制定替代方案，及时发现内部机密数据外泄风险提供有利支撑。

1.2 监控平台设计与实现的研究目标与方法

1.2.1 研究目标

企业员工的终端使用环境复杂多变，涉及到内部不同密级的资料混合处理，私人资料处理以及跨终端等多种场景。传统的数据防泄漏措施，如数据防泄漏系统往往依赖于特征库，不具备动态感知、智能分析与主动预警功能，难以及时有效的发现员工使用大语言模型应用过程中的数据安全风险。伴随着员工办公过程中与 AI 应用的不断深入融合，针对员工数据安全风险的监控方式也急需数字化和智能化。本研究旨在开发一个高效智能的大模型数据安全风险监控平台，以提升企业或机构对员工使用大语言模型过程中的数据安全风险监控能力。具体而言：

一是构建一个能够实现大语言模型应用流量日志实时采集、处理和分析的智能化监控平台。对大模型应用流量数据进行存储、分析和展示，其中采集端使用 C/S 架构[31]实时采集终端大语言模型流量数据，管理端则采用 B/S 架构[32]和可视化技术，帮助管理人员直观查看和处理大语言模型数据安全风险，帮助其快速发现处置相关数据外泄风险。

二是提升企业大模型数据安全管控效率和智能化水平。借助大数据存储技术、监控平台以及大语言模型的自然语言处理分析能力，实时预警和智能分析员工使用大语言模型过程中的数据安全风险，如第三方恶意大语言模型应用预警、高密级数据外泄预警，使企业或机构能够及时发现和处理相关数据安全风险，提供针对数据安全事件的应急响应速度和处置效率，确保企业数据安全。

三是设计一个基于小语言模型的流量特征智能发现算法，利用语言模型强大的语义分析和泛化能力，实现大语言模型应用流量特征的自动化发现，解决人工发现效率低、发现不及时的问题，发现的流量特征将通过实时数据接口自动推送至监控规则管理、主动预警模块，提升管控覆盖面和及时性。

为了实现本文研究目标，本文采用了不同维度的研究方法。首先，通过文献分析研究方法，本文梳理了国内外大模型发展现状、大语言模型应用的数据安全风险以及现有针对大语言模型数据安全风险的管控方法，为本文提供了研究方向和现实依据。其次，通过系统设计研究方法，本文确定了平台的整体架构和基于用户需求的功能模块，同时根据需求制定了详细的设计方案和 开发计划。最后，通过软件开发和测试研究方法，本文实现了平台的各项功能，并基于需求开展了全面的测试和评估。

本研究开发实现了一个高效智能的企业级大语言模型数据安全风险监控平台。平台的落地应用显著提高了企业或机构内部对员工使用大语言模型过程中的数据安全风险监测管控效率和能力，为员工合法合规使用大语言模型应用和企业研究应用替代提供了数据支持。未来，将继续完善该监控平台的功能和性能，探索在大模型全生命周期中的数据安全风险监测和自动化响应能力和新的应用趋势，为企业管控数据安全风险，保障核心数据不外泄以及平衡员工正常使用需求和风险之间提供更加高效、智能的解决方案。

1.2.2 研究方法

在企业大语言模型数据安全风险监控平台的设计与实现研究中，本文以文献分析，软件开发以及系统设计多种严谨科学的研究方法，来确保本文研究成果的准确性和实用性。

文献分析是本文研究的基础，通过对国内外关于大模型发展历程，数据安全风险管控，监控平台设计等领域的文献进行多维度搜集和深入研读，得以全面了解现有大模型发展现状及其数据安全风险变迁趋势。在文献分析过程中，本文重点关注了大模型数据安全风险现状，管控手段以及监控分析的基本原则和工具。通过对这些文献进行梳理和总结发现，尽管国内外在大模型数据安全风险管控方面已经取得了诸多成果，但更多是在模型本身的管控，对于企业来说想要管控大语言模型在内部应用过程中的数据收集，传输，使用，存储，销毁阶段的风险，尤其是员工使用大语言模型过程的风险，仍存在较大的研究空间。本文确定了整体的研究方向和目标，即开发并实现一个智能，高效的大语言模型数据安全风险监控平台。

平台设计直接决定了监控平台的整体架构和功能实现。在平台设计过程中，本文首先进行了目标群体分析和用户需求分析，通过问卷调研和访谈的方式，收集了企业管理人员，信息安全管理人员等群体对监控平台的

具体期望和使用场景，为平台设计提供了依据。在梳理完需求和使用场景的基础上，进行了系统架构设计，包含采集客户端、数据存储端、前端、后端的设计思路以及技术选型。采用 C/S（客户端/服务器）加 B/S（浏览器/服务器）的混合架构，以确保数据采集和监控平台的分离和可扩展性及易用性。同时对监控平台进行了模块和功能划分，包括流量日志采集、监控，预警，智能分析等核心功能模块，明确了各模块的实现方式和具体功能细节。

在软件开发方面，基于需求和系统架构，选择了合适的开发工具及技术环境。其中前端使用 HTML5、Tailwind CSS 和 JavaScript，使用 Python 作为后端及采集端开发语言，Flask 作为后端服务框架，MariaDB 作为数据库管理系统，ElasticSearch 作为流量日志存储管理系统。在软件开发过程中，参考瀑布模型开展软件开发流程，进行了需求分析、设计、编码、测试、优化等工作，过程中利用 Git 进行版本版本控制和协作，使用禅道进行项目管理和协作，通过持续的迭代和优化工作，本文最终实现了企业级大语言模型数据安全风险监控平台。

在软件开发完成后，本文进行了系统测试及信息安全测试，以确保监控平台的稳定可靠性以及安全性。通过制定详细的测试计划和策略，对监控平台进行了功能测试、性能测试、用户体验测试以及信息安全测试。通过测试，发现并修复了一些潜在的信息安全漏洞和影响用户使用的问题，提升了监控平台的稳定性和安全性。

1.3 创新点

在企业级大语言模型数据安全风险监控平台的研究和开发过程中，基于用户调研和需求痛点，重点针对企业传统管控大语言模型应用的薄弱环节和痛点进行了创新以提升其管理效率和用户体验。

1.3.1 基于 RAG 技术的告警智能分析方法

传统告警分析系统面临着一些核心痛点，也即海量、冗杂、专业化的安全告警与安全运营人员（尤其是初级人员）有限的专业知识和经验之间存在巨大鸿沟。这常常导致运营人员告警研判效率低下、误报处置消耗大量资源、真正的高危威胁被淹没在“告警噪音”中。为解决这一挑战，本平台引入检索增强生成（Retrieval-Augmented Generation, RAG）技术，设

计了一套智能告警分析方法，其创新性主要体现在以下两个方面：

（1）构建动态更新的多源异构安全知识库，实现知识赋能。传统系统依赖静态规则库，难以应对快速演变的用户业务场景和数据安全风险。本方法创新性的整合并向量化多源知识，包括：企业内部安全策略与合规要求、历史处置案例与专家经验库、以及最新的外部安全威胁情报。通过 RAG 架构，系统能实时检索与当前告警最相关的多维度知识片段，为信息安全管理人员分析决策提供动态、全面、符合内外部规范的背景信息支撑，打破了传统系统知识固化、更新滞后的局限。

（2）平台可生成具备操作落地性的处置建议与自动化剧本，实现安全运营从“分析”到“行动”的创新。基于告警信息，从知识库中检索匹配处置策略、修复方案和应急预案。大语言模型据此生成具体、可操作的安全响应建议，例如“建议立即联系该员工，处置该事件，同时针对 XX 应用进行域名封禁”。更进一步，该方法可与 SOAR（安全编排、自动化与响应）系统集成，将这些建议转化为可一键执行的自动化处置剧本（Playbook），如自动下发隔离指令、触发扫描任务等，从而极大缩短平均响应时间（MTTR），实现了从智能分析到自动化处置的闭环，极大的提升企业数据安全运营的整体效率与水平。

1.3.2 基于 SLM 的 LLM 应用流量特征自动化发现

对大模型应用的网络流量进行精准识别与特征提取，是实施有效安全监控的前提。然而，传统基于端口、固定关键字或正则表达式的检测方法在面对 LLM 应用时完全失效，因其多使用标准 HTTPS 端口、通信内容为加密且动态变化的 JSON 格式，API 端点与参数结构也因应用而异。当前解决方案多依赖于人工逆向工程或固定配置，耗时费力、滞后性强，无法适应 LLM 应用的快速迭代与多样化发展。针对这一“看不见则管不住”的初始性难题，本平台创新性地提出了一种基于小语言模型（Small Language Model, SLM）[33-34]的 LLM 应用流量特征自动化发现方法，小语言模型又称小型语言模型，他是指相对于大语言模型而言，其规模更小、参数量更少，能结合 RAG、专业领域知识库、智能化信息检索等手段，在有限资源下实现更高效、精准的服务[35]。该算法核心思想如下：

（1）以“AI 理解 AI 流量”，本方法摒弃了传统手动提取特征的思路，转而使用一个轻量级 SLM，使其成为“流量语义理解专家”。该模型以经过解密的 HTTP/S 请求/响应报文中的关键文本字段（如 URL 路径、Host

头、请求体 JSON 片段、响应体 JSON 片段）作为输入。通过输入真实 LLM API 特征数据作为专业领域知识，该 SLM 能够理解不同 LLM 服务提供商（如 OpenAI、Azure OpenAI、Anthropic、国内各大模型厂商）及自研模型的 API 调用模式、参数结构和语言风格，从而仅凭流量内容就能以极高准确率自动识别出流量所属的具体 LLM 应用或服务，并打上相应标签，解决了加密流量中应用身份识别的根本问题。

（2）实现细粒度 API 端点与参数的自动化结构化提取，动态构建监控元数据。SLM 能够对未知流量进行深度语义解析，自动抽取出关键的特征元素，并形成结构化的知识，如 API 端点发现识别出如 /v1/chat/completions、/v1/embeddings 等核心功能端点。其次还包含参数键名提取，自动发现如 model、messages、temperature、stream 等关键请求参数，以及响应中的 message.content、id、usage 等字段。整个过程自动化，无需人工干预。提取出的特征将动态更新平台的监控策略库，配置针对特定参数（如 model 字段）的监控规则，实现了监控策略与 LLM 应用本身的同步迭代，解决了手动配置的滞后性问题以及人工维护的大量成本。

（3）平衡效能和成本，与直接调用通用大模型 API 接口（如 DeepSeek）进行流量分析相比，可本地部署无需大量资源和成本的 SLM 是本方法的关键优势。SLM 模型体积小、推理速度快、计算资源消耗极低，可满足企业级监控对性能的要求。同时，基于领域知识赋能，它在特定任务上的表现可以媲美甚至超越通用大模型，实现了专业化、高效率与低成本的统一，保证了技术方案落地可行性。

2 监控平台关键技术与理论基础

本章主要介绍 AI 大语言模型数据安全风险监控平台在设计与实现过程中的关键技术，包括中间人代理技术、RAG 检索增强生成技术、ElasticSearch 框架、Flask 及 FastAPI 框架、ElastAlert 告警技术的理论基础和选型原因。

2.1 中间人代理技术

中间人代理技术（Man-in-the-Middle, MITM）作为一种网络技术[36]，它允许中间人拦截并潜在地修改两个通信实体之间的数据传输。在这种技术中，中间人将自己置于通信双方之间，使得所有的数据都必须通过中间人控制的中继点。通过这种方式，中间人可以监听、篡改甚至重新路由数据，而通信双方往往对此毫无察觉。中间人代理原理，如下图 2-1 所示：

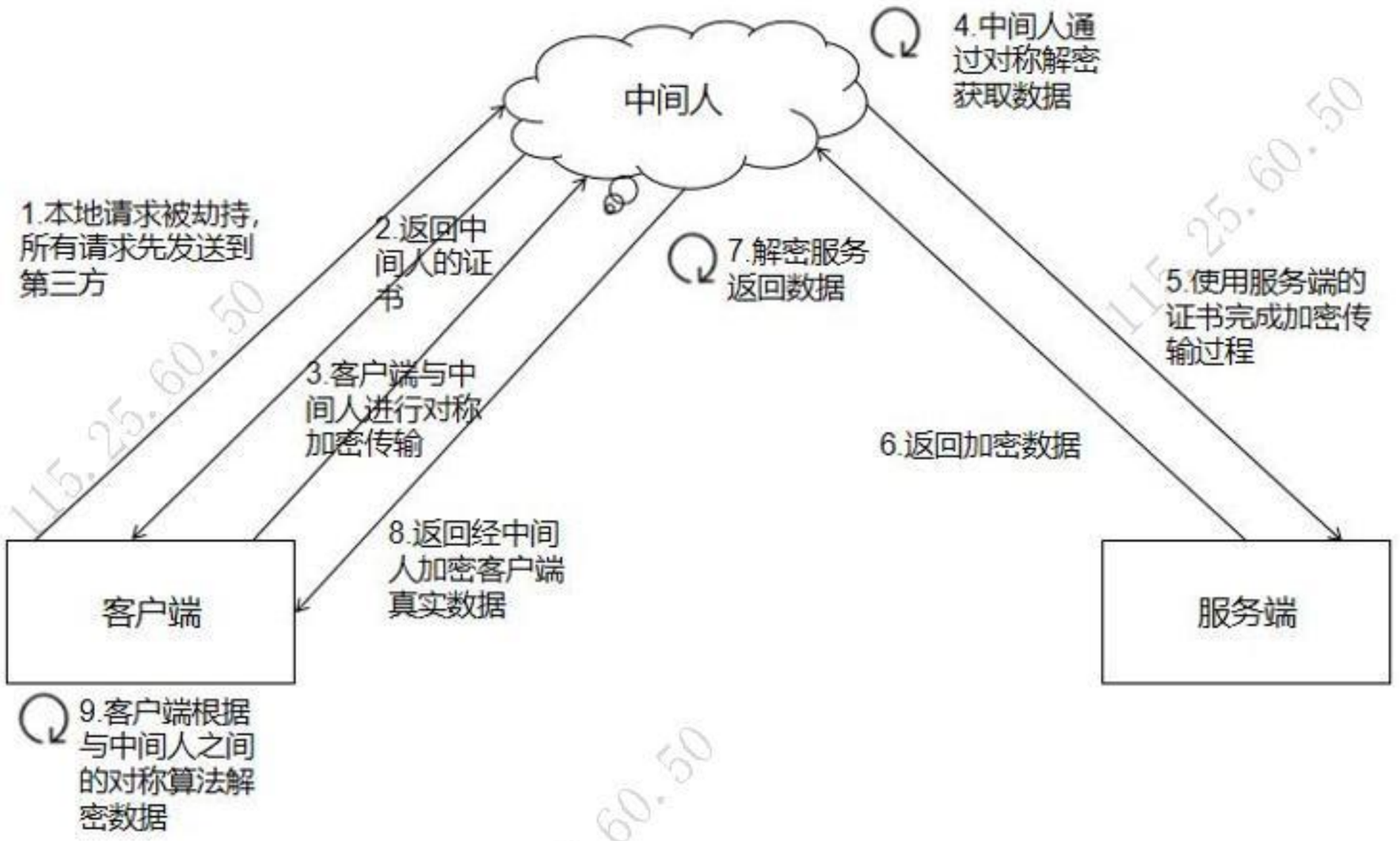


图 2-1 中间人代理原理

Fig.2-1 Theory of Man-in-the-Middle

MITM Proxy（Man-in-the-Middle Proxy）是一个开源的、交互式的 HTTP/HTTPS 代理，安全专家和网络开发者可以使用其拦截、检查、修改和重放网络流量。作为一个中间人代理[37]，MITM proxy 在客户端和服务器之间建立一个透明的代理，使得用户能够监视和操纵网络通信。提供命令行界面、带有图形用户界面的 MITM proxy 和用于脚本化控制的 MITM

dump。MITM proxy 支持 SSL/TLS 加密流量的解密，这要求用户在客户端设备上安装 MITM proxy 的根证书以信任代理。它的功能包括请求和响应的实时过滤、流量保存和重放、自定义脚本编写以修改流量等。这些特性，使得 MITM proxy 在网络安全测试、API 调试、网络监控和数据抓取等领域应用广泛。可以帮助开发者理解应用程序的网络行为，还可以帮助安全研究人员发现和利用潜在的安全漏洞，十分适合本研究中针对终端 LLM 应用流量进行采集的场景。

2.2 ElasticSearch 框架

用户 LLM 应用流量日志十分巨大，需要一套成熟的大数据存储方案才能满足本文记录用户操作 LLM 应用的基础需求。Elasticsearch 是一个基于 Lucene 构建的开源、分布式搜索与分析引擎。它提供了一个具有高扩展性和近实时性能的全文检索平台[38]，能够高效地处理海量数据。其核心优势在于分布式架构、强大的全文搜索能力以及灵活的数据分析功能。

在技术层面，如图 2-2 所示，Elasticsearch 将数据存储为 JSON 文档，并通过倒排索引机制实现极速的文本检索。它支持复杂的聚合操作，能够从数据中快速提取有价值的统计信息。此外，其卓越的水平扩展性允许通过增加节点来轻松应对数据增长与并发压力。

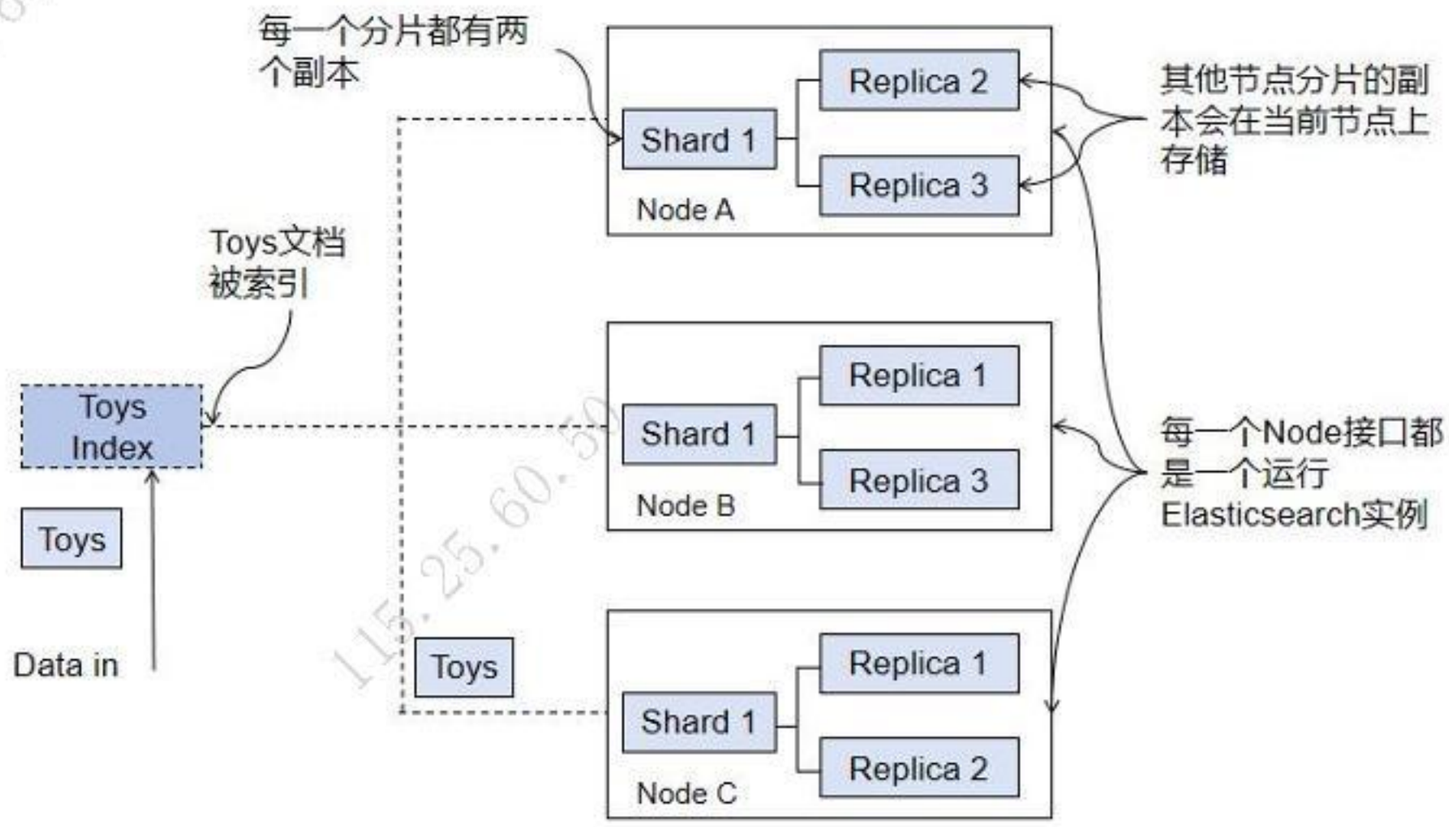


图 2-2 ElasticSearch 存储架构

Fig.2-2 Storage architecture of the elasticsearch

在应用层面，Elasticsearch 远超传统搜索引擎的范畴，已成为日志系统、安全分析、业务指标监控等领域的核心底层技术[39]。在本研究构建的数据

安全风险监控平台中，Elasticsearch 被用作大数据存储与查询引擎，负责高效索引与快速检索海量的流量日志和告警数据，为本研究中平台的实时监控与分析能力提供了关键支撑。

2.3 Flask 及 FastAPI 框架

Flask 是一个微型 Python Web 应用框架，由 Armin Ronacher 开发并于 2010 年首次发布[40]。它被设计为简单、灵活且易于扩展，适用于构建中小型的 Web 应用程序。 如下图 2-3 所示，Flask 的核心特性包括：轻量级与模块化：Flask 本身只提供了核心的 Web 应用功能，如路由、模板和错误处理。它允许开发者根据需要添加扩展，从而构建复杂的应用程序[41]；Flask 使用装饰器来定义路由，即将特定的 URL 映射到处理请求的 Python 函数上。这使得 URL 到函数的映射变得直观且易于管理；Flask 内置了 Jinja2 模板引擎，支持 HTML 文档的动态渲染。开发者可以在模板中嵌入变量和表达式，以及使用控制结构来生成动态内容；Flask 内置开发服务器和调试器，便于在开发过程中进行错误追踪和调试；Flask 基于 WSGI（Web Server Gateway Interface），这是 Python Web 应用程序和服务器之间的一种通用接口标准；Flask 还提供安全性相关的功能，如会话管理、CSRF（跨站请求伪造）保护和数据加密。Flask 社区提供大量的扩展，包含数据库集成、表单验证、用户认证和其他常用功能，可轻松集成到 Flask 应用中[42]。

FastAPI 是一个基于 Python 的现代 Web 框架[43]，具有良好的性能和高效的开发效率。框架基于标准 Python 类型提示构建，并遵循 OpenAPI 与 JSON Schema 等开放的标准。

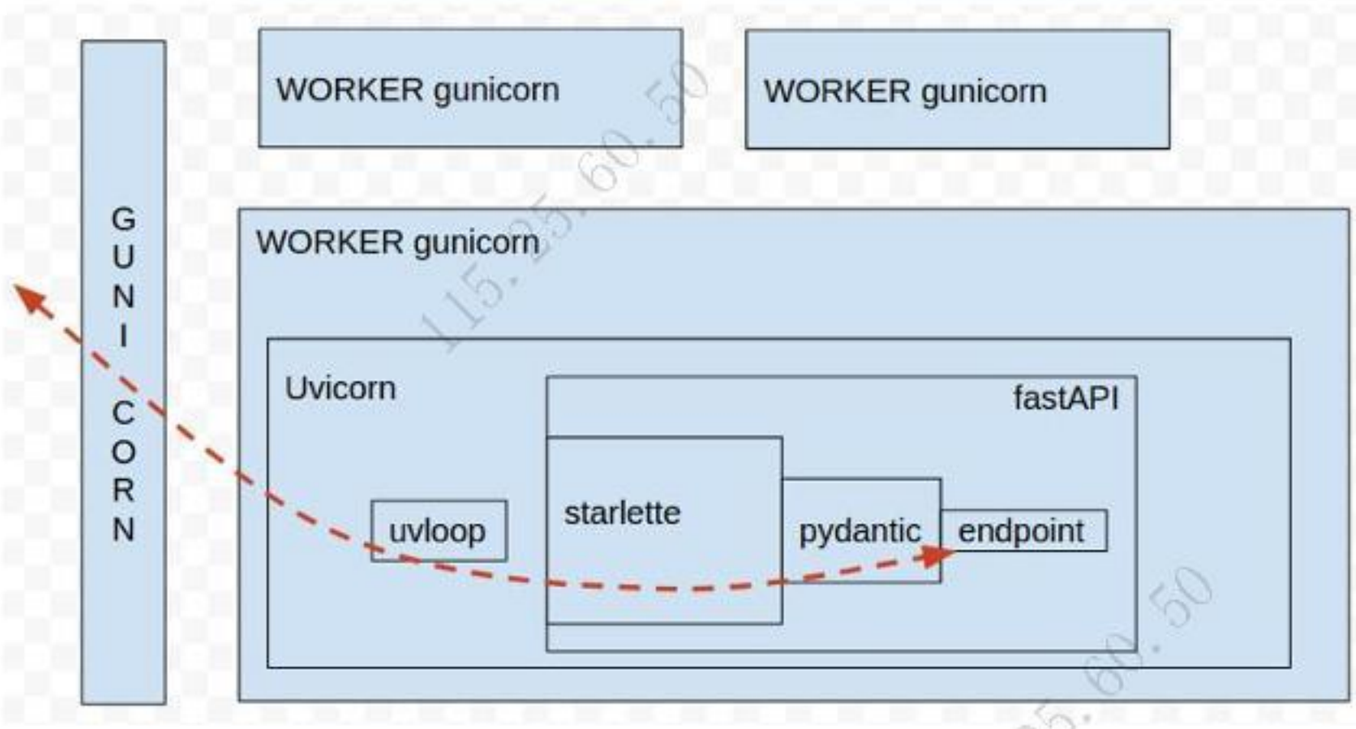


图 2-3 FastAPI 架构图

Fig.2-3 Architecture of the FastAPI

FastAPI 框架的核心优势与本研究的适配性主要体现在以下几个方面：依托 Starlette 与 Pydantic，FastAPI 提供了可与 NodeJS 及 Go 媲美的请求处理性能，满足了监控平台对高并发和低延迟的要求；同时，类型提示系统与编辑器智能补全相结合，可以提高平台开发迭代的速度，通过静态类型检查在编码阶段也可以减少潜在的人为错误，增强了平台代码的健壮性；框架还能自动生成交互式 API 文档，可以确保接口设计的一致性与规范性，极大降低了本研究中平台后续的集成、测试与维护成本。目前，FastAPI 广泛应用在人工智能领域，例如高瞻等设计的新型冠状病毒智能检测系统，就使用 FastAPI 框架作为其后端 API 服务[44]。

2.4 ElastAlert 告警技术

ElastAlert 是一个基于 Python 开发的 Elasticsearch 日志告警插件[45]，用于从 ElasticSearch 中的数据中发出异常，尖峰或其他感兴趣的模式的警报。它通过将 ElasticSearch 与两种类型的组件（规则类型和警报）结合使用。定期查询 ElasticSearch，并将数据传递到规则类型，该规则类型确定找到任何匹配项。发生匹配时，它会发出一个或多个警报，这些警报根据不同的类型采取相应的措施。本研究中使用 ElastAlert 技术，为用户提供规则配置、查询与匹配、触发告警的功能，用户可通过 YAML 文件定义监控规则，包括查询条件、规则类型和告警方式，框架周期性地根据规则查询 Elasticsearch，规则类型组件会对返回的数据进行分析以判断是否匹配规则，一旦规则匹配，警报组件会根据配置采取相应的告警行动。

2.5 RAG 检索增强生成技术

RAG（Retrieval-Augmented Generation，检索增强生成）是一种将信息检索与大型语言模型（LLM）相融合的技术架构，源于 2020 年 Facebook 的一篇论文[46]。其核心思想是通过从外部知识源（如专业数据库、文档库或实时更新的知识图谱）中检索相关信息，为 LLM 的生成过程提供准确、可靠的上下文依据，从而显著提升模型输出内容的专业性和事实准确性。

如图 2-4 所示，在技术实现上，RAG 首先将用户查询转化为检索指令，从指定的知识库中获取最相关的信息片段；随后，将这些检索到的信息与原始查询组合成一个增强的提示，输入至 LLM 中进行内容生成。这种“检索-增强-生成”的机制，使模型能够突破其训练数据的时间和内容限制，有

效减少模型幻觉现象，同时确保生成内容与最新、最权威的知识保持一致。

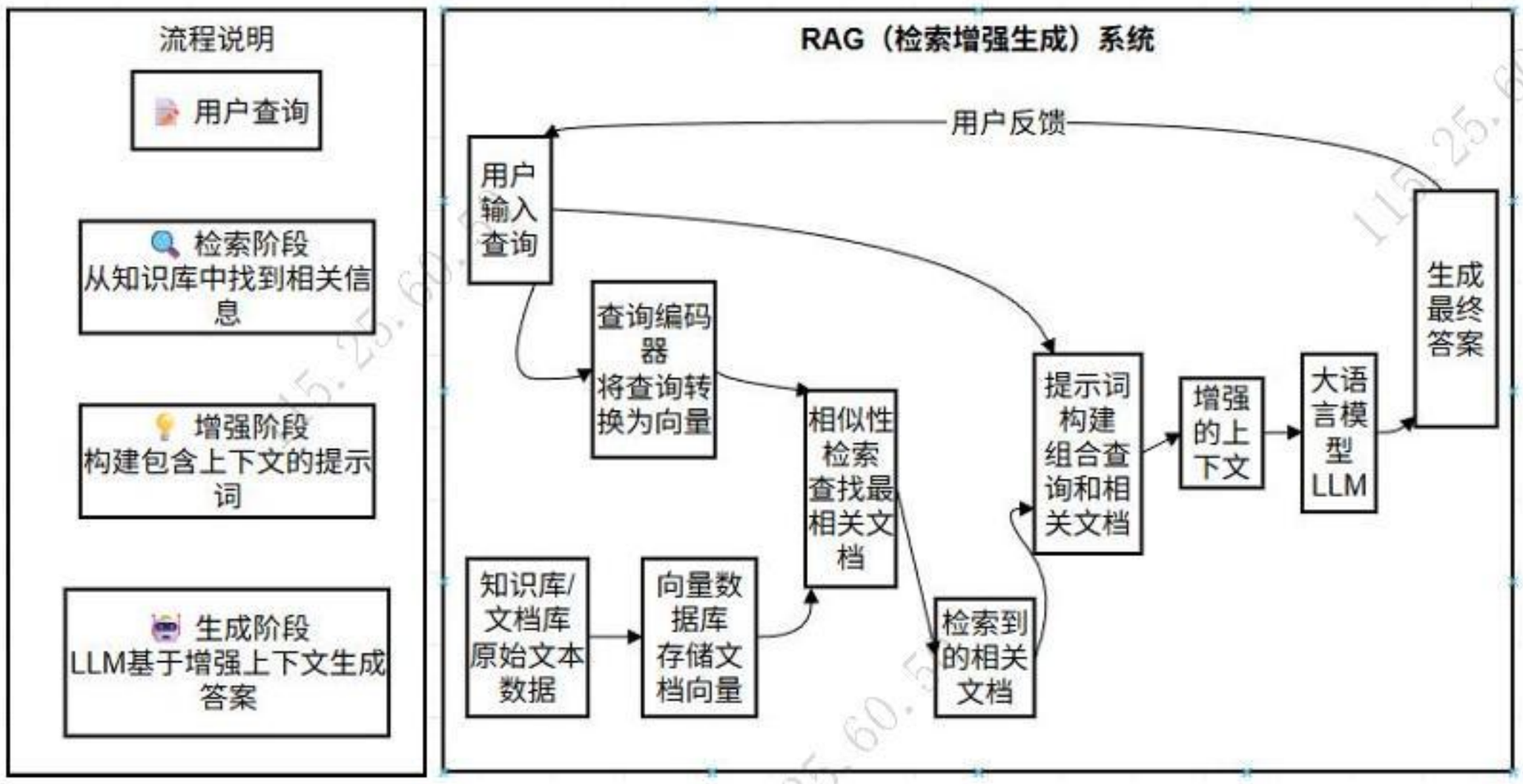


图 2-4 RAG 技术原理图

Fig.2-4 Theory of the RAG

RAG 技术特别适合应用于对准确性和时效性要求较高的专业领域，例如智能客服、学术研究辅助、医疗诊断支持[47] 以及企业知识管理系统[48]。不仅可以提升人工智能处理复杂专业问题的能力，还能为构建可信、可靠的专业级 AI 应用提供坚实的技术基础。本研究中使用 RAG 检索增强生成技术进行告警智能分析，提升安全人员对风险事件的研判和响应效率。

3 监控平台需求分析

在大语言模型数据安全风险监控平台设计的初始阶段，对企业多类相关用户开展系统且深入的需求调研具有至关重要的意义。调研所获得的结论将直接影响平台功能模块的构建与用户体验的塑造。通过系统性的需求收集与分析，明确高效监控平台所应具备的核心功能，是指导本平台设计与开发工作的关键前提。

3.1 监控平台需求目标

大语言模型应用数据安全风险监控平台是针对企业员工使用大语言模型应用过程中数据安全风险监控的智能化监控平台。需要实现多个目标，一是需要实现终端侧大模型应用流量的自动化采集和远程管理，保障监控平台数据来源稳定可用。二是依托大量日志数据处理能力，实现数据安全外泄风险的实时告警。三是实现大模型应用的自动发现和监控特征自动更新。四是提供对告警的智能化分析，能够提供用户专业的分析和处置建议。

3.2 监控平台用户及其需求调研

在大语言模型应用数据安全风险监控平台的设计和开发过程中，识别目标用户群体，并深入了解用户需求是至关重要的环节。本系统用户设定为企业或组织内部管理人员，主要用户包括：

（1）企业信息安全管理人员：负责企业内部数据安全管理、应急响应管理的专业人员。

（2）信息安全监控分析人员：专门负责监控企业内部各渠道和场景下数据安全风险情况，以及开展风险应急响应的专业人员。

为了确保该监控平台能够真正满足实际企业内部数据安全运营过程中的需求，本文通过问卷和访谈的方式对上述用户群体进行了深入调研，以下为对各用户调研的详细结果分析。

企业信息安全管理人员作为信息安全第一责任人着重强调监控平台对大模型应用监控的全面性以及平台本身的稳定和安全性，面对企业内部工作各个环节都存在使用大语言模型进行工作场景，员工可能会使用多种模型应用，如何快速准确无遗漏的识别是监控的关键。其次是平台本身的稳

定性，在面对大量的流量日志时如何能够做到快速检索分析，监控不中断。最后，其期望平台成为大语言模型数据安全管理的统一平台，通过自动发现、归类、形成内部的模型特征库，为管理层决策智能化办公的发展方向提供数据支持，提升企业内部管理的数字化、智能化水平。

安全监控人分析员作为平台运营使用的核心角色，对大语言模型应用数据安全风险监控平台的核心诉求集中在大语言模型应用的自动化发现和告警 AI 辅助分析上。在其日常监控运营工作中，如何及时准确发现员工在使用那些 LLM 应用是其痛点之一，其次就是产生出来的告警如何更加准确和可落地，最好能结合RAG技术使用之前积累的专家经验对告警进行分析，并提供响应处置经验。

3.3 监控平台功能需求分析

在大语言模型应用数据安全风险监控平台的设计和实现过程中，核心功能模块的定义也是其实现的关键环节。这些模块直接影响到企业或组织数据安全风险管控水平，也对企业拥抱大语言模型应用在企业内落地起着重要推动作用。本小节将详细阐述终端流量采集功能、特征管理及自发现模块、监控规则管理、数据存储管理模块以及基于 RAG 技术的智能分析模块的具体内容及其应用场景。

3.3.1 终端流量采集模块

终端流量采集模块作为整个监控平台的数据感知层，其核心功能是从企业网络环境中海量、异构的原始流量中，捕获与大模型应用相关的通信日志，并为后端智能分析提供初步处理后的数据基础。模块设计与实现遵循高效性与低侵入性的原则。具体而言，该模块通过在终端上部署轻量级的采集代理（Agent）软件实现数据获取。Agent 采用中间人代理（Man-in-the-Middle Proxy）的技术架构，在不中断正常业务通信的前提下，对经过该节点的 HTTPS/HTTP 等网络流量进行解密。中间人代理技术（Man-in-the-Middle, MITM）是一种网络攻击手段，它允许攻击者拦截并潜在地修改两个通信实体之间的数据传输。在这种攻击中，攻击者将自己置于通信双方之间，使得所有的数据都必须通过攻击者控制的中继点。通过这种方式，攻击者可以监听、篡改甚至重新路由数据，而通信双方往往对此毫无察觉。

Agent 具备两级核心处理能力来实现对数据的处理。第一级为精准过滤：Agent 内置轻量级规则匹配引擎[49]，动态加载并应用从监控平台下发的“大模型流量特征库”，对流量进行实时解析与模式匹配，只有当流量与已知特征库规则吻合时，才会将其判定为目标流量并记录其日志。第二级为特征预处理：对于未被特征库覆盖的未知或可疑流量[50]，Agent 不会直接丢弃，而是对其进行去重和关键特征提取。去重策略基于流量五元组、请求体哈希值等关键信息，有效避免了对高度相似流量的重复处理；特征提取则聚焦于 URL 路径、Host 头、请求/响应体中的代表性片段等核心元数据。这些经过去重和标准化处理的未知流量特征样本，将被一同上传至监控平台。

已知大模型流量的详细日志和未知流量的特征样本，都通过 HTTPS 传输加密并以准实时或定时批处理的方式，回传至监控平台的数据存储模块。这一设计可以为上层风险分析提供准确的已知流量日志，同时为后端“特征自发现模块”的自动化学习与识别功能，提供无冗余的原始数据输入，形成从数据采集到智能分析的闭环，增强平台对未知大模型应用的主动发现能力，具体功能如表 3-1。

表 3-1 终端流量采集功能

Tab.3-1 Functions of collection of the endpoint flow

功能名称	功能简要描述	功能说明
终端代理部署	在终端设备上部署轻量级采集代理（Agent），实现数据获取的起点。	该功能通过在终端部署专用软件，构建了分布式数据采集能力，确保能从企业网络环境的各个节点获取原始流量，是实现全面监控的基础。
流量透明代理与解密	采用中间人代理技术架构，对 HTTPS/HTTP 等网络流量进行拦截与解密。	此功能是数据可读性的关键。它在不中断正常业务通信的前提下，解密加密流量，使得后续的内容分析与特征匹配成为可能。
精准流量过滤	内置规则匹配引擎，根据下发的特征库，实时识别并筛选出与大模型应用相关的目标流量。	该功能通过动态加载和匹配“大模型流量特征库”，实现了从海量异构网络流量中精准捕获目标通信数据，极大减少了无用数据的传输与处理。
未知流量特征预处理	对不匹配已知规则的未知或可疑流量，执行去重和关键特征提取操作。	此功能避免了简单丢弃未知流量可能造成的风险。通过基于五元组和请求体哈希的去重，以及提取核心元数据，为后端智能发现模块提供了高质量的数据样本。
数据回传	将处理后的已知流量日志和未知流量特征样本，加密传输至平台数据存储模块。	通过 HTTPS 加密和准实时/批处理传输策略，确保采集数据在传输过程中的机密完整性，兼顾了网络传输效率。

3.3.2 特征管理及自发现模块

特征管理及自发现模块是本平台实现对大模型应用流量进行精准识别与动态监控的核心模块。该模块的主要功能是解决大模型 API 流量因多样化、加密化及快速迭代而带来的识别挑战[51]，由以下两个协同工作的子模块构成，具体功能如表 3-2。

大模型流量特征管理子模块。该子模块提供对大模型流量特征数据进行全生命周期管理的能力，支持信息管理员对特征库进行创建、读取、更新与删除等标准化操作。用户可通过该模块定义并维护一套结构化的特征指纹，关键字段包括大模型应用名称、唯一的 URL 路径模式、典型的 Host 头信息、以及请求体与响应体中的关键片段正则表达式。这套特征集合构成了平台识别不同大模型流量的知识基础。模块还提供一键同步功能，可将经过审核的特征策略实时下发并部署至终端流量采集模块。确保新的监控策略及时生效，将指定的大模型应用流量纳入持续的日志采集与监控范围，为后续的安全分析提供可靠的数据来源。

流量特征自发现子模块。为应对企业环境中不断出现的新的大模型应用及未知 API 流量，单纯依赖手动维护特征库的方式存在滞后性与局限性。本平台创新性的集成了特征自发现能力。该子模块以后台服务的形式运行，其核心是基于小语言模型[52]构建的智能分析引擎。该引擎会对采集端上报的、尚未被识别的网络流量元数据（如 URL、Host 头、请求/响应体片段）进行深度语义解析与模式识别。通过将其与已有特征库进行匹配与概率推断，能够自动判别未知流量是否属于大模型应用。分析结果将以置信度排序的列表形式推送给管理用户进行审核。经确认后，用户可将高置信度的特征条目一键导入至正式的特征库中。通过“人机协同”的闭环流程极大地提升特征库的维护效率和平台对未知风险的主动发现能力。

表 3-2 特征管理及自发现模块

Tab.3-2 Feature management and self-discovery module

功能名称	功能简要描述	功能说明
特征库管理	提供对特征指纹的创建、查询、修改和删除等标准化管理功能。	支持管理员通过可视化界面对特征库进行维护，包括定义应用名称、URL 路径模式、Host 头信息及请求响应体特征等关键字段，形成平台识别流量的知识基础。
特征策略实时同步	将审核通过的特征策略下发至终端采集模块。	通过一键同步机制，确保新的监控策略能够快速部署到所有终端节点，实现对指定大模型应用流量的及时采集与持续监控。
未知流量智能分析	基于语义分析引擎对未识别流量进行自动识别和分类。	采用 SLM 小语言模型对采集端上报的未知流量元数据进行解析，通过语义匹配和领域知识，自动判断其是否属于大模型应用流量。
特征审核	提供人机协同的特征发现与确认机制。	系统将智能分析结果按置信度排序后推送给管理员审核，经人工确认后方可纳入正式特征库，确保特征质量的同时降低误判风险。

3.3.3 监控规则管理模块

监控规则管理模块是本平台实现数据安全监控的核心功能模块，模块功能如表 3-3 所示。通过集中化的策略管理能力，确保平台监控行为的一致性、可审计性与高效性。

模块提供监控规则管理功能[53]，支持安全管理员进行规则的创建、查询、修改、删除与启用/禁用操作。每条规则均包含多个可配置维度：规则名称、风险类型（如敏感信息泄露、政策合规性、模型滥用等）、触发条件、告警级别（如高危、中危、低危）以及应用范围。其中，应用范围策略是确保规则精准生效的关键，管理员可根据企业组织架构，将规则定向应用于特定的部门、用户组或单个用户，从而实现差异化的安全管控，避免“一刀切”带来的运营干扰。在规则定义层面，模块支持基于关键字的精细化策略配置，来应对典型的滥用场景。例如，为防范员工无意或恶意向公共大模型服务上传敏感数据，管理员可创建规则，对传输内容进行实时扫描。当检测到请求体或上传文件中包含预设的敏感关键词（如“机密”、“源代码”、“客户名单”）或符合特定正则表达式模式（如身份证号、银行卡号模式）时，系统将立即触发告警。

表 3-3 监控规则管理模块

Tab.3-3 Functions of management of monitoring rule

功能名称	功能简要描述	功能说明
规则管理	支持对监控规则的创建、查询、修改、删除及启用/禁用操作。	提供完整的规则管理功能，安全管理员可通过界面完成规则从创建到删除的全过程管理，确保规则与安全需求保持一致。
规则配置	提供规则名称、风险类型、触发条件、告警级别等多参数配置能力。	每条规则包含多个可配置维度，管理员可根据实际需求灵活定义规则属性，确保规则能够准确反映不同的安全场景和风险等级。
范围控制	支持按部门、用户组或单个用户进行规则定向应用。	通过组织架构级联选择，实现规则的精准投放，避免对非目标用户造成干扰，提升安全管控的针对性和实用性。
关键字检测策略	基于关键词和正则表达式的敏感内容识别机制。	管理员可预设敏感关键词（如"机密"）或正则模式（如身份证号），系统对传输内容进行匹配，实现对敏感数据泄露的有效监控。
告警触发	对符合规则的异常操作立即生成安全告警。	当系统检测到请求体或上传文件命中预设规则时，将立即触发相应级别的告警，确保安全团队能够第一时间获取风险信息。

3.3.4 智能分析模块

智能分析模块的核心目是提升平台运营的智能化水平与响应效率。模块基于 RAG 检索增强生成技术，用于解决传统安全运营中心在面对大模型应用告警时存在的分析上下文缺失、研判依赖专家经验、响应建议生成困难等核心痛点。

模块是一个动态的、知识驱动的闭环流程。当规则引擎产生一条初始告警（如“检测到疑似源代码上传至公共大模型”），该告警事件及其关联的原始流量数据（经由终端采集模块获取，并已由 SLM 进行特征提取与结构化）将被送入分析模块。引擎首先从平台内置的动态知识库中实时检索相关信息片段。这些信息包括但不限于：该告警所涉用户的基础信息、所访问大模型应用的安全策略与合规要求、以及企业内部关于数据分类的最新政策。

检索到的相关知识片段与告警的完整上下文（包括 SLM 解析出的具体 API 端点、请求/响应参数内容等）[54]被共同构成一个增强的提示词，输入到大语言模型中进行深度分析与推理。LLM 在此基础上执行智能研判与根因分析，其输出不再是简单的二元判断，而是一份结构化的分析报告。该

报告会阐明告警的潜在风险本质（例如，判定此次上传是员工无意识的违规行为，还是有目的的敏感数据外泄尝试），评估其真实风险等级。同时，模块还会自动生成响应处置建议。基于上述深度研判，模块会进一步从知识库中检索匹配的处置预案、最佳实践。LLM 据此生成具体、可操作的响应建议，例如："建议立即通过 API 令牌吊销该用户的此次会话权限，并通知其直属经理进行安全意识培训，同时对该用户近期所有大模型访问记录的进行检查"。这些建议将直接推送给安全监控人员进行决策，从而降低平均检测时间和响应时间，实现从被动告警到主动、智能分析的提升，具体功能如表 3-4 所示。

表 3-4 智能分析模块

Tab.3-4 Functions of intelligent analysis

功能名称	功能简要描述	功能说明
告警上下文增强	为原始告警自动关联用户信息、应用策略等背景数据。	当系统产生告警时，自动从知识库中检索相关背景信息，包括用户身份、数据安全策略和数据分类标准，为分析提供完整上下文。
风险深度研判	对告警事件进行多维度分析，识别风险本质并评估威胁等级。	基于增强的告警上下文，系统分析风险性质（如区分无意违规与恶意泄露），并准确评估风险级别，为后续处置提供依据。
处置建议	根据研判结果，自动生成具体可行的处置方案。	结合风险分析结果，系统提供明确的操作建议，如暂停用户权限、通知相关负责人或安排安全复查，供安全监控分析人员参考。
结构化输出	生成包含风险分析、等级评估和处置建议的完整分析结果。	所有分析结果以标准化格式输出，包含风险定性、等级评定和具体处置步骤，便于安全人员快速理解和执行。

3.3.5 数据存储管理模块

数据存储管理模块其设计决定监控平台的数据处理能力、查询效率与可扩展性。面对平台内产生和收集的多源、异构数据（包括高度结构化的系统元数据与半结构化/非结构化的日志数据），采用混合式存储架构，针对不同类型的数据特征与应用场景，提供最优的存储与访问方案。具体而言，将平台数据划分为两大类并进行分治管理：

1）结构化数据存储：对于平台自身产生的元数据，如用户账户、权限配置、监控规则策略、大模型特征库、资产信息等，其数据模型固定，关系性强，且需要保证事务的一致性。选用 MariaDB 关系型数据库作为此类

数据的存储引擎。MariaDB 凭借其成熟的 ACID 特性、强大的 SQL 查询能力以及良好的数据完整性约束，能够高效、可靠地管理这些核心业务数据，为平台的稳定运行提供坚实基础。

2）非结构化日志数据存储：从终端流量采集模块持续上报的原始流量日志、系统运行日志以及安全告警记录，其数据格式主要为非结构化的 JSON 数据格式，其数据量巨大、且写入与查询需求以高并发和全文检索为主[55]。针对这一特点，采用 Elasticsearch 作为核心的日志存储与检索引擎。Elasticsearch 作为一种分布式、近实时的搜索与分析引擎，其出色的水平扩展能力能够轻松应对日志数据的持续增长。更重要的是，其强大的倒排索引与全文检索功能，可以提供高效、灵活的多维度查询手段[56]。

4 监控平台设计与算法研究

4.1 监控平台架构设计

监控平台架构设计是实现企业级平台安全稳定运行要求的关键，需要结合企业终端环境的复杂性、用户需求和技术实现的可行性，构建满足企业级场景下针对大语言模型数据安全风险监控需求的技术体系。在明确系统功能需求的基础上，本文设计了一个基于经典四层架构模式的大语言模型数据安全风险监控系统架构，包含应用层、服务层、基础层和采集层，结合五大核心功能模块协同运行，形成高度自动化、智能化的采集、存储、分析、告警的监控数据流。

4.1.1 监控平台整体架构

为满足大模型应用数据安全风险监控的需求，本系统采用模块化的设计思路，确保各功能模块能够高效协同工作，同时还需要具备良好扩展性和可维护性。系统架构需支持大模型应用日志采集、特征管理及自发现、智能分析、告警等核心功能的大量性能需求，通过良好的可视化设计来实现用户体验和信息交互能力提升。

系统整体架构采用四层结构，包括应用层、服务层、基础层和采集层[57]。该架构设计能够满足系统稳定性及可靠性，并具备良好的扩展性。详细设计如图 4-1 所示：

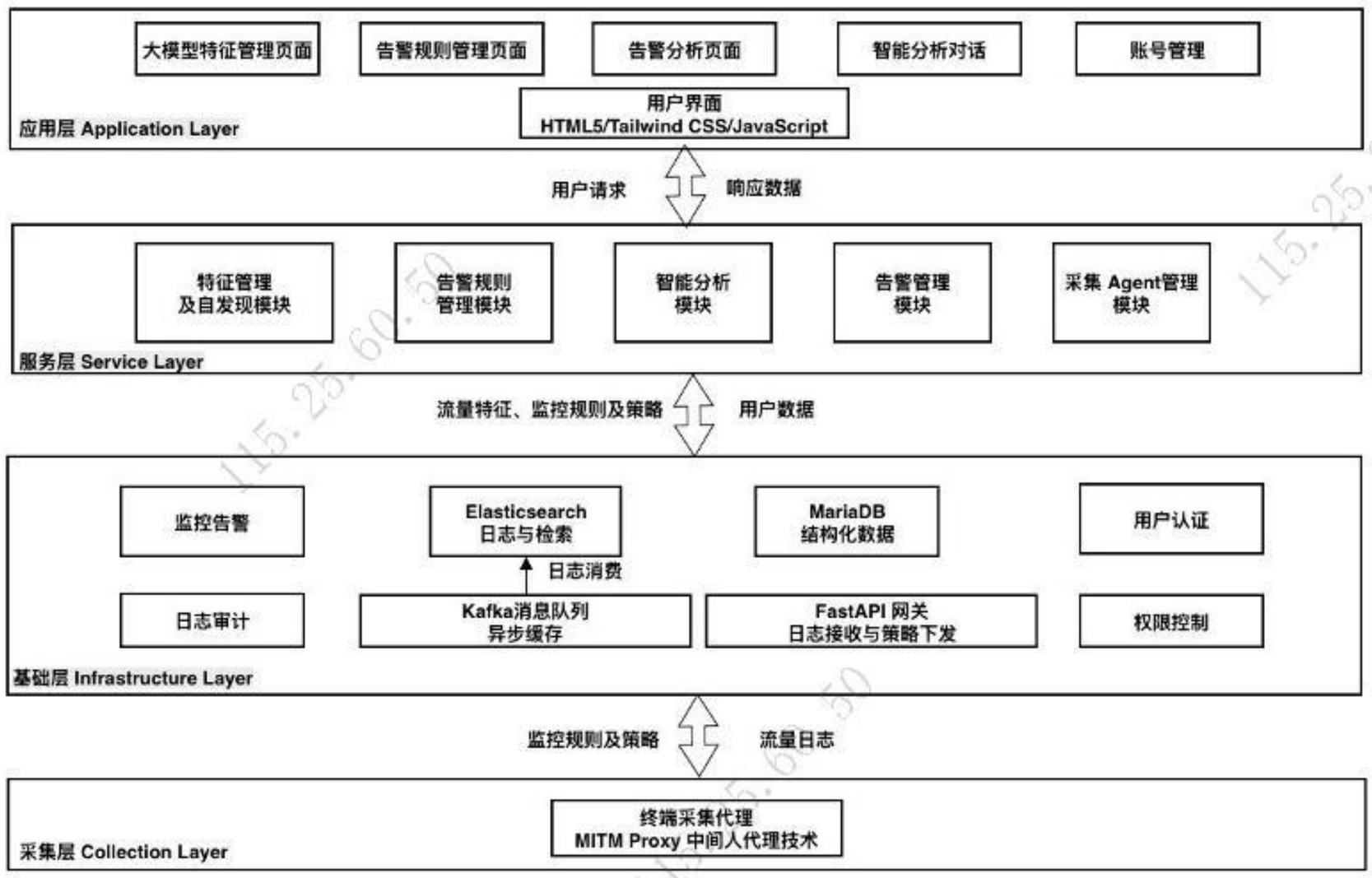


图 4-1 系统整体架构图

Fig.4-1 Architecture of the monitoring system

（1）应用层

应用层作为本平台架构中的顶层，是平台与终端用户进行信息交互的直接前端载体。其核心功能为构建一个统一、可用的用户界面，负责接收用户的操作指令与查询请求，并将其规范地传递至下层的服务层进行处理；同时，该层也承担着将服务层返回的数据结果与业务状态，通过图形化用户界面以清晰、直观的形式渲染并呈现给用户。

用户界面为用户提供直观简洁的交互方式，用户通过该层次及界面能够进行所有服务层的能力交互，包括大模型特征管理、告警规则管理、告警管理、智能分析对话等场景。

为实现上述目标，平台规划采用 HTML5、Tailwind CSS 和 JavaScript 等前端技术，简化前端代码的编写以及美观程度的统一，构建一个简洁、美观且功能丰富的用户界面[58]。

（2）服务层

服务层是本平台的核心部分，负责处理平台业务逻辑。服务层主要包括大模型特征管理及自发现、终端流量采集管理、告警规则管理、告警管理、智能分析等模块。服务层的作用主要是划分各业务逻辑，避免系统结构混乱，各模块功能重叠冲突，保障各模块低内聚性，让各模块协同工作，为用户提供服务，同时也可通过增加模块的方式，快速扩展平台的能力，

快速迭代以满足用户需求和新的业务场景。

在框架方面，本文规划使用 Flask 框架。Flask 框架简单、灵活且易于扩展，提供大量数据库集成、安全性的功能需求，能够根据开发者需要进行扩展来构建复杂的应用程序，十分适合本平台的需求，能够快读构建出稳定高效且安全的后端服务。

（3）基础层

基础层是平台的基础及通用服务层，包括数据存储、用户认证、权限控制、监控告警、日志审计等功能。为应用层及服务层提供必要的基础能力，并保障平台的安全稳定运行。

本平台计划采用 MariaDB 作为结构化数据存储[59]，Elasticsearch 集群作为非结构化日志存储。同时为实现流量日志的异步处理和数据缓存的功能，防止系统故障时日志丢失，同时进一步提升平台的稳定性，还将使用消息队列来进行日志缓存，避免终端采集发送过来的日志丢失。

（4）采集层

采集层作为本平台数据来源与感知前端，承担着最基础且关键的数据获取职责。其核心功能是依据服务层下发的采集策略（包括大模型流量特征指纹与监控规则），在企业内各目标终端上进行部署，持续性地嗅探、捕获并过滤出与大模型应用相关的用户对话流量日志。完成采集后，该层负责将日志数据通过安全信道[60]传输至基础层进行集中存储，为更上层的服务层分析与告警提供原始数据输入。采集层作为整个监控体系的数据源头，其运行的可靠性、数据采集的完备性与准确性，直接决定了平台后续风险感知、分析与预警等核心价值能否得以有效实现。

本平台将基于中间人代理技术（Man-in-the-Middle, MITM）设计 LLM 大语言模型应用流量日志采集模块，通过在 LLM 大语言模型应用客户端和服务器间建立透明代理，解密 SSL/TLS 加密流程，可以在用户无感知的情况下对企业员工使用大语言模型应用过程中的外发行为进行实时监控。终端流量采集架构如图 4-2 所示。



图 4-2 终端流量采集架构

Fig.4-2 Endpoint traffic collection architecture

4.2 监控平台数据库设计

4.2.1 设计原则

本平台数据库结构设计遵循以下原则：

数据分类与异构存储，其中结构化数据（如系统元数据、配置信息、用户信息）采用关系型数据库如 MariaDB 进行存储，利用其事务特性保证数据一致性与完整性；对于对半结构化或非结构化数据（如原始的流量日志、告警详情、分析报告）则采用高性能的搜索引擎 Elasticsearch 进行存储，以满足其大数据量、高并发写入与复杂查询的需求[61]。

在可扩展性方面，设计上应避免单点瓶颈，对于 Elasticsearch 等分布式存储组件，应采用集群化部署，支持通过增加节点的方式实现存储容量与处理能力的线性提升，确保平台长期运行的性能稳定。

数据安全与隐私保护方面，需严格保护员工敏感数据。如涉及个人身份信息、敏感业务等数据，需进行加密存储或展示脱敏，并在传输过程中使用 TLS 协议。同时，数据库访问应遵循最小权限原则，通过严格的账户权限隔离与 IP 白名单访问控制列表，确保只有授权服务和来源才能访问特定数据，并详细记录数据访问日志以备审计。

4.2.2 数据库模型设计

在大语言模型应用数据安全风险监控平台的系统架构设计中，数据库模型的设计对于整个系统稳定运行十分重要。一个合理的数据库模型不仅要满足对平台运行状态数据及终端LLM应用流量日志的高效存储与管理需求，还能在数据一致性、安全性和可扩展性方面具备良好的保障。在模型设计阶段，需要综合考虑实际业务场景中的数据特征、查询效率需求，并为后续可能出现的业务扩展与结构升级预留弹性空间。下表列出了本平台中部分关键数据库表的名称定义及对应的 ElasticSearch 索引名称。如下表 4-1 所示：

表 4-1 主要数据库表/索引名

Tab.4-1 Primary database table name and index name

序号	数据表名/索引名	所属数据存储系统	数据说明
1	user	MariaDB	系统用户信息，如用户名、密码哈希、角色、所属部门等。
2	departments	MariaDB	企业组织架构信息，用于规则和权限的精准应用。
3	app_signatures	MariaDB	大模型应用流量特征库，存储 URL 模式、Host、关键字段正则等。
4	monitoring_rules	MariaDB	监控规则策略，包括规则名称、风险类型、触发条件、告警级别、应用范围等。
5	assets	MariaDB	终端资产信息，记录安装了采集代理的终端设备。
6	agent_config	MariaDB	采集代理的配置策略，包括下发的特征库版本和监控规则。
7	system_logs	ElasticSearch	Elasticsearch 索引，存储平台系统自身的运行日志。
8	analysis_result	ElasticSearch	Elasticsearch 索引，存储智能分析模块生成的结构化分析报告与处置建议。
9	traffic_log	ElasticSearch	Elasticsearch 索引，存储终端采集的原始/处理后的流量日志。
10	alerts_log	ElasticSearch	Elasticsearch 索引，存储产生的安全告警记录，关联规则、用户、资产等信息。

4.2.3 数据存储与管理

在数据存储方案设计阶段，平台面临结构化数据与非结构化数据并存的场景。为满足不同类型数据的存储与访问需求，平台采用混合存储架构，选取 MariaDB 作为关系型数据库，ElasticSearch 作为全文检索与分析引擎。

选择 MariaDB 主要基于以下考量：该数据库在开源关系型数据库中表现出优异的事务处理能力，完全支持 ACID 特性，能够确保用户配置信息、规则库、系统元数据等关键业务数据的一致性与完整性。其成熟的权限管理体系与平台既有的身份认证机制可无缝集成，为数据安全管理提供基础保障。

ElasticSearch 主要用于处理平台大量的半结构化与非结构化数据终端流量日志。该类数据具有数据量大、字段灵活、高频写入与复杂查询等特点。ElasticSearch 强大的全文检索能力与近实时分析性能，刚好能够满足多维度、多条件的日志检索与聚合分析需求，满足安全事件调查与审计场景下对日志数据快速探索与溯源的需求。

ElasticSearch 集群采用分布式部署以满足系统的高可用性与可扩展性。数据在写入时将自动进行分片，并在多个节点间分布存储与建立副本。这种机制不仅实现了数据的高可靠存储，避免了单点故障导致的数据丢失风险，还可以通过将查询请求并行分发至各数据节点，提升大规模数据检索与分析场景下的效率。

对于数据隐私保护，平台在存储层面实施了加密措施。MariaDB 中对存有敏感信息的数据库表列实施了透明数据加密，确保静态数据的安全。

在数据库运行环境安全方面，平台部署了多层次的安全防护体系。在网络层面，数据库服务器部署于独立的网络安全域，通过防火墙严格限制访问源，仅允许特定的应用服务器进行访问。同时，在网络边界部署了态势感知设备，对出入数据库区域的网络流量进行深度检测与异常行为分析。

在主机防护层面，为所有数据库服务器部署了主机入侵检测系统（HIDS），实时监控系统进程、文件完整性及恶意代码活动，并对系统层面的异常操作进行告警和阻断。

通过 MariaDB 与 ElasticSearch 相结合的混合存储方案，构建一个能够兼顾事务处理与大量数据检索能力的存储底座。分布式部署则保障了系统的高可用与可扩展性。同时，通过建立涵盖定时备份、操作审计、存储加密、安全防护与恢复演练在内的多层次数据管理体系，提升了数据的可靠

性、安全性与可恢复性，为平台的稳定、安全运行奠定了坚实的数据基础。

4.3 LLM 应用流量特征自发现功能的算法研究

针对传统应用流量检测方法存在的特征滞后性、高人工维护成本及泛化能力差等问题，本章提出了一种基于小语言模型（Small Language Model, SLM）的端到端流量特征自发现算法。该算法摒弃了传统多阶段复杂特征工程与模型架构，采用"语义理解即服务"的设计理念，通过精心构建的提示词模板[62]，将预处理后的流量关键信息直接输入 SLM 小参数量语言模型进行理解与判断，实现了从原始流量到结构化特征的直接映射。

4.3.1 LLM 应用流量特征发现问题定义

伴随着大语言模型技术的快速演进和广泛应用，涌现出大量源自不同服务商的 LLM API 应用及其流量，如何自动化识别这些应用流量特征属于何种 LLM 应用成为急需解决的问题。传统基于规则库的应用流量识别方法面临严峻挑战，主要体现在三个维度：

（1）特征滞后性问题：考虑如下真实场景，当 OpenAI 将其聊天补全 API 从 /v1/chat/completions 更新至 /v2/chat/completions 时，传统规则库需要人工介入更新，存在显著的时间窗口滞后。这种滞后性在 LLM 应用和服务快速迭代的背景下被急剧放大。

（2）维护成本问题：假设某企业网络中存在 N 个 LLM 服务商，每个服务商平均提供 M 种 API 类型，传统方法需要维护的规则数量呈 O(N×M) 增长。当 N=20, M=5 时，仅基础规则就达 100 条，加上参数变体，维护成本呈指数级上升。

（3）泛化能力问题：传统分类器在训练集{OpenAI, Anthropic}上表现良好，但当新兴 LLM，如 DeepSeek 出现时，由于特征空间的分布偏移，模型识别准确率急剧下降至随机猜测水平。

以上挑战，形式化描述问题为：设传统方法在时间 t 的识别准确率为 $A(t)$，规则库更新周期为 $\mathbf{t}$，则平均识别准确率为 $\bar{A}=(1/\triangle t)\int_{t}^{t+\Delta t}A(\tau)d\tau$，随着 $\mathbf{t}$ 的增大，其平均识别准确率将衰减。

4.3.2 形式化数学建模

为精确描述算法处理的数据对象和转换过程逻辑，建立如下形式化数据模型，并按照预处理-推理-解析三个阶段，通过具体示例阐明各组件语义。

（1）预处理阶段

设原始流量数据空间Ω为所有可能的 HTTP 请求-响应报文，对于任意样本 D　Ω，有 D = (D_req, D_res)，其中 D_req 表示请求报文，D_res 表示响应报文。

以图 4-3 所示的 ChatGPT 聊天 API 调用为例：

```
D_req: POST /v1/chat/completions HTTP/1.1
       Host: api.openai.com
       Authorization: Bearer sk-...
       Content-Type: application/json

       {
         "model": "gpt-4",
         "messages": [
           {"role": "user", "content": "你好，请介绍你自己"}
         ],
         "max_tokens": 100,
         "temperature": 0.7
       }

D_res: HTTP/1.1 200 OK
       Content-Type: application/json

       {
         "id": "chatcmpl-...",
         "object": "chat.completion",
         "model": "gpt-4",
         "choices": [...]
       }
```

图 4-3 LLM 应用调用示例

Fig.4-3 LLM Application call example

随后，对报文进行预处理，预处理函数φ: Ω → X 将原始报文数据映射到富含语义的文本特征空间。具体而言，对于 D　Ω，有：

$$x = \varphi(D) = (f_{path}(D), f_{host}(D), f_{req}(D), f_{res}(D)) \quad (4\text{-}1)$$

其中各组件通过以下方式构造：

f_path(D)：提取 URL 路径，如/v1/chat/completions

f_host(D)：提取主机名，如 api.openai.com

f_req(D)和 f_res(D) ，基于预定义的 LLM API 知识库 K = {"model", "messages", "prompt", "max_tokens", "temperature", "stream", ...}进行操作，其核心是键值对的筛选拼接：

$$f_{req}(D) = concat(k, v)|k \quad K, v = lookup(B_{req}, k), len(v) < L \qquad (4\text{-}2)$$

B_{req}为请求体 JSON 对象。此操作本质上是对高维、稀疏的 JSON 空间进行了一次基于领域知识的降维与投影，得到一個低维、稠密的文本表示，如图 4-4 所示。

```
// 路径提取
f_path(D) = extract_path(D_req.url)

// 主机名提取
f_host(D) = extract_host(D_req.url)

// 请求体特征提取
f_req(D) = extract_keyvalues(D_req.body, K, L)

// 响应体特征提取
f_res(D) = extract_keyvalues(D_res.body, K, L)
```

图 4-4 报文特征提取

Fig.4-4 LLM application traffic message feature extraction

以其中 extract_keyvalues 函数为例，其核心逻辑如图 4-5 所示：

```
function extract_keyvalues(json_obj, knowledge_base K, length_threshold L):
    # 初始化结果列表，用于存储提取的键值对
    result = []

    # 遍历JSON对象中的所有键（包括嵌套键）
    for each key in recursive_keys(json_obj):
        # 检查当前键是否存在于知识库K中
        if key in K:
            # 从JSON对象中获取该键对应的值
            value = get_value(json_obj, key)

            # 检查值的字符串长度是否小于阈值L
            if string_length(value) < L:
                # 将键值对格式化为字符串并添加到结果列表
                result.append(f"{key}: {value}")

# 将结果列表中的所有元素用逗号和空格连接成一个字符串返回
return join(result, ", ")
```

图 4-5 特征提取函数实现示例

Fig.4-5 Feature extration function implementation example

例如，设知识库 K = {"model", "max_tokens", "temperature", "object"}，L = 50，则预处理结果如图 4-6 所示：

```
x = {
    path:"/v1/chat/completions",
    host:"api.openai.com",
    req:{model: gpt-4, max_tokens: 100, temperature: 0.7},
    res:{model: gpt-4, object: chat.completion}
}
```

图 4-6 处理结果示例

Fig.4-6 Processing result example

（2）SLM 推理与生成

随后，进入算法 SLM 推理及生成阶段。本算法的核心在于通过精心设计的提示词模板以及部分领域知识，将流量分析任务转化为 SLM 的自然语言理解任务。提示词模板 T 定义了任务描述、输入格式和输出规范：

$$T = T_task + T_format + T_example + T_knowledge$$

其中任务描述部分明确 SLM 的角色和任务，以及包含如图 4-7 所示的已知 LLM 应用流量特征作为背景知识，输入格式将预处理后的流量数据组织成结构化文本，输出格式定义期望的结构化响应：

$$y' = M(T, x) \tag{4-3}$$

```
一、核心通信模式特征
请求-响应模式，但非对称性极强

请求（用户 -> 服务器）：通常数据量很小，只包含文本提示词（Prompt）、一些参数（如 max_tokens, temperature）和 API 密钥。一个几百字的 Prompt 可能只有几 KB。

响应（服务器 -> 用户）：数据量可能非常大，尤其是生成长文本时。一个生成千字回复的响应，其数据量可能是请求的数十倍甚至上百倍。这种上行小流量、下行大流量的模式是其最显著的特征之一。

普遍使用长连接与流式传输

长连接：为了减少建立 TCP/TLS 连接的 overhead，客户端通常会与 API 服务器保持长时间的连接，并在同一个连接上发送多个请求。

流式响应：为了避免用户长时间等待，绝大多数 LLM API 都支持流式传输（如 HTTP Server-Sent Events）。服务器会一边生成 Token，一边通过同一个连接将数据分块（chunk）发送回客户端。这在网络流量上表现为：在一个较长的持续时间内，服务器端持续地、间歇性地向客户端发送小尺寸的数据包。

二、协议与载荷特征
基于 HTTPS

所有主流 LLM API（OpenAI, Claude, 文心一言等）都强制使用 HTTPS，因此载荷内容本身是加密的。网络层面的监控无法直接读取 Prompt 或生成的文本。

使用 RESTful API 或 gRPC

RESTful JSON：这是最常见的形式。请求体和响应体都是 JSON 格式。特征包括：

请求头：包含 Authorization: Bearer sk-...（API Key）、Content-Type: application/json。

请求体：JSON 结构，包含 "model"、"messages"、"stream" 等字段。

响应体：JSON 结构，包含 "choices"、"usage"（记录了消耗的 Token 数，这是计费的关键）等字段。

gRPC：一些为了追求更高性能的接口可能会使用 gRPC（如某些内部的推理服务）。gRPC 基于 HTTP/2，流量也是加密的，但其二进制格式和多路复用等特征与 RESTful API 不同。

三、时序与行为特征
响应时间较长

与传统 Web 请求（通常在 100ms - 1s 内返回）不同，LLM API 的响应时间（TTFT, Time To First Token）和总完成时间要长得多。

TTFT：从发送请求到收到第一个 Token 的时间，可能在几百毫秒到几秒。

总时间：对于长文本生成，整个响应过程可能持续数十秒甚至数分钟。这在网络流量上表现为一个长时间的、持续的 TCP 连接。

"打字机"式的流式流量

在流式传输模式下，网络流量图会呈现独特的"阶梯"或"锯齿"模式。服务器每生成完一个或多个 Token 就发送一个数据块，导致下行流量是一阵一阵的、非连续的突发，而不是平滑的流。

可能出现的周期性"心跳"或"Ping"
```

图 4-7 LLM 应用流量已知核心特征知识

Fig.4-7 Known core characteristics of LLM application traffic

算法构建的提示词示例如图 4-8 所示：

```
T_task = """
你是一个网络流量分析专家。请分析以下流量数据，识别出：
1. LLM服务提供商 (vendor)
2. API功能类型 (api_type)
3. 关键参数结构 (parameters)
"""

T_format = """
请严格按照以下JSON格式输出：
{
    "vendor": "服务商名称",
    "api_type": "API类型",
    "parameters": ["参数1", "参数2", ...],
    "confidence": {
        "textual": 0.0-1.0,
        "structural": 0.0-1.0,
        "valid": 0.0-1.0
    }
}
"""

T_examples = """
示例1:
输入: 路径: /v1/chat/completions, 主机: api.openai.com, 请求: model: gpt-4, max_tokens: 100
输出: {"vendor": "OpenAI", "api_type": "chat_completion", "parameters": ["model", "max_tokens"], ...}
"""

T_knowledge = LLM_API_Knowledge_Base #LLM API知识库
```

图 4-8 提示词示例

Fig.4-8 Prompt example

（3）结果解析与结构化输出

解析函数ψ: y' → Y 将模型原始输出转换为结构化特征：

$$Y = \psi(y') = (\text{vendor}, \text{api_type}, P) \tag{4-4}$$

其算法实现示例如图 4-9 所示：

```
function ψ(y'):
    // 1. JSON解析与验证
    try:
        obj = json.parse(y')
    catch JSONError:
        return DEFAULT_UNKNOWN

    // 2. 服务商分类映射
    vendor = map_vendor(obj.vendor, C_vendor)

    // 3. API类型分类映射
    api_type = map_api_type(obj.api_type, C_api)

    // 4. 参数结构规范化
    parameters = normalize_parameters(obj.parameters, P_template)

    // 5. 置信度验证
    if not validate_confidence(obj.confidence):
        parameters = filter_low_confidence(parameters, obj.confidence)

    return (vendor, api_type, parameters)
```

图 4-9 算法逻辑示例

Fig.4-9 Algorithm logic example

算法解析过程示例为：

输入：y' = '{"vendor": "OpenAI", "api_type": "chat_completion", "parameters": ["model", "max_tokens", "temperature"]}'

步骤 1: json.parse → Python 字典
步骤 2: map_vendor("OpenAI") → "OpenAI" C_vendor
步骤 3: map_api_type("chat_completion") → "chat_completion" C_api
步骤 4: normalize_parameters(...) → {("model", "string"), ("max_tokens", "integer"), ("temperature", "float")}
步骤 5: 返回 Y = ("OpenAI", "chat_completion", {("model", "string"), ("max_tokens", "integer"), ("temperature", "float")})

算法最终输出空间为三元组，公式如下：

$$Y = C_vendor \times C_api \times P \tag{4-5}$$

其中× 表示笛卡尔积，定义了一个三元组空间，C_vendor 为服务商分类的离散集合，C_api 为 API 类型分类的离散集合，P 为参数结构的集合，每个元素是(参数名, 参数类型)对，该定义建立了算法输出的严格数学空间。

C_vendor = {OpenAI, Anthropic, 智谱 AI, 月之暗面, 未知, ...}

C_api = {chat_completion, embeddings, fine_tuning, 未知, ...}

P {参数名} × {参数类型}，参数类型 {string, integer, float, boolean, array, object}

对于前述流量，算法期望输出为：

Y = (OpenAI, chat_completion,
{("model", "string"), ("max_tokens", "integer"), ("temperature", "float")})

总结上述信息，于是可得到 LLM 应用流量特征自发现算法，如图 4-10 所示。

输入： 原始解密流量 D;
LLM API 知识库 K;
提示词模板 T.

过程：

$\forall d \in D, V = \{\}$;
for i = 1 to |D| **do**

$$x_i = \phi(d_i) = \langle f_{path}(d_i), f_{host}(d_i), f_{req}(d_i, K), f_{res}(d_i, K) \rangle$$

$$y_i^{'} = M(T, x_i)$$

$$Y_i = \Psi(y_i^{'})$$

$$V = V \cup \{Y_i\}$$

end for

输出： LLM 应用特征集合 V

图 4-10 LLM 应用流量特征自发现算法

Fig.4-10 LLM traffic feature self-discovery algorithm

4.3.3 算法正确性与收敛性

本算法本质上是基于一个预训练的、参数固定的 SLM 进行推理，因此其“收敛性”概念不同于迭代优化算法。我们在此重点讨论其正确性与稳定性。

（1）正确性

算法的正确性高度依赖于两个组件：提示词模板 T 的质量和 SLM 本身的能力。

提示词的可验证性，可以通过构造一组覆盖不同服务商、API 类型和边缘案例的测试集S_{test} Ω来验证，对于每个测试样本 Di，运行算法得到输出 ，并与人工标注的基准真值 进行比较。计算准确率指标来量化其正确性：

$$\text{Accuracy} = (1/|S_{test}|) \times \Sigma_{i=1}^{|S_{test}|} I(Y_i = \hat{Y}_i) \quad (4\text{-}6)$$

Accuracy：算法整体的准确率，取值范围 [0, 1]，表示正确预测的比例。

$|S_{test}|$：测试集的大小，即包含的样本总数。

$\Sigma_{i=1}^{|S_{test}|} I(Y_i = \hat{Y}_i)$表示对测试集中所有样本的预测结果进行累加，预测结

果使用 I 指示函数，当预测结果Y_i与真实结果标签$\hat{Y}_i$完全相等时，I 等于 1，不相等为 0。

SLM 能力边界方面，本算法的理论上限受限于所选 SLM 的语义理解与指令跟随能力。如果 SLM 无法可靠地理解提示词 T 中的指令或无法从 x 中提取关键信息，则正确性无法保障。因此，算法正确性的一个关键前提就是 SLM 模型 M 在任务(T, X)上具备基本能力。根据 Zhenyan Lu、Xiang Li 等人的研究，如图 4-11[33]，Qwen2 作为一款涵盖 0.5B、1B 至 7B 参数范围的模型，7B 参数的模型特别适合需要较强模型来执行摘要或文本生成等复杂任务的场景，选用 Qwen2 SLM 模型尤其适合本算法这类需要快速响应和资源受限的应用场景。

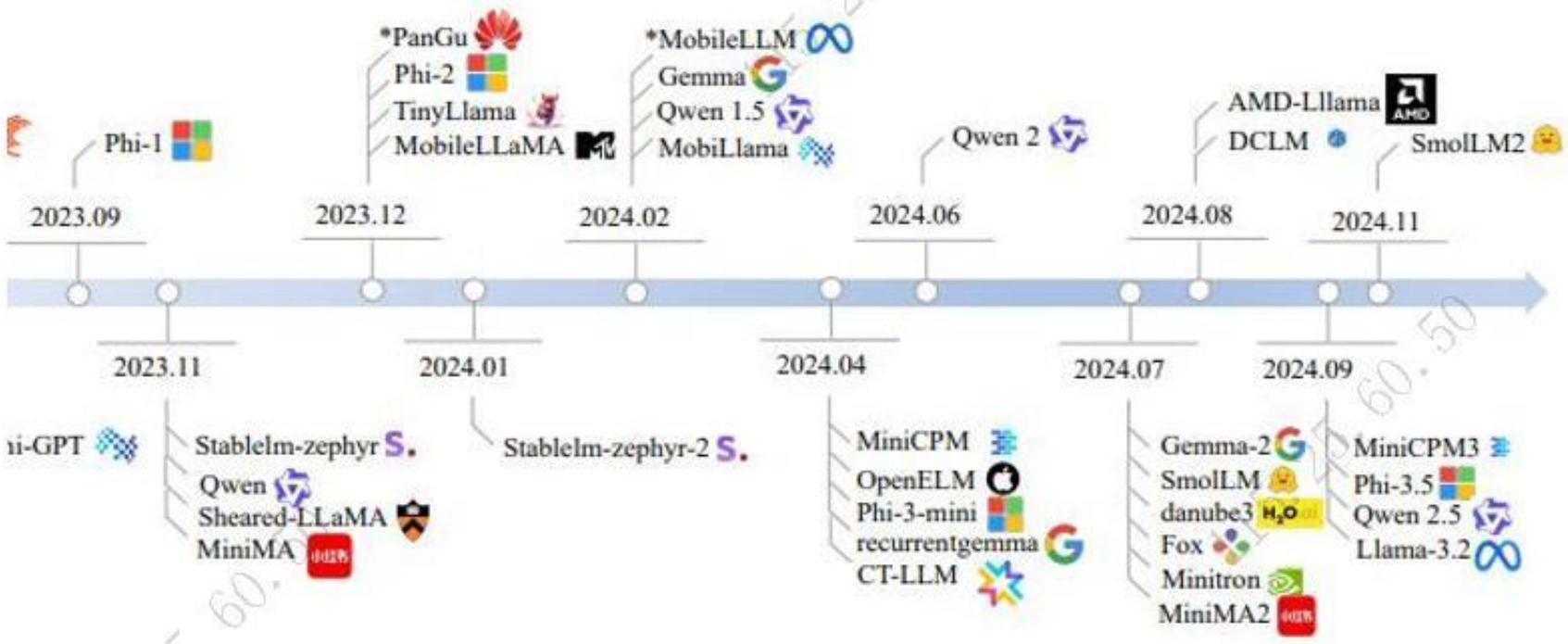


图 4-11 SLM 模型发展轨迹图

Fig.4-11 Known core characteristics of LLM application traffic

（2）稳定性（一种“收敛性”的体现）

对于相同的输入 x 和提示词 T，由于 SLM 的自回归生成可能引入随机性（如采样策略），输出 y'可能存在波动。为保证本算法的稳定性，采取了以下措施：

生成策略：在推理时设置极低的温度（Temperature=0），从而确保对于相同的(T, x)，SLM 模型总是生成相似或相同的 y' 。

解析函数鲁棒性：解析函数ψ能够处理模型输出 y' 在一定范围内的变异（如多余的空格、同义词），确保 Y = ψ(y')的结果是确定性的。

当生成和解析策略固定后，算法 A(D) = ψ(M(T, ϕ(D)))可以视为一个确定性函数，从而实现输出的稳定收敛。

4.3.4 算法复杂度分析

本章提出的基于 SLM 语义理解分析 LLM 流量特征发现算法，其时间复杂度如下：

在数据预处理阶段$\phi(D)$，主要涉及 URL 解析、Host 提取、JSON 解析与递归搜索、字符串拼接等操作。设原始 JSON 体的节点数为 n，递归搜索和键值查找的时间复杂度为$O(n)$，其他操作均为常数时间或与字符串长度成线性关系。因此，该阶段整体时间复杂度为$O(n)$。

在 SLM 推理阶段 M(T, x)，算法主要时间开销为 SLM 的前向传播，基于 Transformer 架构的 SLM 模型，其时间复杂度简化情况下仅与输入序列长度有关[11]，假设序列长度为 n，则其时间复杂度为$O(n^2)$。

在结果解析阶段$\psi(y')$阶段，涵盖字符串匹配、正则表达式、字典查找等操作，该阶段处理的字符串 y' 长度固定且较短，因此所有操作都可以视为在常数时间内完成，即$O(1)$。

综上所述，本算法的总时间复杂度由 SLM 推理阶段主导，为$O(n^2)$。

5 监控平台实现

5.1 监控平台实现概述

本章主要介绍AI大语言模型应用数据安全风险监控平台的核心实现内容。将阐述系统在原型开发阶段所采用的技术栈，并说明各核心功能模块的实现方式，重点包括终端流量采集模块以及基于RAG的数据安全告警分析等关键能力的具体实现方法。

5.1.1 技术栈和工具选择

AI大语言模型应用数据安全风险监控平台架构划分为应用层、服务层、基础层与数据采集层，每一层均基于相应的技术组件构建，各层协作共同提供完整的LLM应用数据采集、存储、监控与告警服务。以下对各技术组件进行详细介绍：

（1）应用层

平台前端基于Tailwind CSS、HTML5以及Java script进行开发，提供简洁美观的响应式用户界面。简洁化设计使得开发者能够轻松的构建和维护易于用户使用的用户界面，同时也确保了代码的可维护性和扩展性。为了给用户提供动态的数据可视化能力，平台集成了Echarts，用于展示监控平台运行过程中的关键数据。通过Echarts，用户可以清晰地查看实时告警数据、已识别的LLM大语言模型特征和告警趋势，支持多种图表类型，包括折线图、饼图、柱状图等，从而辅助安全分析人员和管理人员进行数据分析研判和管理决策。

系统核心页面如下图5-1、图5-2、图5-3所示：

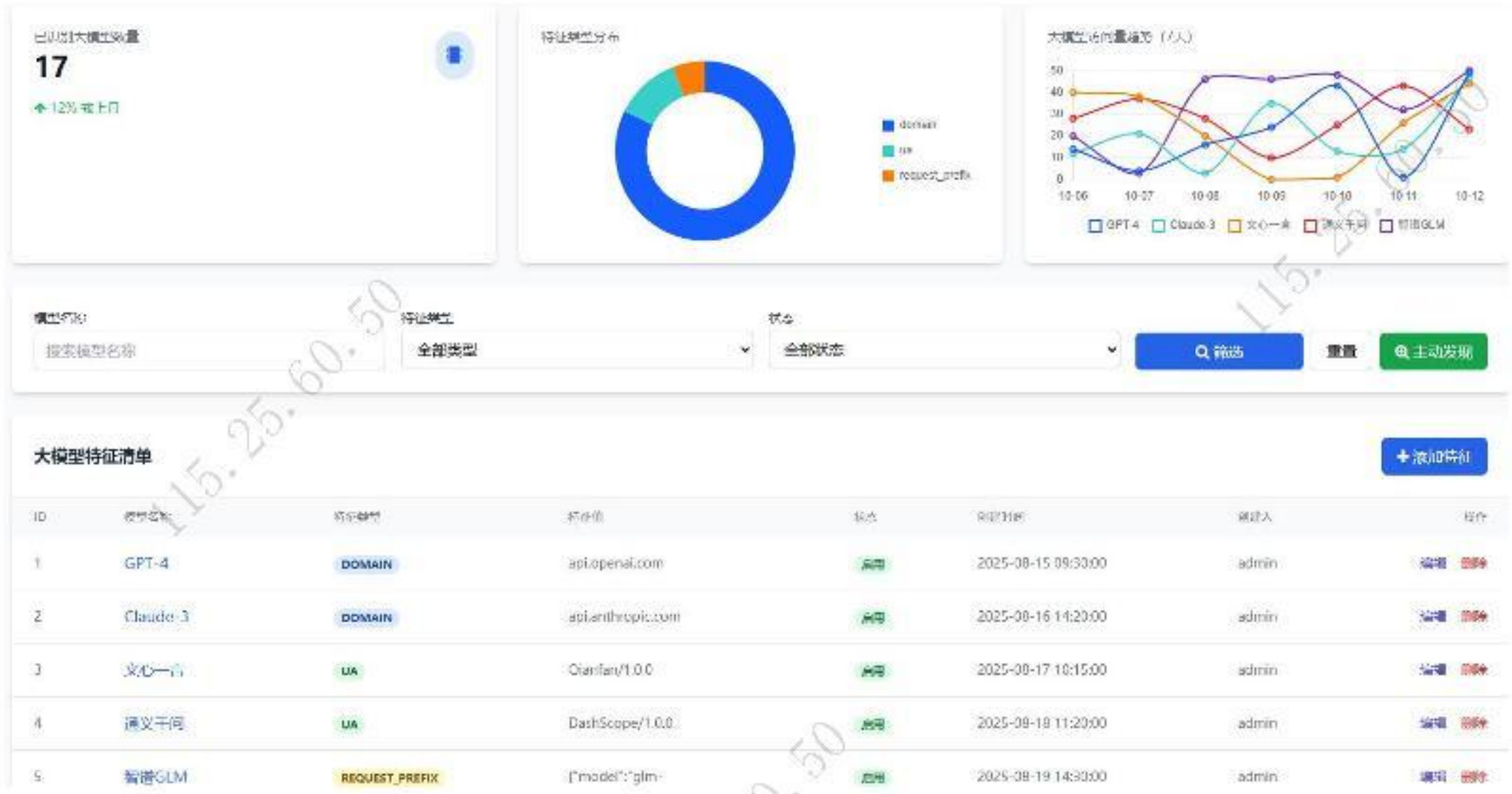


图 5-1 大语言模型特征管理页面

Fig.5-1 LLM model feature management page

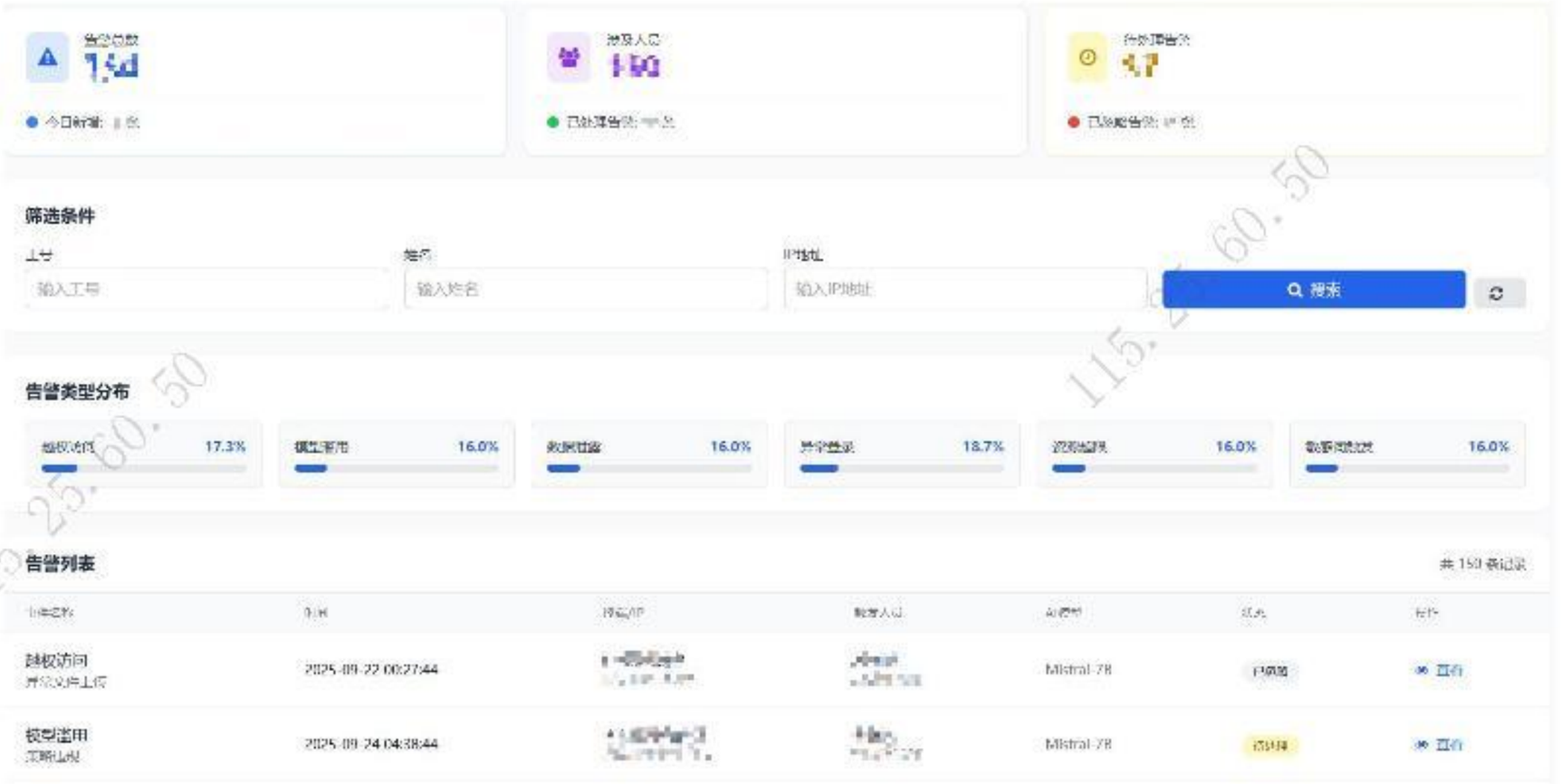


图 5-2 告警列表

Fig.5-2 List of alert

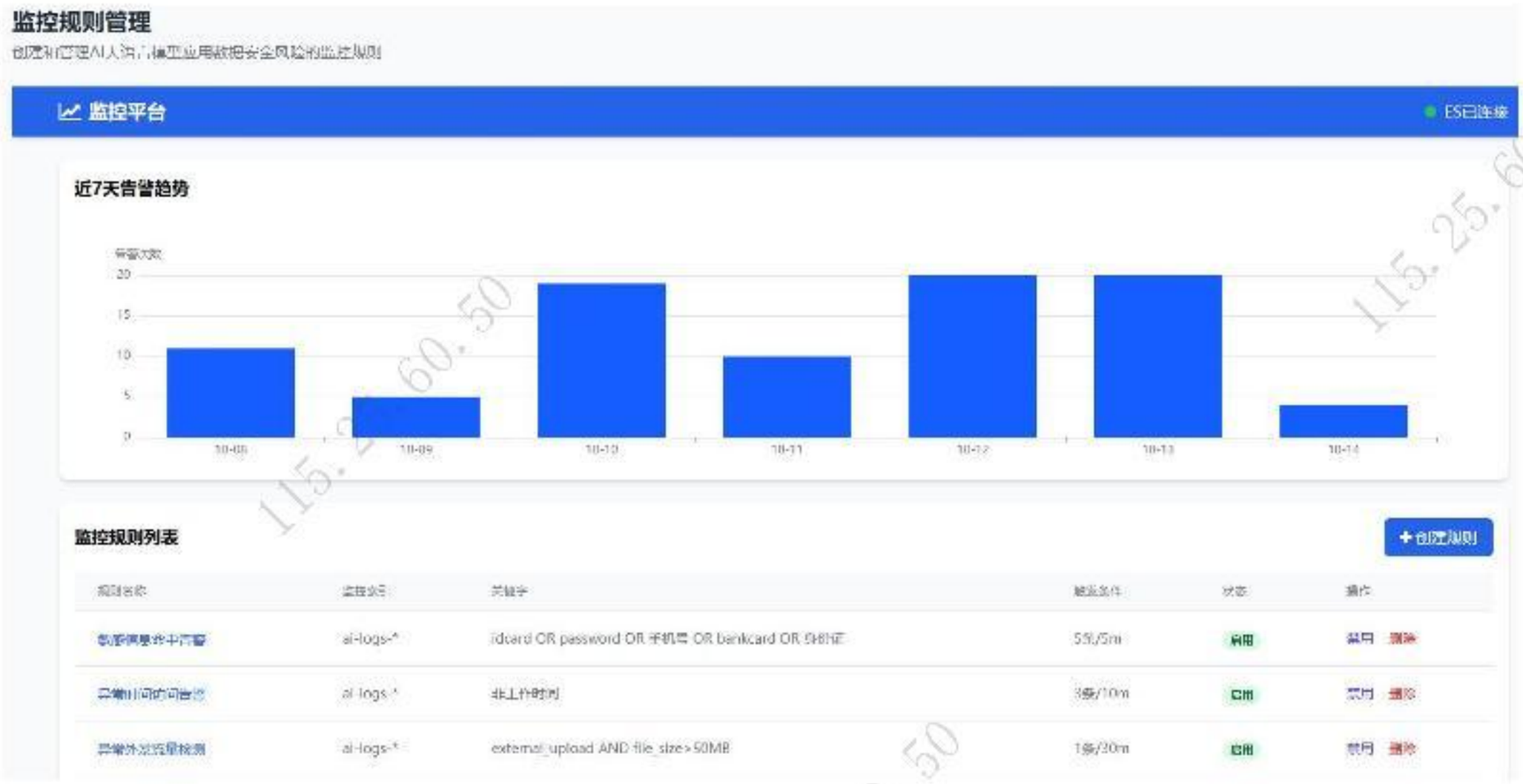


图 5-3 规则管理列表

Fig.5-3 List of rule management

（2）服务层

服务层作为本平台的核心业务枢纽，承担着处理业务逻辑、协调不同模块间通信与保障系统稳定可靠运行的关键职责。为实现高内聚、低耦合的架构目标，本平台的服务层采用 Python 生态中成熟、高效的技术栈进行构建，具体设计与实现如下。

为兼顾开发效率、性能表现与接口规范性，本平台采用 Flask 与 FastAPI 双框架混合的开发模式。Flask 凭借其轻量级、灵活性强和生态丰富的特点，被广泛用于快速构建平台内部的管理界面以及一些对实时性要求不高的传统 Web 服务。其简洁的设计哲学使得本监控平台管理页面的开发与维护成本显著降低。

对于需要高性能、强类型检查和自动 API 文档生成的核心数据接口与异步任务接口，如接受来自终端流量采集代理 agent 的状态日志，本平台选用 FastAPI 作为主要框架。FastAPI 基于 Python 类型提示（Type Hints）原生构建，能够自动生成 OpenAPI 标准文档，极大地便利了前后端联调与外部系统集成。更重要的是，其异步支持能力能够有效提升 I/O 密集型接口的并发处理能力。

为确保系统安全性与数据一致性，本平台在多个层面实施了严格的数据校验。在 FastAPI 服务中，集成 Pydantic 模型进行请求与响应数据的校验。Pydantic 利用类型提示在运行时强制执行数据类型和约束，自动验证传入数据的有效性，从源头杜绝了非法数据的注入，显著增强了 API 的安全

性。

系统的可观测性是进行故障排查、性能分析和业务审计的重要保障。本平台采用 Python 原生的 logging 库作为统一的日志记录方案。通过为不同服务模块配置统一的日志格式、级别（如 DEBUG, INFO, WARNING, ERROR）和输出目标（如文件、控制台），构建了一套完整的日志收集体系。该体系能够清晰记录服务运行过程中的关键事件、错误信息及用户操作行为，为系统的长期稳定运行与后期运维分析提供了坚实的数据支持。

（3）基础层

基础支撑层为整个平台提供数据持久化、实时监控与高吞吐量数据缓冲等底层核心能力，是保障系统可靠性、可观测性与可扩展性的基石。本平台的基础层采用了一系列业界成熟的开源技术栈，构建了一个稳定、高效且易于维护的技术底座。

为应对平台内结构化与非结构化数据的不同特性与访问需求，本平台设计了混合式数据存储方案。

针对结构化数据存储：对于用户信息、系统配置、安全规则等具有固定模式且关系严谨的核心业务数据，选用 MariaDB 关系型数据库进行存储。MariaDB 作为 MySQL 的兼容分支，提供了 ACID 事务支持、数据强一致性和强大的 SQL 查询功能，确保了核心业务数据的准确性与完整性。其稳定成熟的特性为平台的数据管理提供了可靠保障。

针对非结构化日志存储：对于海量的、半结构化的 HTTP 流量日志等安全数据，采用 Elasticsearch 分布式搜索引擎构建日志存储与分析集群。Elasticsearch 具备强大的全文检索、近实时查询和复杂的聚合分析能力，能够高效地支持安全分析人员进行历史日志追溯、模式发现与威胁狩猎等高级操作。其分布式架构也赋予了存储系统水平扩展的能力，以应对数据量的持续增长。

为解决前端数据采集端与后端数据处理端之间的速率不匹配问题，并防止流量洪峰导致的数据丢失，本平台引入了 Kafka 作为高吞吐量的分布式消息队列。Kafka 充当了平台的数据总线。所有从终端设备采集的原始流量日志通过日志流量采集网关后，首先被写入 Kafka 进行缓存。这一设计实现了数据的异步处理与解耦，后端消费服务（如日志解析、入库服务）按照自身处理能力从 Kafka 中拉取数据，从而平滑了数据流，有效避免了系统在突发流量下的过载风险。Kafka 的高持久化、高可靠性和可重放特性，不仅保证了数据在入库前“至少一次”的可靠传递，也为后续可能的流式

实时分析任务提供了数据源支持。

监控平台流量日志核心架构如下图 5-4 所示。

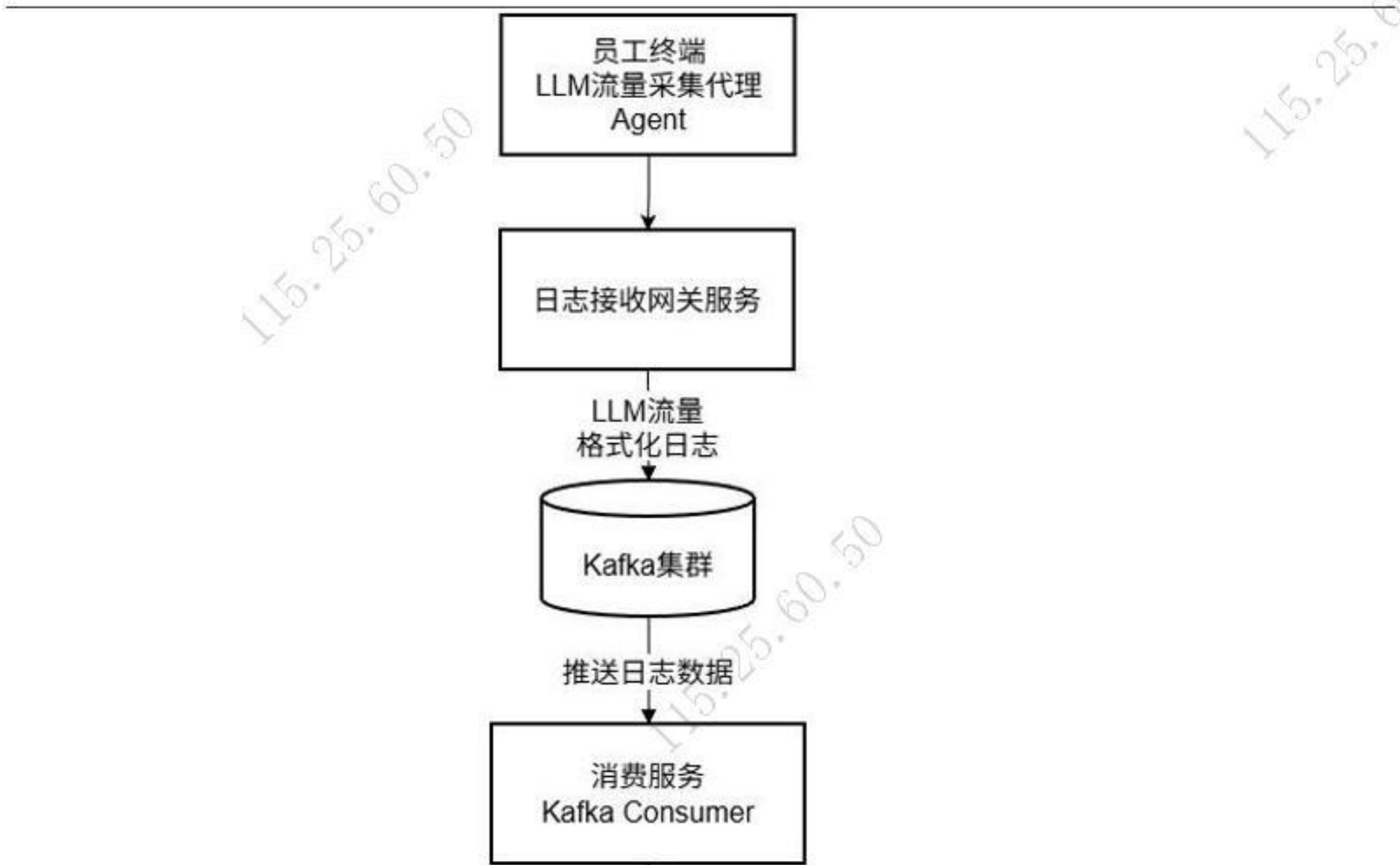


图 5-4 流量日志采集存储架构

Fig.5-4 Architecture of flow log collection and storage

为确保平台自身运行状态的透明可知，构建了多层次、一体化的监控告警体系。该体系由 Zabbix、Prometheus 与 Grafana 协同构成。

数据采集与汇聚使用 Prometheus 和 zabbix，其中 Prometheus 主要负责从平台的各种微服务、Kafka 集群、Elasticsearch 集群中拉取并存储时间序列指标数据，如 API 接口 QPS、服务响应延迟、系统资源利用率等。其灵活的拉取模型和强大的多维数据模型非常适合云原生环境的监控。Zabbix 作为企业级监控解决方案，更侧重于对服务器基础设施（如 CPU、内存、磁盘 I/O）和网络设备的监控，并通过其主动 Agent 模式补充监控覆盖范围。

数据可视化与告警使用 Grafana，其作为统一的监控数据可视化平台，Grafana 无缝对接 Prometheus 和 Zabbix 等多个数据源。基于其强大的仪表盘定制能力，运维与安全人员可以直观地查看平台各项关键性能指标（KPIs）与业务数据的实时状态与历史趋势，实现了监控信息的集中化与可视化展示。同时，利用 Grafana 的告警规则配置功能，实现基于资源阈值的监控平台自动化监控告警。

5.1.2 开发平台与环境搭建

本节详细描述开发环境和系统的整体部署架构，涵盖开发过程中使用的技术平台、编程工具以及生产环境的基础设施配置。

（1）开发平台

本项目的主要开发工作基于 Windows 操作系统环境展开。其具有良好的软件兼容性、成熟的系统架构以及完善的开发者工具链，为项目提供了稳定且高效的开发体验。在集成开发环境（IDE）方面，选用 Visual Studio Code 作为核心开发工具，其轻量级的设计、丰富的扩展生态系统以及对 Python 语言的深度支持，显著提升了代码编写与调试的效率。

在团队协作与代码质量管理方面，项目采用 Git 作为分布式版本控制系统，以实现对代码变更的精确追踪与管理。结合基于 GitLab 搭建的内部代码托管平台，团队能够高效开展分支管理、代码审查及持续集成等协作流程。此外，为进一步保障代码质量与安全性，项目集成 SonarQube 静态代码分析工具，对代码库进行扫描，识别潜在的程序错误及安全漏洞，从而在开发早期阶段有效控制技术风险，提升监控平台的可靠性与可维护性。

（2）生产环境

在生产环境部署中，为保障系统的高可用性与计算性能，本系统其硬件基础设施由四台搭载 CentOS 操作系统的物理服务器构成。该服务器集群的每台服务器均配备 64 核心的高性能中央处理器，并搭载 128GB 内存，以此确保处理高并发请求与执行复杂业务逻辑时具备充足的算力资源与内存带宽。同时，集群配备了总计 6TB 的存储空间，为系统日志、LLM 应用流量日志提供存储支持。

为满足 Qwen2 7B SLM 模型的计算需求，集群中的两台服务器额外配置了 NVIDIA GeForce RTX 4080（16GB GDDR6 显存）显卡。此专用 GPU 节点主要负责承载参数量为 70 亿的 Qwen2 7B 语言模型，通过其强大的单精度与半精度浮点运算能力，实现对模型推理任务的高效、低延迟执行。

5.2 终端流量采集模块

终端流量采集模块作为整个监控平台的数据感知层，其设计遵循高效性与低侵入性两大核心原则，模块架构如图 5-5 所示。模块需要在不影响终端正常业务通信的前提下，从网络流量中精准捕获与大模型应用相关的通信数据，并为后端智能分析提供高质量的预处理数据。

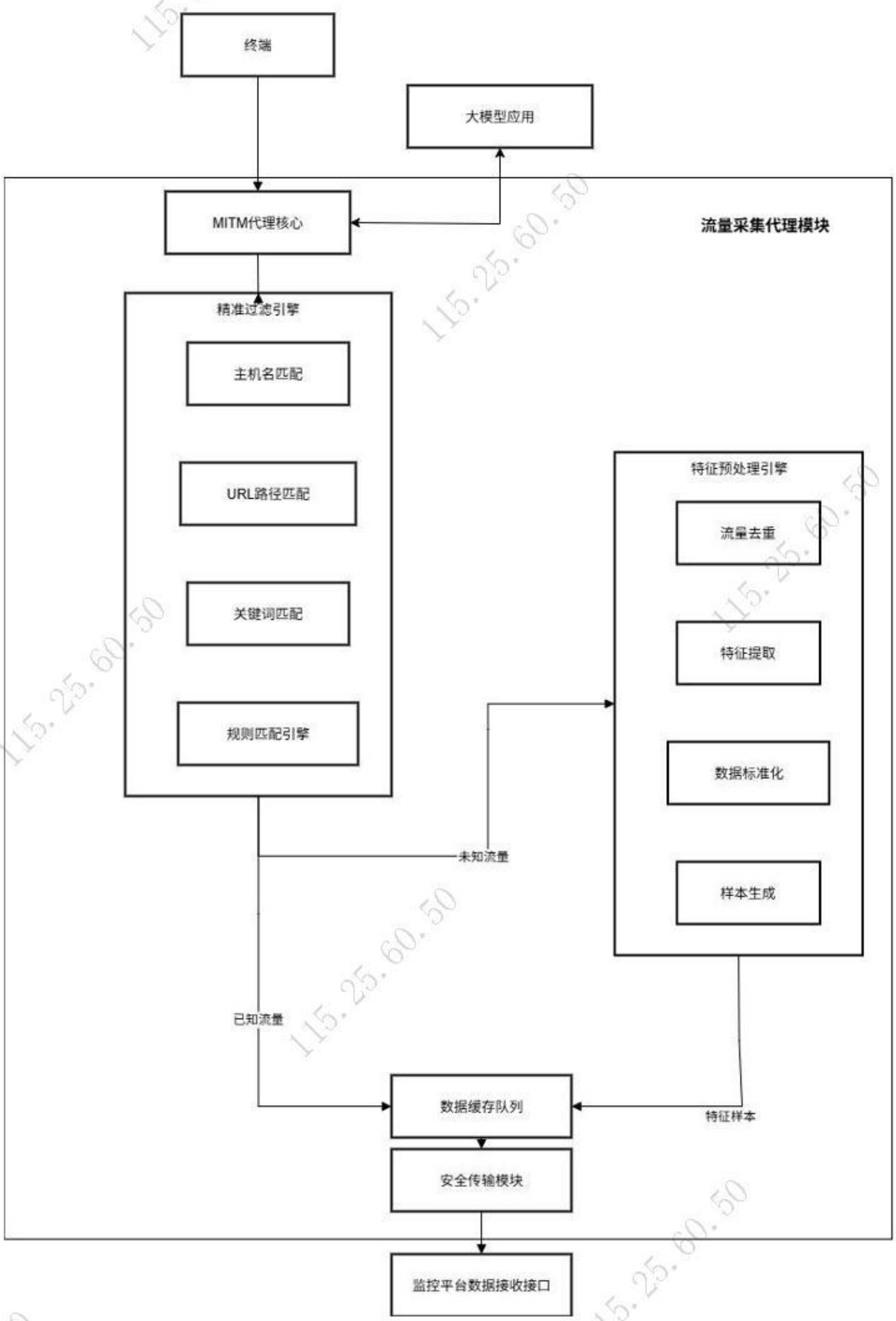


图 5-5 流量采集代理模块架构

Fig.5-5 Architecture of flow collection agent

中间人代理技术是本模块的基础支撑技术，通过在通信路径中插入透明代理节点，实现对网络流量的全面捕获。关键技术实现如下：

（1）HTTPS 流量解密：基于 MITM proxy 技术，如图 5-6 所示使用 Python mitmproxy 库作为中间人代理，在终端安装并信任根证书后，实现 TLS 解密，Agent 以非侵入方式工作，不改造上层应用，仅插入网络路径。

```python
class AgentAddon:
    def __init__(self):
        self.loader = RuleLoader(RULES_PATH)
        self.dedupe = Deduplicator(ttl_seconds=300)
        self.uploader = Uploader(UPLOAD_ENDPOINT, api_token=API_TOKEN)

    def request(self, flow: http.HTTPFlow) -> None:
        # inspect request
        try:
            host = flow.request.host
            path = flow.request.path
            method = flow.request.method
            body = flow.request.raw_content
            rule = self.loader.match(host, path, method, body)
            meta = {
                "src_ip": flow.client_conn.address[0] if flow.client_conn and flow.client_conn.address else None,
                "src_port": flow.client_conn.address[1] if flow.client_conn and flow.client_conn.address else None,
                "dst_ip": flow.server_conn.ip_address[0] if flow.server_conn and flow.server_conn.ip_address else flow.server_conn.address[0] if flow.server
                "dst_port": flow.server_conn.ip_address[1] if flow.server_conn and flow.server_conn.ip_address else flow.server_conn.address[1] if flow.serv
                "protocol": flow.request.scheme,
                "method": method,
                "path": path,
                "url": flow.request.url,
            }
```

图 5-6 流量代理解密代码

Fig.5-6 Code of flow proxy and decrypt

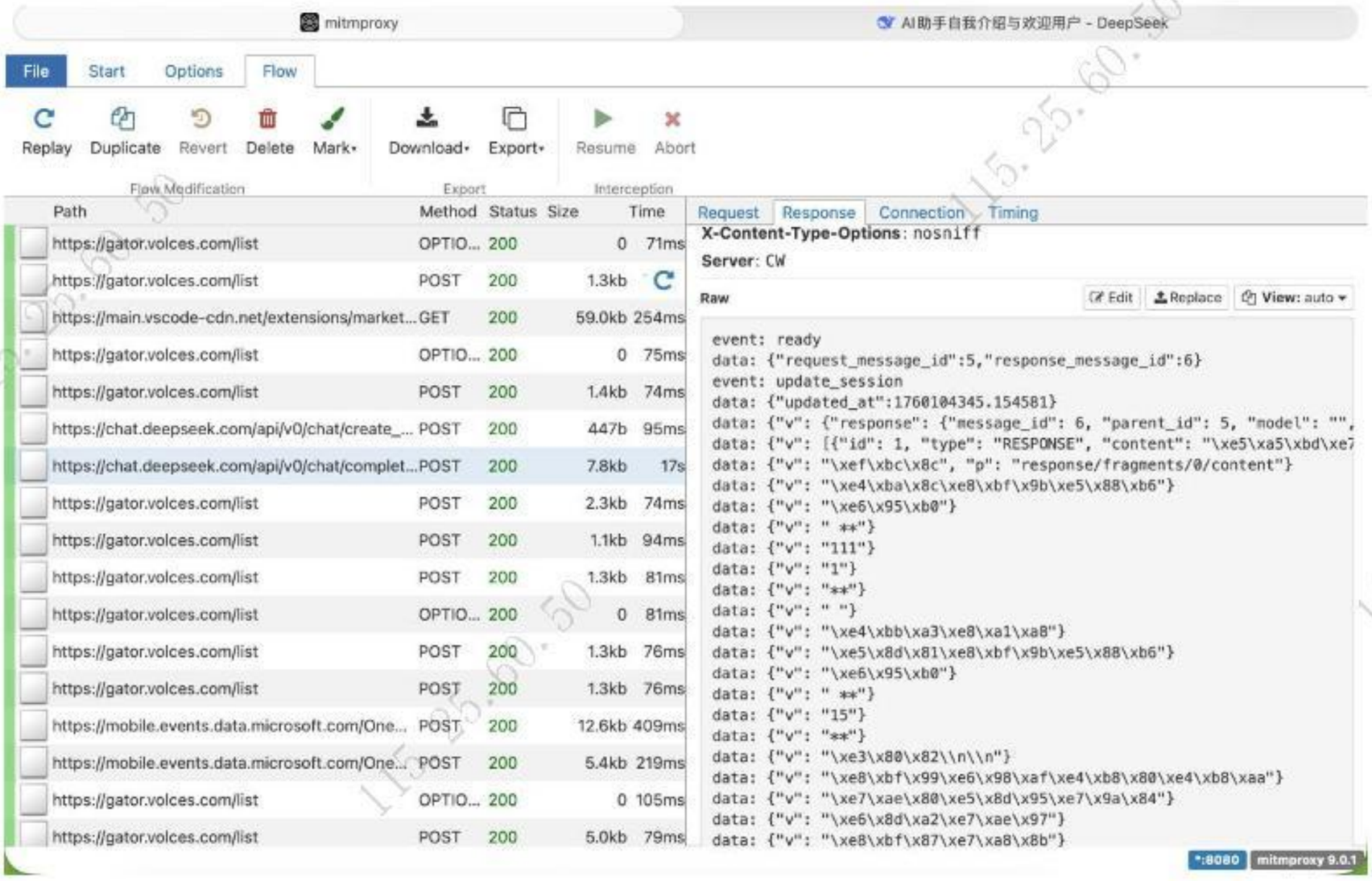


图 5-7 流量中间人代理截取

Fig.5-7 MITM traffic interception

（2）精准过滤引擎：如图 5-8、图 5-9 通过预置规则和监控平台下发的特征进行匹配，包括域名匹配，识别已知大模型服务域名；URL 路径匹配，匹配大模型应用 API 端点特征路径；关键词匹配，检测请求内容中的

大模型相关关键字，如 model、stream、tokens、temperature 等[63]。

```python
class PrecisionFilterEngine:
    def _load_rules_config(self, config_path: str) -> None:
        """
        加载规则配置文件
        支持动态规则更新
        """
        try:
            with open(config_path, 'r', encoding='utf-8') as f:
                config = json.load(f)

            # 解析主机名规则
            for i, host in enumerate(config.get('known_hosts', [])):
                self.rules.append(MatchingRule(
                    rule_id=f"host_{i}",
                    rule_type='host',
                    pattern=host.lower(),
                    description=f"已知大模型服务域名: {host}"
                ))

            # 解析路径规则
            for i, path in enumerate(config.get('known_paths', [])):
                self.rules.append(MatchingRule(
                    rule_id=f"path_{i}",
                    rule_type='path',
                    pattern=path.lower(),
                    description=f"大模型API路径: {path}"
                ))

            # 解析关键词规则
            for i, keyword in enumerate(config.get('keywords', [])):
                self.rules.append(MatchingRule(
                    rule_id=f"keyword_{i}",
                    rule_type='keyword',
                    pattern=keyword.lower(),
                    description=f"大模型关键词: {keyword}"
                ))

            # 解析正则表达式规则
            for i, regex_pattern in enumerate(config.get('regex_patterns', [])):
                self.rules.append(MatchingRule(
                    rule_id=f"regex_{i}",
                    rule_type='regex',
                    pattern=regex_pattern,
                    description=f"高级匹配模式: {regex_pattern}"
                ))

            self.logger.info(f"成功加载 {len(self.rules)} 条过滤规则")
```

图 5-8 精准匹配引擎

Fig.5-8 Engine of precise matching

```json
{
  "llm_feature_rules": [
    {
      "app_name": "OpenAI API",
      "description": "OpenAI官方API接口",
      "domains": ["api.openai.com", "api.openai.azure.com"],
      "ip_ranges": [],
      "api_endpoints": [
        "/v1/chat/completions",
        "/v1/completions",
        "/v1/embeddings",
        "/v1/models",
        "/v1/moderations"
      ],
      "request_keywords": [
        "model",
        "messages",
        "temperature",
        "stream",
        "max_tokens",
        "top_p",
        "frequency_penalty",
        "presence_penalty"
      ],
      "response_keywords": [
        "choices",
        "usage",
        "finish_reason",
        "completion_tokens",
        "prompt_tokens",
        "total_tokens"
      ],
      "headers_patterns": {
        "authorization": "Bearer sk-*",
        "content-type": "application/json"
      },
      "is_streaming": true,
      "is_websocket": false,
      "confidence_level": 0.95
    },
    {
      "app_name": "Claude API",
      "description": "Anthropic Claude API接口",
      "domains": ["api.anthropic.com"],
      "ip_ranges": [],
```

图 5-9 预置大模型流量特征

Fig.5-9 Pre-configure LLM traffic patterns

（3）特征预处理引擎：如图 5-10 所示，首先进行流量去重，基于五元组和 API 端点的重复流量检测；其次进行特征提取：从未知应用流量中提取包含可疑特征关键字的五元组、API 端点及请求体内容；然后将特征数据标准化，统一数据格式便于后续处理；最后，生成标准化特征样本，将其传输至监控平台进行特征分析和发现。

```python
class LLMTrafficCollector:

    def response(self, flow: mitmproxy.http.HTTPFlow) -> None:
        """
        MITM代理响应拦截处理
        补充响应特征匹配
        """
        try:
            # 仅处理之前标记为未知但可能包含LLM响应的流量
            if hasattr(flow, 'is_unknown_traffic') and flow.is_unknown_traffic:
                response_confidence = self._check_response_for_llm_patterns(flow)
                if response_confidence > 0.6:  # 响应匹配阈值
                    # 发现新的LLM特征，创建临时规则并记录
                    self._handle_new_llm_traffic(flow, response_confidence)

        except Exception as e:
            ctx.log.error(f"响应处理错误: {str(e)}")

    def _calculate_flow_fingerprint(self, flow: mitmproxy.http.HTTPFlow) -> str:
        """计算流量指纹用于去重"""
        fingerprint_data = {
            'method': flow.request.method,
            'host': flow.request.pretty_host,
            'path': flow.request.path,
            'headers_hash': hashlib.md5(
                str(sorted(flow.request.headers.items())).encode()
            ).hexdigest()[:8],
            'body_hash': hashlib.md5(
                (flow.request.get_text() or '').encode()
            ).hexdigest()[:8]
        }

        fingerprint_str = json.dumps(fingerprint_data, sort_keys=True)
        return hashlib.md5(fingerprint_str.encode()).hexdigest()

    def _is_duplicate_flow(self, fingerprint: str) -> bool:
        """检查流量是否重复"""
        if fingerprint in self.flow_deduplication_cache:
            return True

        # 添加到去重缓存（带过期时间）
        self.flow_deduplication_cache.add(fingerprint)
        # 简单实现：定时清理过期缓存
        if len(self.flow_deduplication_cache) > 10000:
            self.flow_deduplication_cache.clear()

        return False
```

图 5-10 特征预处理

Fig.5-10 Pre-process of patterns

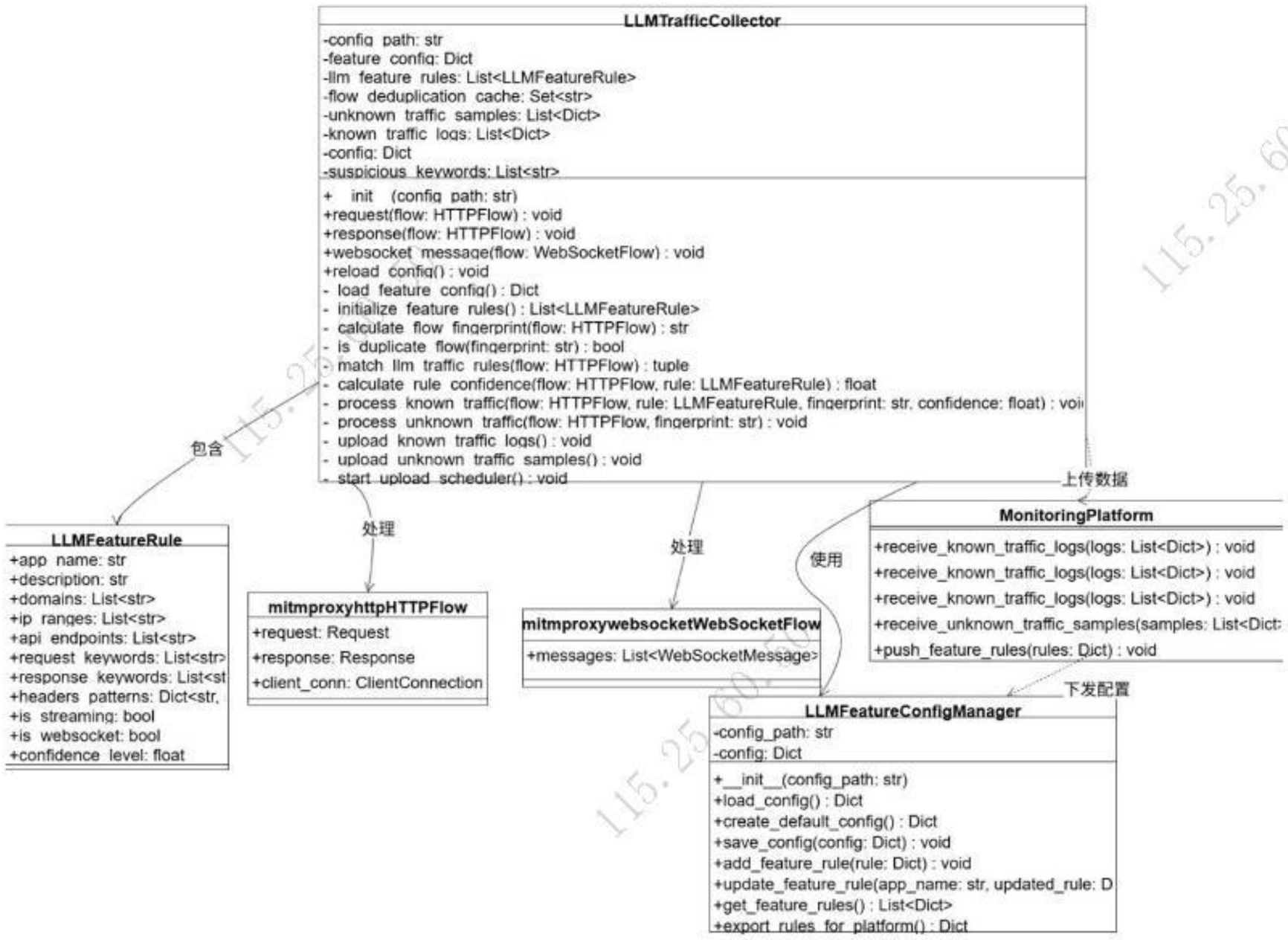


图 5-11 流量采集代理 UML 图

Fig.5-11 UML Design Diagram of a Traffic Collection Agent

（4）实现效果如图 5-12 所示：该模块部署安装于员工终端电脑上，通过导入证书至系统，以中间人代理的方式无损获取流量，并从中截取符合大模型应用特征的流量以及可疑流量至监控中心进行分析和告警。同时，提供状态信息供员工及管理人员进行判断分析，确保模块运行正常。



图 5-12 流量采集代理客户端

Fig.5-12 Agent of the flow collection proxy

5.3 基于 RAG 技术的告警分析

传统告警分析系统面临核心痛点：海量、冗杂、专业化的安全告警与安全运营人员（尤其是初级人员）有限的专业知识和经验之间存在巨大鸿沟。这常常导致告警研判效率低下、误报处置消耗大量资源、真正的高危威胁被淹没在“告警噪音”中。

本文基于 Ollama、qwen32b、maxbai-embed-large、LangChain、RecursiveCharacterTextSplitter 和 Chroma 这一技术组合，构建 RAG 流水线[64]。

5.3.1 知识库构建与向量化存储

多元异构知识采集与预处理，知识来源于企业内部文档库以及外部威胁情报库，其中企业内部文档库，包括 PDF 格式的企业信息安全制度、合规文件、文本格式的历史处置报告、专家经验总结、国家及行业网络安全要求、以及 JSON/CSV 格式的威胁情报 IOC 信息等，如图 5-13、5-14 所示。



图 5-13 数据安全知识库

Fig.5-13 Knowledge base of the data security

```python
from langchain.document_loaders import DirectoryLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.vectorstores import Chroma

# 1. 加载安全文档
loader = DirectoryLoader('./security_docs/', glob="**/*.pdf")
documents = loader.load()
```

图 5-14 从知识文件中加载数据

Fig.5-14 Load data from knowledge file

文本分割能力，使用 RecursiveCharacterTextSplitter 实现。安全文档通常篇幅较长，若整篇都进行处理，则会引入大量无关信息，干扰检索精度。RecursiveCharacterTextSplitter 采用递归的方式，优先按段落、其次按句子、再按换行符等分隔符进行分割，能在尽量保证语义完整性的前提下，将长文本切分成大小适宜的文本块。将参数设置为 chunk_size（如 512 或 1024 个字符）和 chunk_overlap（如 100 个字符）来控制块的大小和重叠区域，确保关键信息（如一个完整的处置步骤）不会因被切断而丢失上下文，如图 5-15 所示。

```
# 2. 文本分割
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=100
)
chunks = text_splitter.split_documents(documents)
```

图 5-15 文本切分

Fig.5-15 Text splitting

文本向量化与存储，采用 maxbai-embed-large 嵌入模型。该模型在中文语义表示上表现优异，能精准地将安全领域的专业术语和描述映射为高维空间中的向量，从而捕捉到“数据泄露”和“敏感信息外泄”等不同表述之间的语义相似性。

如图 5-16 使用 Chroma 向量数据库，一个轻量级且高性能的向量数据库。将上一步得到的所有文本块，通过 maxbai-embed-large 模型转换为向量（Embeddings），并持久化存储到 Chroma 中。Chroma 会为这些向量建立索引，支持高效的近似最近邻搜索，为后续的实时检索提供支撑[65]。

```
# 3. 向量化存储
embeddings = HuggingFaceEmbeddings(model_name="maxbai-embed-large")
vector_db = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db"
)
```

图 5-16 文本向量化与存储

Fig.5-16 Text embedding and storage

5.3.2 告警触发与智能检索

当平台接收到一条新的安全告警（例如：“检测到员工 A 在非工作时

间向大模型应用上传大量客户数据”）时，RAG 分析流程随即启动，如图 5-17 所示。

告警信息向量化，将原始的告警文本描述作为查询 Query，使用与知识库相同的 maxbai-embed-large 模型，将其转换为一个查询向量。

语义相似度检索，在 LangChain 的框架下，向 Chroma 向量数据库发起查询。Chroma 通过计算查询向量与库中所有向量之间的余弦相似度，快速找出最相关的 K 个（如 top-3）知识片段。

```python
def retrieve_related_knowledge(alert_message, vector_db, top_k=3):
    # 1. 告警信息向量化并检索
    related_docs = vector_db.similarity_search(alert_message, k=top_k)

    # 2. 构建检索上下文
    context = "\n\n".join([doc.page_content for doc in related_docs])
    return context
```

图 5-17 向量库检索

Fig.5-17 Vector database retrieval

提示词工程与上下文构建，利用 LangChain 的提示模板功能，设计一个结构化的系统提示词。如图 5-18 所示，该提示词明确界定了大模型的角色（一位资深安全专家）、任务（分析告警并提供建议）和输出格式。

将第二步中检索到的多个相关知识片段，与原始的告警信息一起，整合填充到提示词的指定位置，形成一个富含上下文信息的完整提示。

```
    # 1. 构建提示词模板
    prompt_template = PromptTemplate(
        template="""
你是一名资深的企业安全运营中心（SOC）专家，负责识别和应对潜在的数据泄露与合规风险。请根据以下背景知识、当前告警信息以及用户行为上下文，
对用户是否涉嫌上传敏感信息至LLM大模型应用进行综合研判，并结合企业内部制度与外部法规要求，提出分析结论与处置建议。

【相关背景知识】
知识片段1：{retrieved_chunk_1}
知识片段2：{retrieved_chunk_2}
知识片段3：{retrieved_chunk_3}

【当前告警信息】
{alert_message}

【用户行为与环境上下文】
用户身份：{user_type}
访问时间：{access_time}（是否非工作时间？）
访问目标大语言模型：{llm_name}（是否已知风险网站？）
涉及数据类型：（如：个人身份信息 PII、财务数据、源代码、商业机密等）
合规要求参考：知识片段4：{retrieved_chunk_4}

【请按以下结构输出】
一、研判分析
结合背景知识与告警信息，判断该行为是否构成敏感信息外传。
分析用户行为是否违反企业内部制度或外部法规。
评估是否存在误报可能性（如：授权业务操作、误操作等）。

二、关联风险
数据泄露影响范围（如：影响客户隐私、造成商业损失、引发法律追责等）。
该行为是否属于已知威胁模式或内部高风险行为清单。
是否涉及第三方或供应链风险。

三、处置建议（需具体可操作）
立即措施：如阻断访问、暂停账户、留存日志证据等。
调查建议：如进一步检索该用户历史行为、联系其主管核实业务合理性。
合规与上报：如是否需启动事件响应流程、通报DPO、数据保护官、或法务团队。
长期改进建议：如加强员工培训、调整DLP策略、增强监控规则等。---
```

图 5-18 Prompt 提示词组装

Fig.5-18 Prompt assembly

如图 5-19，将组合后的 Prompt 信息放入 LLM 中进行交互，获取输出。

```
# 填充提示词
prompt = prompt_template.format(
    context=context,
    alert_message=alert_message
)

# 调用大模型生成
llm = Ollama(model="qwen32b", temperature=0.3)
response = llm(prompt)

return response
```

图 5-19 与 LLM 交互获取分析结果

Fig.5-19 interact with LLM to obtain analysis results

测试验证结果，如图 5-20、图 5-21 所示。

```
[2025-03-15 09:30:02] Info 一、研判分析
[2025-03-15 09:30:03] Info 结合背景知识与告警信息，判断该行为是否构成敏感信息外传：
[2025-03-15 09:30:04] Info   根据知识片段1和当前告警信息，用户行为模式与已知的数据泄露特征高度匹配。
[2025-03-15 09:30:05] Info - 涉及数据类型包括个人身份信息（PII）和商业机密，属于企业定义的敏感信息范畴。
[2025-03-15 09:30:06] Info
[2025-03-15 09:30:07] Info 分析用户行为是否违反企业内部制度或外部法规：
[2025-03-15 09:30:08] Info - 该行为违反《企业数据安全管理制度》第3.2条禁止未经授权向外部平台传输敏感数据。
[2025-03-15 09:30:09] Info - 同时可能违反外部法规：知识片段4中提到的GDPR第32条（数据处理安全要求）。
[2025-03-15 09:30:10] Info
[2025-03-15 09:30:11] Info 评估是否存在误报可能性：
[2025-03-15 09:30:12] Info - 非工作时间访问和未知LLM应用的使用降低了误报概率。
[2025-03-15 09:30:13] Info - 但需进一步排查是否存在授权研发测试或试操作可能性。
[2025-03-15 09:30:14] Info
[2025-03-15 09:30:15] Info 二、关联风险
[2025-03-15 09:30:16] Info 数据泄露影响范围：
[2025-03-15 09:30:17] Info - 潜在影响：约5000条客户PII泄露，可能导致监管罚款（最高占年营收4%）。
[2025-03-15 09:30:18] Info - 商业机密泄露可能被竞争对手利用，造成战略损失。
[2025-03-15 09:30:19] Info
[2025-03-15 09:30:20] Info 威胁模式匹配：
[2025-03-15 09:30:21] Info - 该行为匹配内部高风险行为清单第7类非授权数据外传。
[2025-03-15 09:30:22] Info - 与已知威胁模式TTP-2023-LLM-01相似度85%。
[2025-03-15 09:30:23] Info
[2025-03-15 09:30:24] Info 第三方风险：
[2025-03-15 09:30:25] Info - 使用的LLM应用（ChatGPT）为第三方服务，其数据保留政策可能不符合企业合规要求。
[2025-03-15 09:30:26] Info
[2025-03-15 09:30:27] Info 三、处置建议
[2025-03-15 09:30:28] Info 立即措施：
[2025-03-15 09:30:29] Info - 1. 立即阻断该用户对LLM应用的所有访问
[2025-03-15 09:30:30] Info - 2. 暂停用户账户并保存所有会话日志
[2025-03-15 09:30:31] Info - 3. 隔离可能受影响的数据存储区域
[2025-03-15 09:30:32] Info
[2025-03-15 09:30:33] Info 调查建议：
[2025-03-15 09:30:34] Info - 1. 检索该用户30天内所有外部服务访问记录
[2025-03-15 09:30:35] Info - 2. 联系部门主管确认是否存在业务需求
[2025-03-15 09:30:36] Info - 3. 分析剪贴板历史和文件操作记录
[2025-03-15 09:30:37] Info
[2025-03-15 09:30:38] Info 合规与上报：
[2025-03-15 09:30:39] Info - 1. 立即启动安全事件响应流程（IRP-002）
[2025-03-15 09:30:40] Info - 2. 24小时内通报DPO和法务团队
[2025-03-15 09:30:41] Info   3. 根据合规要求决定是否上报监管机构
[2025-03-15 09:30:42] Info
[2025-03-15 09:30:43] Info 长期改进建议：
[2025-03-15 09:30:44] Info   1. 在DLP策略中添加LLM应用检测规则
```

图 5-20 测试验证结果

Fig.5-20 testing result



图 5-21 分析验证结果

Fig.5-21 result of llm analysis

5.4 大模型应用流量特征发现模块

5.4.1 LLM 应用流量特征自发现算法

算法的核心创新在于将复杂的特征抽取、语义建模、分类决策等多个任务统一到一个简洁的 SLM 推理框架中，充分利用预训练语言模型已有的语义理解能力和推理能力，显著降低了系统复杂度，同时保证了识别准确率。如图 5-22 为算法的整体架构。

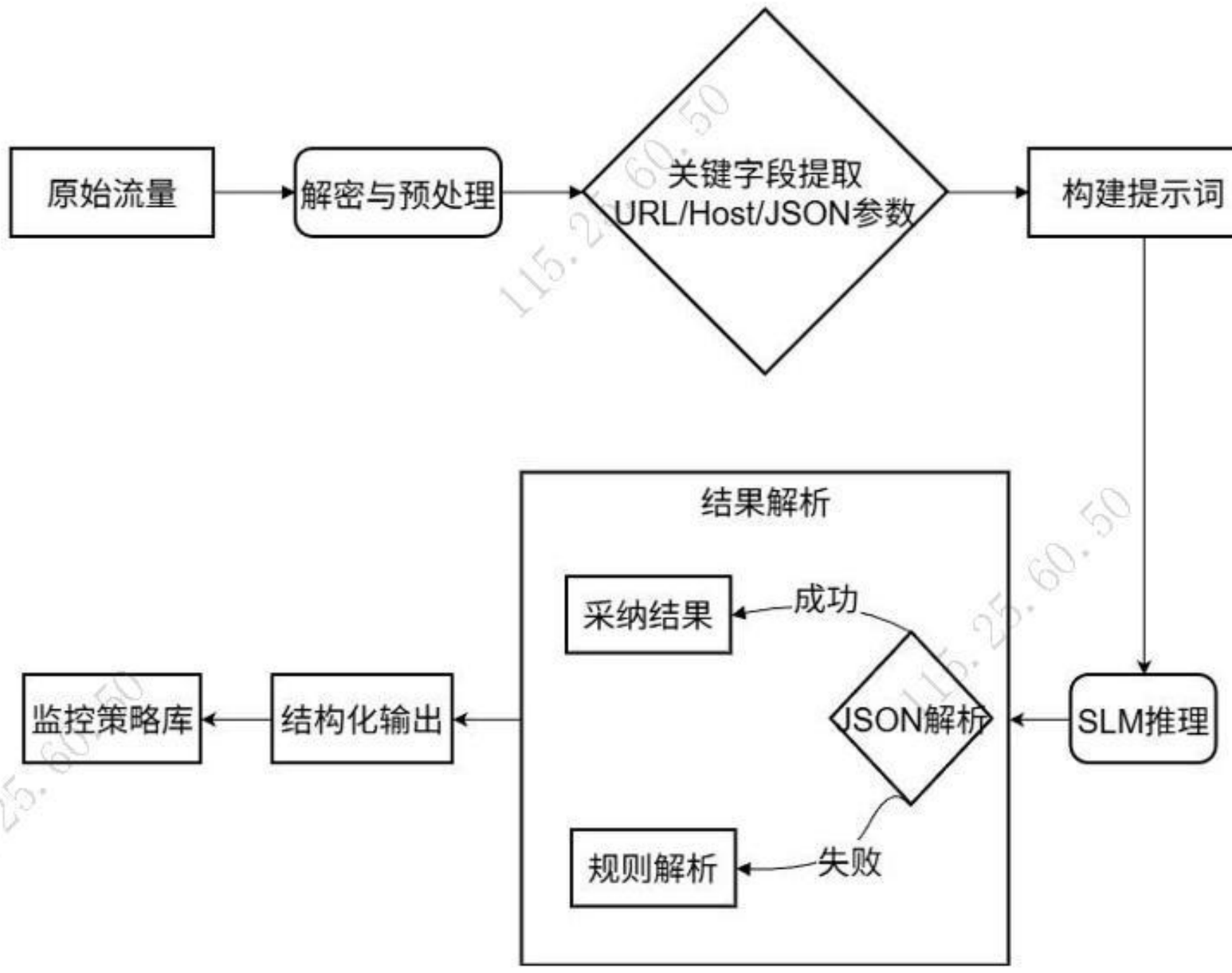


图 5-22 LLM 应用流量特征发现分析架构

Fig.5-22 LLM app traffic feature discovery and analysis architecture

（1）流量数据采集与预处理

```
类 TrafficPreprocessor:
    常量 LLM_PARAM_KNOWLEDGE = {
        'request_params': ['model', 'messages', ...],
        'response_fields': ['id', 'object', ...]}
    方法 preprocess(原始流量数据):
        // 预处理原始流量并返回清洗后的数据
        url 路径 = 调用_extract_url_path(原始流量数据['url'])
        host 域名 = 原始流量数据['host']
        请求参数 = 调用_extract_llm_parameters(
```

```
        原始流量数据['request_body'],
        LLM_PARAM_KNOWLEDGE['request_params'])
响应字段 = 调用_extract_llm_parameters(
        原始流量数据['response_body'],
        LLM_PARAM_KNOWLEDGE['response_fields'])
返回 ProcessedTraffic(
    url = url 路径,
        host = host 域名,
        request_params = 请求参数,
        response_fields = 响应字段)
私有方法 _extract_llm_parameters(json 文本, 关键字段列表):
    // 从 JSON 中提取指定字段，并进行格式化处理
    尝试:
        data = 解析 json 文本
        已提取列表 = []
        遍历 关键字段列表 中的每个字段 key:
            如果 key 存在于 data 中:
                值 = 转换为字符串(data[key])
                如果 值长度 > 100:
                    值 = 值前 100 字符 + "..."
                添加 "key:值" 到已提取列表
        如果 已提取列表非空:
            返回 用" | "连接的已提取列表
        否则:
            返回 "无相关参数"
    捕获 JSON 解析异常:
        返回 "JSON 解析失败"
```

（2）基于 SLM 语义理解分析 LLM 流量特征伪代码

```
类 SLMTrafficAnalyzer:
    // 基于 SLM 的流量分析器
    方法 __init__(模型路径):
        tokenizer = 加载 tokenizer(模型路径)
        model = 加载模型(模型路径)
        prompt_manager = PromptTemplateManager()
    方法 analyze_traffic(processed_traffic):
        // 分析处理后的流量数据构建分析提示词
        prompt =
prompt_manager.APP_IDENTIFICATION_TEMPLATE.format(
            url = processed_traffic.url,
```

```
            host = processed_traffic.host,
            request_params = processed_traffic.request_params,
            response_fields = processed_traffic.response_fields)
    // SLM 推理
    raw_response = 调用_query_slm(prompt)
    // 解析结构化结果
    structured_result = 调用_parse_analysis_result(raw_response)
    返回 AnalysisResult(
            raw_response = raw_response,
            structured_result = structured_result,
            confidence = 调用
_evaluate_confidence(structured_result))
私有方法 _query_slm(prompt, max_length=500):
    // 查询 SLM 获取分析结果
    inputs = tokenizer 编码(prompt)
    outputs = model 生成(
            input_ids = inputs.input_ids,
            max_length = 输入长度 + max_length,
            temperature = 0,
            do_sample = True,
            pad_token_id = tokenizer.eos_token_id,
            eos_token_id = tokenizer.eos_token_id )
    response = tokenizer 解码(outputs[0])
    // 提取模型生成部分（去除提示词）
    返回 response[prompt 长度:].去除首尾空格()
私有方法 _parse_analysis_result(raw_response):
    // 解析 SLM 的原始响应为结构化数据
    尝试:
        // 首先尝试解析 JSON 格式
        如果 raw_response.去除首尾空格().以'{'开头:
            返回 json 解析(raw_response)
    捕获异常:
        // 备用方案：基于规则的文本解析
    返回 调用_rule_based_parsing(raw_response)
私有方法 _rule_based_parsing(text):
    // 基于规则的文本解析兜底方案
    result = {
        "service_provider": "未知",
        "api_type": "未知",
        "confidence": "中",
```

```
        "key_parameters": []}
    // 服务商识别规则
    provider_patterns = {
        "OpenAI": ["openai", "api.openai.com"],
        "Azure OpenAI": ["azure", "openai.azure.com"],
        "Anthropic": ["anthropic", "claude", "api.anthropic.com"],
        "国内厂商 A": ["company-a", "a.com/api"]
    }
    遍历 provider_patterns 中的 (provider, keywords):
        如果 任意 keyword 在 text.lower() 中:
            result["service_provider"] = provider
            跳出循环
    返回 result
```

（3）LLM 应用流量特征结构化输出

```
类 StructuredOutputGenerator:
    方法 generate(analysis_result, confidence, processed_traffic):
        structured_result = analysis_result.structured_result
        返回 {
            "metadata": {
                "analysis_id": 调用_generate_analysis_id(),
                "timestamp": 当前时间.isoformat(),
                "confidence_score": confidence,
                "analysis_version": "1.0"
            },
            "identification": {
                "service_provider":
structured_result.get("service_provider", "未知"),
                "api_function": structured_result.get("api_type", "未
知"),
                "traffic_category": 调用
_categorize_traffic(structured_result),
                "is_llm_traffic": 调用
_is_llm_traffic(structured_result)
            },
            "technical_details": {
                "api_endpoint": processed_traffic.url,
                "domain": processed_traffic.host,
                "detected_parameters":
structured_result.get("key_parameters", []),
```

```
                    "parameter_patterns": 调用
_extract_parameter_patterns(processed_traffic)
                },
                "monitoring_metadata": {
                    "suggested_monitoring_rules": 调用
_generate_monitoring_rules(structured_result),
                    "risk_level": 调用
_assess_risk_level(structured_result),
                    "update_strategy": "auto" if confidence > 0.7 else
"manual_review"
                }
            }
```

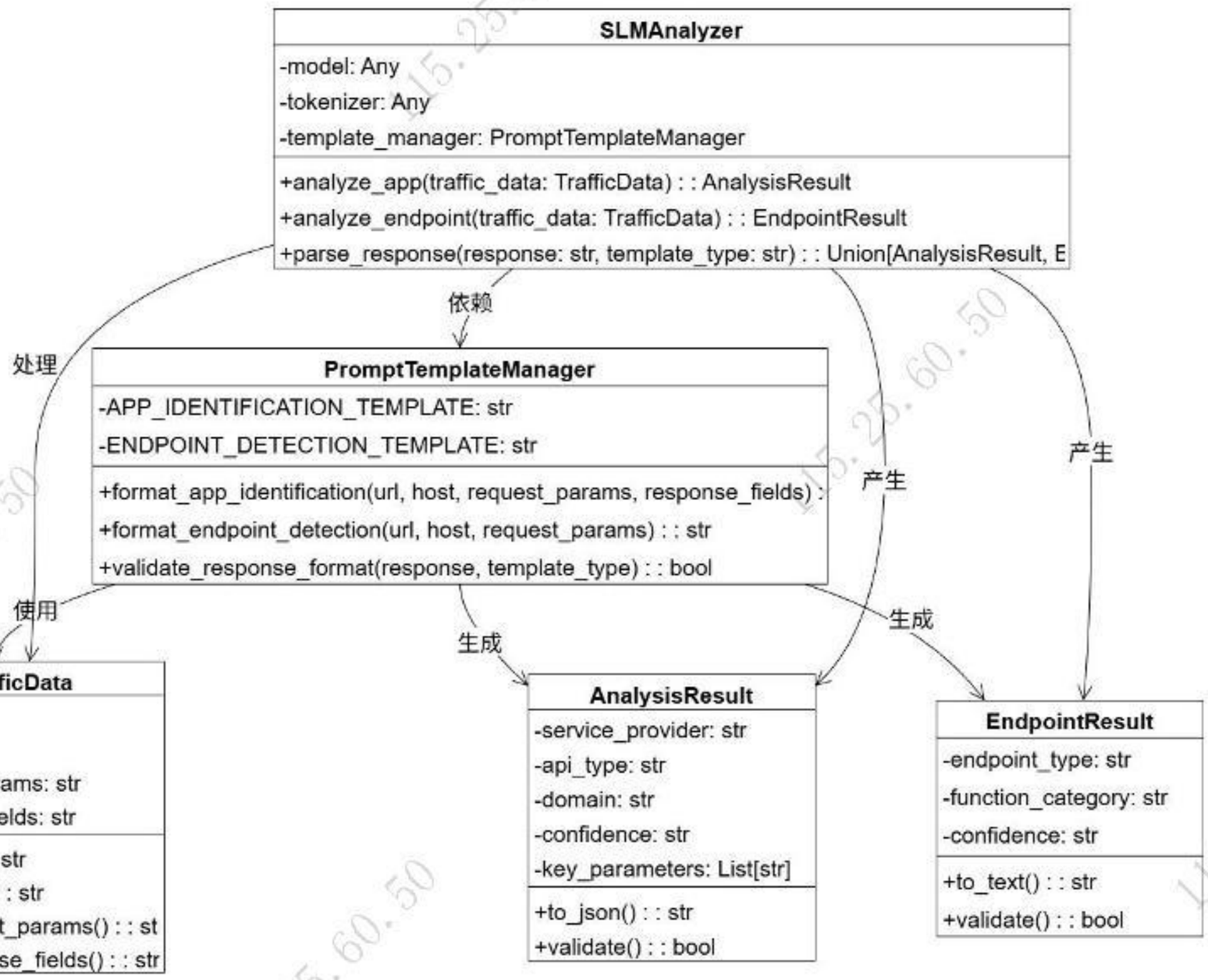


图 5-23 LLM 应用流量特征发现 UML 图

Fig.5-23 llm traffic feature discovery module UML diagram

（5）实现效果

如图 5-24、图 5-25 所示，用户能够通过系统，执行主动发现或被动发现功能以发现 LLM 大模型应用特征。其中主动发现功能由用户主动发起，而被动发现功能，会定时对已传输到后台的待分析可疑流量特征进行自动化分析发现，并将结果输出到大模型特征库中，供用户启用。

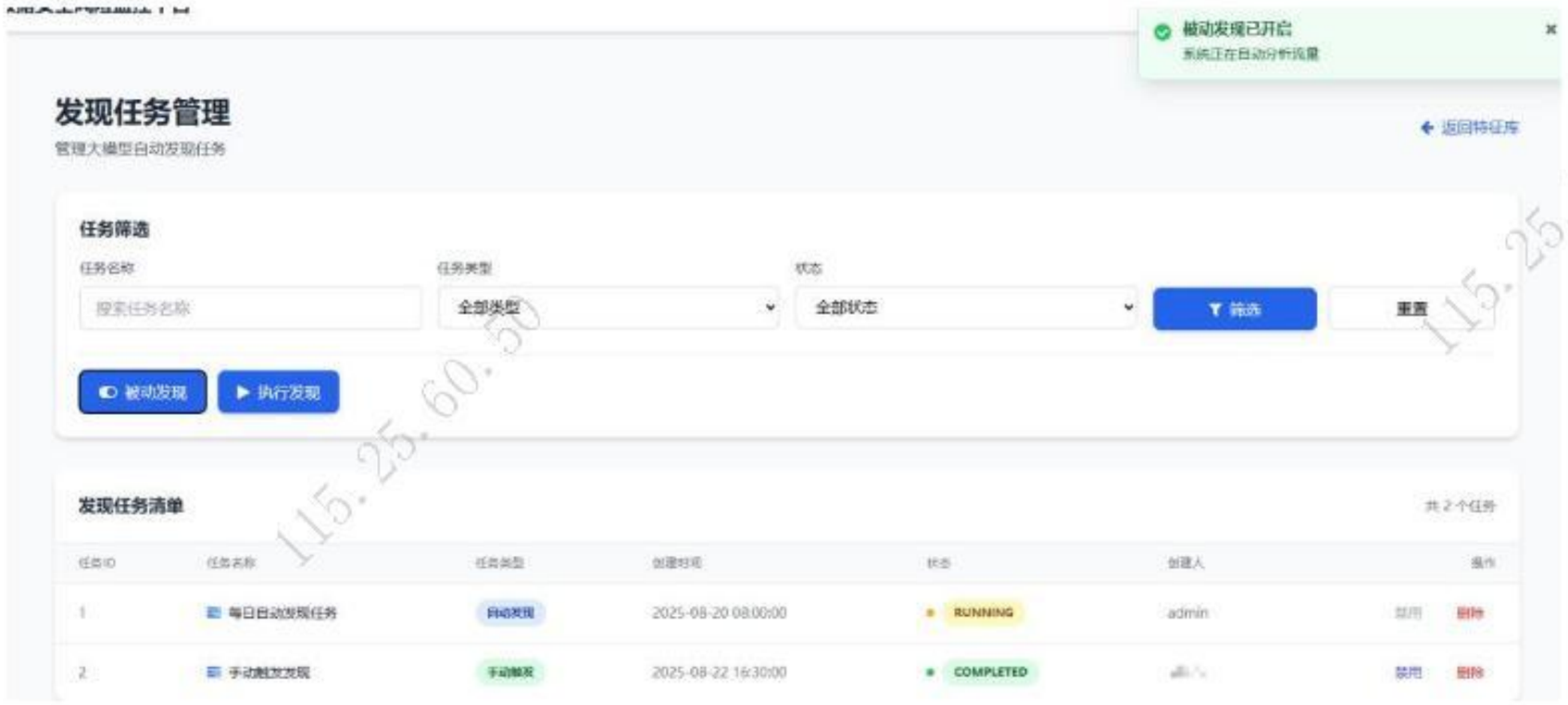


图 5-24 LLM 大模型流量特征主动及被动发现功能界面

Fig.5-24 Interface of active/passive LLM traffic feature discovery



图 5-25 已发现特征管理界面

Fig.5-25 Discovered feature management interface

如表 5-1 所示，通过 1000 条流量特征测试，基于 SLM 小模型的算法识别率在 90%以上，单条流量特征分析耗时为 1s 左右。在小数据、实时性方面与 GPT-4 零样本相比具有显著优势，在自动化方面效率方面相比人工规则匹配优势巨大。

表 5-1 算法性能对比

Tab.5-1 Algorithm performance comparison

算法/指标	规则匹配	基于 Qwen2 7B SLM 和领域知识	DeepSeek R1 零样本
准确率	80.1% 依赖规则完备性 已知模式准确率高，未知为 0	90% 较强泛化能力，具备领域知识误报率低	87% 强泛化能力，但存在幻觉
单条处理时间(秒)	0.4 纯规则引擎，无模型计算	1 模型小，计算开销低	5（API） 模型庞大，API 延迟高
效率	自动化程度低 人力维护成本极高 规则更新滞后	自动化程度高 人力投入少	自动化程度高 API 成本高
数据依赖	依赖专家经验 需要持续收集新规则	LLM 应用流量已知特征 依赖提示词 利用预训练知识	依赖提示词 利用预训练知识
可解读性	规则明确，完全透明 匹配原因清晰可追溯	自然语言输出，易于理解 可要求输出推理链	自然语言输出，易于理解 可要求输出推理链
适用场景	资源受限 流量相对固定 对可解释性要求极高	低成本，快速适配 需要平衡性能与成本	快速适配验证想法 作为专家辅助工具

（注：DeepSeek R1 为大语言模型[66]，依赖 API 调用实现零样本解析）

本方法在 LLM 应用流量特征发现场景中，凭借部分已知 LLM 应用流量特征知识和 SLM 较强的泛化能力，避免了完全依赖专家经验或零样本较大的误报率，发现效率较人工具有大幅提升，满足了企业环境下快速适配，同时还需要平衡性能与成本的需求。

6 监控平台测试

6.1 监控平台测试计划与策略

在系统开发完成后，为确保本 AI 大语言模型应用数据安全风险监控平台在部署上线后能够稳定、可靠且高效的运行，本章节将围绕测试计划、测试策略、测试方法[67]及测试结果与分析等方面，对 AI 大语言模型应用数据安全风险监控平台的系统测试工作进行全面阐述。

首先制定详尽且可执行落地的测试计划，用以明确测试目标、范围策略、方法。测试目标是系统性的验证平台各项功能的正确性、系统的稳定性、性能表现的达标情况以及安全防护能力的有效性。测试范围则涵盖终端流量采集测试、LLM 应用流量特征发现功能、告警规则管理等多个模块。其中终端流量采集模块负责在员工终端上以中间人代理的形式采集LLM应用流量及可疑流量特征并将其发送至后端存储和分析，其准确性关系平台的可用性。告警规则模块则服务于安全管理人员用于进行告警规则设置和维护，并在触发规则时产生预期的告警，对于安全管理人员进行安全分析具有非常重要的意义。

在测试策略上，针对不同的功能模块和测试目标，采取了黑盒测试、白盒代码审查、灰盒测试、压力测试、安全漏洞扫描、渗透测试以及用户测试相结合的多种策略。其中黑盒测试侧重于验证软件功能是否符合需求规格说明。测试人员在不关心内部代码结构与实现方式的情况下，通过输入预定义的数据，检验输出结果是否正确。此策略主要应用于业务逻辑和用户界面的功能验证。白盒测试则通过对平台核心模块的源代码进行静态审查与动态分析，确保代码逻辑清晰、结构合理，无潜在的内存泄漏、资源竞争等缺陷。灰盒测试则结合黑盒与白盒测试的优点，在了解系统部分内部结构的基础上进行测试。此策略常用于接口测试和数据流测试，以验证模块间数据传输与处理的正确性。系统安全测试方面，采用安全漏洞扫描和渗透测试结合的方式进行安全缺陷发现，确保系统安全可靠。其中安全漏洞扫描利用自动化工具对系统进行扫描，检测已知的常见安全漏洞（如 SQL 注入、跨站脚本 XSS 等）。而渗透测试则模拟恶意攻击者的行为，对平台进行主动的、深入的安全性探测，以发现逻辑性安全漏洞和更深层次的潜在风险。在测试末期，邀请安全管理人员在测试环境中进行操作，以

确认系统功能与用户体验满足业务需求。

根据测试策略和目标，为确保测试用例能够高效、全面地覆盖各类测试场景，我们采用了系统化的测试用例设计方法。选择了边界值分析、错误推测法等多种方法。其中，边界值分析法基于“大量错误倾向于发生在输入或输出范围的边界上”这一经验。对平台中所有存在明确数值范围或长度限制的输入域与接口参数进行了重点测试。例如，在测试“告警规则阈值”配置功能时，不仅测试了阈值范围内的常规值，更聚焦于最小值、最大值等边界点，以发现在极值情况下可能出现的计算溢出或逻辑判断错误。

错误推测法则是凭借测试人员的经验与对系统架构的理解，列举系统中可能存在缺陷或容易出错的特殊场景，并据此设计测试用例。例如，模拟网络中断时终端流量采集 Agent 的重连与数据恢复机制；在系统高负载运行时，强制触发新的安全告警以验证其处理优先级；或构造畸形的、不符合约定的 API 请求数据以测试后端服务的鲁棒性与异常捕获能力。这种方法能够有效地发现那些通过常规逻辑推导难以覆盖的深层错误。

表 6-1 功能测试

Tab.6-1 Functional testing

测试模块	测试场景	测试步骤	预期结果	测试方法
终端流量采集模块	流量采集与上报	1. 在终端部署并启动采集 Agent。 2. 模拟用户向大模型应用发送正常的问答请求流量。 3. 在平台管理端查询该终端的流量数据。	1. Agent 启动成功，状态为“运行中”。 2. 平台成功捕获到该次 API 调用记录。 3. 解析出的流量详情中应包含：正确的请求 URL、请求体（含预设关键词）、响应状态码（200）、以及响应内容（模型生成的文本）。	边界值分析因果图法
	采集模块异常处理	1. 在 Agent 正常采集流量时，手动强制终止 Agent 进程。 2. 再次发送若干条 LLM API 请求。 3. 重新启动 Agent 进程。 4. 检查平台是否接收到 Agent 重启后发送的数据，且数据序列完整无误。	1. Agent 终止后，平台不再收到该终端的新流量。 2. Agent 重启后，其状态迅速恢复为“运行中”。 3. 平台成功接收并处理 Agent 重启后新产生的流量数据，日志流连续，无数据丢失或大量重复。	错误推测法

表 6-1 续

Tab.6-1 Cont

测试模块	测试场景	测试步骤	预期结果	测试方法
LLM 应用流量特征自发现功能模块	新 LLM 应用与 API 端点自动识别	1. 配置并开启特征自发现功能。 2. 在终端上模拟访问一个之前未被监控的新 LLM 服务（或一个已知服务的新 API 路径）。 3. 观察系统的大模型应用特征库。	1. 系统在经过一个分析周期后，能在“未知/新发现应用”列表或类似界面中，展示新识别出的 LLM 服务端点。 2. 识别出的信息至少应包含：主机地址、API 路径，并可初步推断其所属的 LLM 服务提供商（如 OpenAI、Azure 等）	因果图法
告警规则管理模块	创建并触发自定义告警规则	1. 以管理员身份登录平台，进入告警规则管理页面。 2. 创建一条新规则：当任何请求中检测到“身份证号”模式时，触发“PII 泄露风险”告警，级别为“高危”。 3. 保存并启用该规则。 4. 模拟一次包含身份证号的 LLM API 请求。 5. 检查平台的告警中心。	1. 规则创建成功，状态为“已启用”。 2. 在发送模拟请求后，平台告警中心在预期时间内新增一条“PII 泄露风险”告警。 3. 告警详情中应包含：触发时间、触发的规则名称、风险级别、相关的会话 ID 或请求详情。	因果图法 边界值分析
	告警规则的编辑与状态管理	1. 找到一条已存在的告警规则。 2. 对其进行编辑，修改其告警级别和触发条件，然后保存。 3. 禁用该规则。 4. 再次执行原本会触发该规则的流量操作。 5. 重新启用该规则，并再次测试。	1. 规则信息修改成功。 2. 规则被禁用后，即使满足条件，也不再触发告警。 3. 规则被重新启用后，能够再次正常触发告警。 4. 所有操作均有相应的成功提示或日志记录。	错误推测法

表 6-2 性能及安全性测试

Tab.6-2 Performance and safety test

测试模块	测试场景	测试步骤	预期结果	测试工具及方法
性能测试	高并发流量采集与处理	1. 使用压力测试工具，模拟 1000 个以上的并发终端在 30 分钟内持续向平台发送 LLM API 请求与响应流量。 2. 监控平台服务器的 CPU、内存、磁盘 I/O 和网络带宽使用情况。 3. 监控平台应用层的指标：请求队列长度、处理延迟、错误率。	1. 平台服务无宕机，各功能模块响应正常。 2. 资源利用率（如 CPU、内存）处于安全水位（如 <80%），无内存泄漏迹象。 3. 流量数据处理无积压，平均处理延迟低于 2 秒，错误率低于 0.1%。	Apache JMeter、压力测试
安全性测试	身份认证与授权绕过	1. 在未登录状态下，直接尝试访问需要权限的管理员 API 接口 URL（如 /api/alerts/delete）。 2. 使用低权限用户（如只读用户）的 Token，尝试执行高权限操作(如创建或修改告警规则）。 3. 检查 Cookie 和 Token 的强度（如是否使用 HTTPS、是否可预测）。	1. 所有未授权访问尝试均应返回 401 Unauthorized 或 403 Forbidden 错误。 2. 低权限用户无法执行高权限操作，并收到明确的拒绝信息。 3. 认证凭证具备足够的强度，无法被轻易破解或重放。	Burp Suite、渗透测试、权限提升测试
	SQL 注入与命令注入	1. 在平台的每一个输入框、查询框和 API 参数中，尝试输入 SQL 注入和 OS 命令注入 Payload。 2. 特别关注告警规则的条件配置、流量查询等核心功能的输入点。	1. 平台防御注入攻击，不会返回数据库错误信息或执行异常命令。 2. 系统能安全处理非法输入，可能表现为返回空结果、记录安全日志并返回通用错误信息。	SQLMap、安全漏洞扫描、渗透测试

表 6-3 易用性测试

Tab.6-3 Usability test

测试模块	测试场景	测试步骤	预期结果	测试工具及方法
界面友好性	整体布局与导航	1. 招募 5-8 名信息安全专员。 2. 不给予任何指导，直接让测试者访问平台主界面。 3. 要求他们口头阐述他们认为平台是做什么的，以及如何完成几个核心任务（如“查看今天的告警”、“查找一条特定会话”）。	1. 测试者能准确描述平台的核心用途（用于监控 LLM 数据安全）。 2. 测试者能通过导航栏和主要 UI 元素（如按钮、搜索框）正确指出完成指定任务的路径。 3. 主要功能模块的命名清晰、无歧义。	可用性测试、观察记录表
操作便捷性	关键路径优化验证	1. 要求测试者创建一条规则：当请求中包含“身份证号”且响应中包含“内部代码”时，触发告警。 2. 不给予步骤指导，仅观察其操作过程。	1. 测试者能轻松找到“创建新规则”的入口。 2. 规则配置界面布局合理，表意清晰（如使用“与”、“或”逻辑关系选择，而非纯代码）。 3. 具备实时验证或示例提示，帮助用户正确填写正则表达式或选择字段。 4. 测试者能在 5 分钟内无需外部帮助完成配置。	可用性测试、观察记录表

6.2 监控平台测试结果与分析

针对平台进行了全面且细致的测试，以确保系统的稳定性和可靠性。测试工作主要包括功能测试、安全测试、性能测试及用户体验反馈收集四个环节，以下是对测试结果及存在问题的详细分析。

在功能测试方面，平台的核心功能在绝大部分场景下运行符合预期。终端流量采集模块能够准确捕获和解析主流的 LLM 应用流量；特征自发现功能可以识别新的 LLM 应用及 API 端点；告警规则管理模块的生命周期管理（创建、编辑、启用/禁用）和触发机制运行正常。所有关键功能路径上的正向测试用例通过率达到 95%。但在终端流量采集模块的测试中，发现当模拟的网络闪断时间过长时，有极低概率导致某个采集 Agent 无法正常

恢复，需要手动重启服务，其容错与自愈能力需进一步加强。

在性能测试方面，平台在常规负载下表现稳定，能够满足日常监控需求。在持续 30 分钟、500 个并发终端的数据压力下，系统资源利用率保持在健康水平（CPU<65%，内存<70%），流量处理无积压，核心功能的响应时间均在可接受范围内。但在模拟的瞬时流量峰值冲击下（如每秒上千条请求），虽然系统服务未宕机，但从事件发生到告警触发的端到端延迟出现了波动，未能始终稳定在 15 分钟以内的设计目标，表明事件处理队列与告警引擎在高负载下的优化不足。

在安全测试方面，平台在基础安全防护方面表现扎实。身份认证与授权机制健全，未发现可越权访问的高危漏洞。能够对常见的 SQL 注入和命令注入等攻击进行防御。但权限控制细粒度不够，目前平台的用户权限模型较为粗放（如仅区分管理员和只读用户），缺乏基于“角色”或“数据范围”的细粒度权限控制。例如，无法配置某个用户只能处理 A 业务线的告警而不能查看 B 业务线的告警及流量日志，存在数据横向越权的风险。

在用户体验反馈方面，用户对平台的整体布局和导航清晰度给予了积极评价。新手用户能够在短时间内理解平台的核心价值，并完成基本的告警查看和处理任务。系统的视觉反馈，如告警等级的颜色编码，得到了测试者的一致认可。但当用户执行批量操作或长时间运行的任务时，如“执行发现”功能，系统仅提供简单的“已开启”图标，缺乏对任务进度、预计剩余时间等的文字描述，用户在等待时容易产生焦虑，并无法判断操作是否正常进行。

总的来说，本平台已具备坚实的产品基础和核心能力，但在从“可用”到“好用、耐用、安全”的企业级产品的演进过程中，仍需在性能优化、安全模型的精细化、以及用户体验的细节打磨上投入更多精力。

表 6-4 测试结果

Tab.6-4 Results of the testing

测试类型	测试内容	测试结果	存在问题分析
功能测试	对终端流量采集、流量特征自发现、告警规则管理等核心模块的功能正确性进行验证。	核心功能实现正常，流量采集、特征发现与告警触发等基本场景均符合预期。	采集 Agent 容错性不足：在极端网络异常下，采集服务存在无法自动恢复的风险，影响数据完整性。
性能测试	通过模拟高并发、大数据量场景，评估系统的处理能力、稳定性和响应速度。	在持续 30 分钟、500 个并发终端的数据压力下，系统资源利用率保持在健康水平（CPU<65%，内存<70%），流量处理无积压	瞬时高流量冲击时，事件处理与告警产生链路出现拥堵，端到端延迟不稳定。
安全测试	对身份认证、授权、输入安全及敏感信息泄露等进行渗透性测试。	基础安全防护牢固，未发现 SQL 注入、越权访问等高危漏洞。认证授权机制有效。	权限模型过于粗放：缺乏基于角色或数据范围的细粒度权限控制，存在数据横向越权风险。
用户体验测试	评估界面的直观性、操作的便捷性以及系统反馈的清晰度。	整体布局与导航获得好评，新手用户能快速上手完成基本操作。	操作反馈不明确：长耗时任务缺乏进度提示，用户感知差，容易产生困惑与焦虑。

测试结果如表 6-4 所示，针对上述测试中发现的问题，本文制定了相应的改进优化措施。对于采集 Agent 容错性不足，优化采集 Agent 的重连与数据恢复机制，提升健壮性与易用性；对于瞬时高流量，通过优化告警处理链路进行异步化与流量削峰改造，确保在高负载下仍能提供稳定服务；对于权限管理粗放的问题，构建精细化权限体系，设计并实施基于角色的访问控制模型；在用户体验提升方面，在界面中增加术语提示与任务引导，并对所有耗时操作提供明确的进度指示，降低用户的学习成本与操作焦虑。

通过本次测试工作，本文全面了解了 AI 大语言模型应用数据安全风险监控平台的性能、安全性和稳定性情况，发现了存在的问题并制定了相应的改进措施。

7 结论与展望

7.1 研究结论

基于上述研究、设计和实现，本研究成果成功在某大型企业落地运行，为企业内部 LLM 应用的数据安全治理提供了可行路径与实践参考。

平台设计方面，研究明确了以高效、便捷管控 LLM 应用数据安全为核心目标，构建了包括终端流量采集、LLM 应用流量特征识别、告警规则管理等在内的核心模块。终端流量采集模块负责实时捕获终端侧与 LLM 应用交互的关键通信数据，为后续分析提供基础支撑。流量特征识别模块则基于领域知识与语义理解模型（SLM），增强对未知 LLM 应用的识别能力，从而降低人工维护成本，提升管控覆盖范围。告警规则管理模块则赋予信息安全人员灵活的规则配置能力，使其能够有效落地数据安全监控策略，识别潜在风险。

功能实现过程中，平台引入先进的可视化设计，通过直观的图形界面帮助安全管理人员迅速掌握企业内部 LLM 应用使用过程中的安全态势，辅助其高效决策。同时，交互设计的优化也显著提升了用户操作的流畅性与平台的整体易用性。

技术架构层面，平台采用前后端分离的设计思路，前端基于 HTML5、Tailwind CSS 与 JavaScript 实现可视化界面与交互逻辑；后端采用 Python 处理业务逻辑并封装服务接口；数据存储方面，选用 MariaDB 关系型数据库支撑平台运营数据，同时结合 Kafka 与 ElasticSearch 构建大数据存储方案，以应对海量流量数据与告警日志的处理需求。该架构设计有效保障了平台的稳定性、安全性与扩展性。

系统测试与验证阶段，研究制定了详尽的测试方案，涵盖功能完整性、性能稳定性及用户体验等多个维度。测试结果表明，各核心模块运行正常，平台整体表现可靠。通过多轮用户反馈收集与迭代优化，平台在核心功能与操作体验方面均得到持续改进。

研究过程中同时也识别出若干问题，例如面对 LLM 应用的快速迭代，如何持续保障流量采集的覆盖能力。为此，平台融合已知流量特征与 SLM 的泛化识别能力，实现了对新增 LLM 应用的自动发现，降低了人工维护负担，提升了系统的适应性与实用性。

综上所述，本研究成功设计并实现了面向 LLM 应用的数据安全风险监控平台，落地某大型企业后显著提升了其内部对 LLM 应用数据安全风险的管控能力，为其规模化应用提供了安全保障。同时，研究和实践也指出平台在数据处理效率、可视化交互体验以及风险自动化处置等方面仍存在优化空间，未来将围绕上述方向展开进一步探索，以推动平台能力的持续完善与实际应用的深入拓展。

7.2 后续工作展望

尽管本研究构建的 AI 大语言模型应用数据安全风险监控平台已取得阶段性成果，但随着 LLM 应用的快速演进与企业安全治理要求的不断提升，平台在智能化水平、自动化闭环能力及用户体验等方面仍面临诸多挑战，亟待进一步探索与完善。

在自动化与智能化方面，当前平台虽已引入 RAG 技术进行告警分析，能够基于安全领域知识对告警信息进行智能分析并生成处置建议，但分析结果仍需依赖人工实施具体操作，未形成“采集—分析—处置”的完整自动化闭环。未来研究应致力于推动智能决策与自动执行的深度融合，探索基于策略引擎与工作流自动化的响应机制，以实现对数据安全风险的实时、自主处置。同时特征识别准确率还有待通过模型微调等方式进一步提升。

在用户体验方面，平台需进一步强化面向不同角色用户的功能设计与可视化支持。作为企业级管理工具，除满足安全分析人员的操作需求外，还应为管理决策层提供 LLM 应用全链路数据安全风险的整体视图，辅助其制定相关发展战略与管控策略。例如，可通过开发 LLM 应用运营看板、风险态势大屏等功能，直观呈现员工使用行为与风险分布，助力企业在保障安全的前提下推进 LLM 技术的合理应用与效能提升。

在系统协同与集成方面，未来研究应关注监控平台与企业现有安全体系及数据管理系统的融合。企业数据安全涉及核心数据资产、商业秘密、知识产权等多类信息，需与资产管理系统、专利平台、DLP 数据防泄漏系统等形成联动机制。通过构建统一的数据接口与规范化的信息交互流程，实现跨系统间策略同步与风险联控，从而提升企业整体数据安全治理的协同性与系统效能。

综上所述，未来研究应围绕智能化闭环能力建设、多用户体验优化以及系统集成等方向持续推进，以进一步提升监控平台实用性、扩展性与企业级适配能力，为企业安全、高效地部署与应用 LLM 应用提供安全支撑。

参考文献

[1] NATH S, MARIE A, ELLERSHAW S, et al. New meaning for NLP: the trials and tribulations of natural language processing with GPT-3 in ophthalmology[J]. British Journal of Ophthalmology, 2022, 106(7): 889-892.

[2] HENDY A, ABDELREHIM M, SHARAF A, et al. How good are G PT models at machine translation? A comprehensive evaluation[EB/O L]. arXiv:2302.09210v1 [cs.CL], 2023[2024-10-01]. https://arxiv.org/ab s/2302.09210v1.

[3] LIAO Y, WANG Y S, LIU Q, et al. GPT-based generation for classi cal Chinese poetry[EB/OL]. arXiv:1907.00151v5 [cs.CL], 2019[2024-1 1-20]. https://arxiv.org/abs/1907.00151v5.

[4] WU C F, YIN S M, QI W X, et al. Visual ChatGPT: talking, drawi ng and editing with visual foundation models[EB/OL]. arXiv:2303.04 671v1 [cs.CV], 2023[2024-08-11]. https://arxiv.org/abs/2303.04671v1.

[5] 喻婉欣. 大语言模型的数据安全风险及治理策略研究[J]. 保密科学技术, 2024(04).

[6] 邓臻宇. 负责任的发展生成式人工智能的挑战及应对路径[J]. 网络安全与数据治理, 2024, 43(7): 69－76.

[7] 王大志, 张挺. 风险、困境与对策: 生成式人工智能带来的个人信息安全挑战与法律规制[J]. 昆明理工大学学报(社会科学版), 2023:23-2.

[8] 赵鑫, 李军毅, 周昆, 唐天一, 文继荣. 大语言模型[EB/OL]. (2023-03 -17)[2024-07-10]. https://llmbook-zh.github.io/.

[9] JELINEK F. Statistical methods for speech recognition[M]. Cambridg e: MIT Press, 1998.

[10] BENGIO Y, DUCHARME R, VINCENT P, et al. A neural probabili stic language model[J]. Journal of Machine Learning Research, 2003, 3: 1137-1155.

[11] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all yo u need[C]//Advances in Neural Information Processing Systems 30 (NIPS 2017). Long Beach, USA: Curran Associates, Inc., 2017: 599

8-6008.

[12] DEVLIN J, CHANG M W, LEE K, et al. BERT: pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. Minneapolis, MN, USA: Association for Computational Linguistics, 2019: 4171-4186.

[13] KAPLAN J, MCCANDLISH S, HENIGHAN T, et al. Scaling laws for neural language models[EB/OL]. arXiv:2001.08361 [cs.LG], 2020 [2024-10-15]. https://arxiv.org/abs/2001.08361.

[14] NVIDIA Corporation. OPEN AI 展示 GPT-3,一个 1750 亿参数的语言模型[EB/OL]. (2020-07-07)[2024-06-14]. https://developer.nvidia.com/zh-cn/blog/openai-presents-gpt-3-a-175-billion-parameters-language-model.

[15] 张奇, 桂韬, 郑锐, 等. 大规模语言模型：从理论到实践[M]. 北京：电子工业出版社, 2023.

[16] 北京市科委. 智源研究院发布超大规模智能模型“悟道 1.0”[EB/OL]. (2021-03-30)[2024-04-12]. https://www.most.gov.cn/dfkj/bj/zxdt/202103/t20210329_173708.html.

[17] 时代周报. 李彦宏发布首个“中国版 ChatGPT”，百度文心一言能否一战?[EB/OL]. (2023-03-17)[2024-07-10]. https://m.thepaper.cn/newsDetail_forward_22343209.

[18] 沙丘社区. 2024 中国大模型先锋案例 TOP30[EB/OL]. (2024-01-26)[2024-07-10]. https://www.zkj.com/white-book/51.html.

[19] 戴枞. ChatGPT：人工智能的通用性发展及其法律规制[J]. 西南政法大学学报, 2023, 25(4): 86-97.

[20] 邓建鹏. ChatGPT 模型的法律风险及应对之策[J]. 新疆师范大学学报（哲学社会科学版）, 2023, 44(5): 92-102.

[21] 刘艳红. 生成式人工智能的三大安全风险及法律规制—以 ChatGPT 为例[J]. 东方法学, 2023(4): 31-43.

[22] BROWN T, MANN B, RYDER N, et al. Language models are few-shot learners[J]. Advances in neural information processing systems, 2020, 33: 1877-1901.

[23] 张海，刘畅，王东波，赵志枭. ChatGPT用户使用意愿影响因素研究[J]. 情报理论与实践，2023(10).

[24] 南方都市报. 多家科技巨头提醒员工：不要向ChatGPT泄露敏感信息[EB/OL]. (2023-02-10)[2024-07-11]. https://www.secrss.com/articles/51920.

[25] NASR M, CARLINI N. Scalable extraction of training data from (production) language models[EB/OL]. arXiv:2311.17035 [cs.LG], 2023 [2024-09-26]. https://arxiv.org/abs/2311.17035.

[26] 任文岱. ChatGPT热度不减：专家建议人工智能领域制定行业安全标准[EB/OL]. (2023-03-22)[2024-05-10]. https://www.thepaper.cn/newsDetail_forward_22398307.

[27] 经济日报. ChatGPT禁令频发为哪般[EB/OL]. (2023-03-02)[2024-07-10]. http://views.ce.cn/view/ent/202303/02/t20230302_38420351.shtml.

[28] 宪瑞. 三星引入ChatGPT不到20天就出事，半导体机密恐已外泄[EB/OL]. (2023-04-03)[2024-07-10]. https://www.51cto.com/article/750961.html.

[29] Gartner. Gartner发布2025年网络安全重要趋势[EB/OL]. (2025-03-26)[2025-04-11]. https://www.gartner.com/cn/newsroom/press-releases/2025-cyber-security-trends.

[30] 赵静. 大模型检索增强生成（RAG）技术浅析[J]. 中国信息化，2024[10].

[31] 张晓明. 软件系统设计与体系结构[J]. 中国铁道出版社,2018，1:31-36.

[32] 严关武，黎宏剑，黄广文. 基于 B/S 的终端应用自动拨测系统设计和实现[J]. 移动通信，2014，1:9-10.

[33] LU Z, LI X, CAI D, et al. Small language models:survey,measurements,and insights[EB/OL]. (2024-09-24)[2025-06-01]. https://arxiv.org/abs/2409.15790.

[34] ZHANG P, ZENG G, WANG T, et al. Tinyllama: an open-source small language model[EB/OL]. (2024-01-04)[2025-06-01]. https://arxiv.org/abs/2401.02385.

[35] 华子荀，王炜，廖绿优，等. 生成式人工智能RAG路径赋能课程设计：课程小语言模型构建与教学实践[J]. 广东第二师范学院学报，2025，45(05):31-44. DOI:CNKI:SUN:GJXY. 0. 2025-05-004.

[36] SSL MITM Proxy[EB/OL]. (2007-3-18)[2024-01-12]. https://crypto.stanford.edu/ssl-mitm.

[37] MITM proxy[EB/OL]. (2015-10-13)[2024-01-12]. https://mitmproxy.org.

[38] 魏涛, 孟方园, 袁平等. 开源搜索引擎 Elasticsearch 和 Solr 对比和分析[J]. 现代计算机(专业版), 2018, (06): 58-61.3

[39] 柳帆. 基于 ElasticSearch 的科技资源检索系统的研究与实现[J]. 现代计算机, 2021, 27(26): 93-100.

[40] 南淑萍, 王莉丽, 王秀友. Python Web 开发案例教程：慕课版：使用 Flask、Tornado、Django[M]. 北京：人民邮电出版社, 2020.

[41] 马旭, 王淑丽. 基于 Flask 框架的展示型网站的设计与实现[J]. 数字技术与应用, 2018, 36(11):137-138.

[42] GRINBERG M. Flask Web 开发：基于 Python 的 Web 应用开发实战[M]. 安道,译. 北京：人民邮电出版社, 2015.

[43] FastAPI-Introduction[EB/OL]. (2023-01-14)[2024-04-10]. https://www.geeksforgeeks.org/python/fastapi-introduction/.

[44] 高瞻, 陈文, 姜鹏, 等. 基于 XGBoost 的新冠肺炎智能检测系统实现[J]. 中国数字医学, 2020, 15(11):123-127.

[45] ElastAlert-Elasticsearch[EB/OL]. (2015-09-24)[2024-01-13]. https://elastalert.readthedocs.io.

[46] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks[J]. Advances in neural information processing systems, 2020, 33: 9459-9474.

[47] SOONG D, SRIDHAR S, SI H, et al. Improving accuracy of GPT-3/4 results on biomedical data using a retrieval-augmented language model[J]. PLOS digital health, 2024, 3(8): e0000568.

[48] 张鹤译, 王鑫, 韩立帆, 等. 大语言模型融合知识图谱的问答系统研究[J].计算机科学与探索, 2023, 17(10): 2377-2388.

[49] HE P, ZHU J, ZHENG Z, et al. Drain: an online log parsing approach with fixed depth tree[C]//2017 IEEE International Conference on Web Services (ICWS). Honolulu, HI, USA: IEEE, 2017: 33-40.

[50] DU M, LI F. Spell: online streaming parsing of large unstructured system logs[J]. IEEE transactions on knowledge and data engineering,

2018, 31(11): 2213-2227.

[51] NEDELKOSKI S, BOGATINOVSKI J, ACKER A, et al. Self-attentive classification-based anomaly detection in unstructured logs[C]//2020 IEEE International Conference on Data Mining (ICDM). IEEE, 2020: 1196-1201.

[52] DEVLIN J, CHANG M W, LEE K, et al. BERT: pre-training of deep bidirectional transformers for language understanding[C]//Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. Minneapolis, MN, USA: Association for Computational Linguistics, 2019: 4171-4186.

[53] 张颖君, 刘尚奇, 杨牧, 等. 基于日志的异常检测技术综述[J]. 网络与信息安全学报, 2020, 6(6): 1-12.

[54] CUI J, NING M, et al. ChatLaw: a multi-agent collaborative legal assistant with knowledge graph enhanced mixture-of-experts large language model[EB/OL]. arXiv:2306.16092 [cs.CL], 2023[2024-11-03]. https://arxiv.org/abs/2306.16092.

[55] 陈敬. 基于 ELK 的日志分析与异常检测系统的设计与实现[D]. 西安: 西安电子科技大学, 2022.

[56] 廖湘科, 李姗姗, 董威, 贾周阳, 刘晓东, 周书林. 大规模软件系统日志研究综述[J]. 软件学报, 2016, 27(8): 1934-1947.

[57] 张晓明. 软件系统设计与体系结构[J]. 中国铁道出版社, 2018, 1: 31-36.

[58] 希尔皮斯科. Python 金融大数据分析[M]. 人民邮电出版社, 2015.

[59] 胡雯, 李燕. MySQL 数据库存储引擎探析[J]. 软件导刊, 2012, 11(12): 129-131.

[60] 郑雷雷, 付义荣, 郭锐等. B/S 架构软件的安全性测试研究[C]. 中国安防展览网. 大数据环境下安防系统安全等级保护研讨会论文集. [出版者不详], 2018: 5.

[61] 潘忠英. 分布式数据库系统设计研究[J]. 分布式数据库系统设计研究, 2023, 1: 33-35.

[62] 郑天宇. 基于大语言模型的低代码应用程序及其测试用例研究[D]. 广州大学, 2025.DOI:10.27040/d.cnki.ggzdu.2025.002060.

[63] WANG L, CHEN J, WANG Z, et al. Semi-supervised log-based anomaly detection via probabilistic label estimation[C]//2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE). IEEE, 2021: 1448-1460.

[64] 张昭云. LangChain 框架下基于 ChatGLM3 模型的个性化儿科健康咨询服务研究[D]. 兰州财经大学, 2025.DOI:10.27732/d.cnki.gnzsx.2025.000546.

[65] Chroma is the open-source AI application database[EB/OL]. (2022-05-01)[2024-05-10]. https://docs.trychroma.com/docs/overview/introduction.

[66] GUO D, YANG D, ZHANG H, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning[J]. Nature, 2025, 645: 633-638. DOI: 10.1038/s41586-025-09422-z.

[67] 李天尘, 周亮亮. 信息系统的测试与质量控制[J]. 上海计量测试, 2011.

附录 A 流量特征自发现算法代码

```
class TrafficPreprocessor:
"""流量数据预处理器"""
# LLM API 核心参数知识库
LLM_PARAM_KNOWLEDGE = {
    'request_params': ['model', 'messages', 'temperature', 'max_tokens',
                       'stream', 'top_p', 'frequency_penalty',
'presence_penalty'],
    'response_fields': ['id', 'object', 'model', 'usage', 'choices',
                        'created', 'finish_reason']
}
    def preprocess(self, raw_traffic: Dict) -> ProcessedTraffic:
        """预处理原始流量数据"""
    # 提取并清洗 URL 路径
        url_path = self._extract_url_path(raw_traffic['url'])
    # 提取 Host 域名
        host = raw_traffic['host']
    # 从请求体中提取关键参数
        request_params = self._extract_llm_parameters(
            raw_traffic.get('request_body', ''),
            self.LLM_PARAM_KNOWLEDGE['request_params']
        )
            # 从响应体中提取关键字段
        response_fields = self._extract_llm_parameters(
            raw_traffic.get('response_body', ''),
            self.LLM_PARAM_KNOWLEDGE['response_fields']
        )
            return ProcessedTraffic(
        url=url_path,
        host=host,
        request_params=request_params,
        response_fields=response_fields
```

```
        )
    def _extract_llm_parameters(self, json_body: str, key_list: List[str]) -> str:
        """从 JSON 体中提取 LLM 相关参数"""
        try:
            data = json.loads(json_body)
            extracted = []

            for key in key_list:
                if key in data:
                    value = str(data[key])
# 对长文本进行智能截断
                    if len(value) > 100:
                        value = value[:100] + "..."
                    extracted.append(f"{key}:{value}")
            return " | ".join(extracted) if extracted else "无相关参数"
        except json.JSONDecodeError:
            return "JSON 解析失败"
```

```
    class SLMTrafficAnalyzer:
    """基于 SLM 的流量分析器"""
    def __init__(self, model_path: str):
        self.tokenizer = AutoTokenizer.from_pretrained(model_path)
        self.model = AutoModelForCausalLM.from_pretrained(model_path)
        self.prompt_manager = PromptTemplateManager()
    def analyze_traffic(self,processed_traffic:ProcessedTraffic) ->
AnalysisResult:
        """分析处理后的流量数据"""
    # 构建分析提示词
        prompt=
self.prompt_manager.APP_IDENTIFICATION_TEMPLATE.format(
            url=processed_traffic.url,
            host=processed_traffic.host,
            request_params=processed_traffic.request_params,
```

```
                response_fields=processed_traffic.response_fields
            )
        # SLM 推理
            raw_response = self._query_slm(prompt)
        #  解析结构化结果
            structured_result = self._parse_analysis_result(raw_response)
            return AnalysisResult(
                raw_response=raw_response,
                structured_result=structured_result,
                confidence=self._evaluate_confidence(structured_result)
            )
        def _query_slm(self, prompt: str, max_length: int = 500) -> str:
            """查询 SLM 获取分析结果"""
            inputs = self.tokenizer(prompt, return_tensors="pt", truncation=True,
max_length=1024)
            with torch.no_grad():
                outputs = self.model.generate(
                    inputs.input_ids,
                    max_length=len(inputs.input_ids[0]) + max_length,
                    temperature=0,   # 温度保证输出稳定性
                    do_sample=True,
                    pad_token_id=self.tokenizer.eos_token_id,
                    eos_token_id=self.tokenizer.eos_token_id
                )
            response = self.tokenizer.decode(outputs[0],
skip_special_tokens=True)
        # 提取模型生成部分（去除提示词）
            return response[len(prompt):].strip()
        def _parse_analysis_result(self, raw_response: str) -> Dict:
            """解析 SLM 的原始响应为结构化数据"""
            try:
        # 首先尝试解析 JSON 格式
                if raw_response.strip().startswith('{'):
                    return json.loads(raw_response)
```

```
        except:
            pass
    # 备用方案：基于规则的文本解析
        return self._rule_based_parsing(raw_response)
    def _rule_based_parsing(self, text: str) -> Dict:
        """基于规则的文本解析兜底方案"""
        result = {
            "service_provider": "未知",
            "api_type": "未知",
            "confidence": "中",
            "key_parameters": []
        }
    # 服务商识别规则
        provider_patterns = {
            "OpenAI": ["openai", "api.openai.com"],
            "Azure OpenAI": ["azure", "openai.azure.com"],
            "Anthropic": ["anthropic", "claude", "api.anthropic.com"],
            "国内厂商 A": ["company-a", "a.com/api"]
        }
        for provider, keywords in provider_patterns.items():
            if any(keyword in text.lower() for keyword in keywords):
                result["service_provider"] = provider
                break
        return result
```

如下代码用于 LLM 应用流量特征结构化输出。

```
class StructuredOutputGenerator:
"""结构化输出生成器"""
def generate(self, analysis_result: AnalysisResult,
                confidence: float,
                processed_traffic: ProcessedTraffic) -> Dict:
    """生成最终的结构化输出"""
    structured_result = analysis_result.structured_result
    return {
```

```
            "metadata": {
                "analysis_id": self._generate_analysis_id(),
                "timestamp": datetime.now().isoformat(),
                "confidence_score": confidence,
                "analysis_version": "1.0"
            },
            "identification": {
                "service_provider": structured_result.get("service_provider", "
未知"),
                "api_function": structured_result.get("api_type", "未知"),
                "traffic_category": self._categorize_traffic(structured_result),
                "is_llm_traffic": self._is_llm_traffic(structured_result)
            },
            "technical_details": {
                "api_endpoint": processed_traffic.url,
                "domain": processed_traffic.host,
                "detected_parameters":
structured_result.get("key_parameters", []),
                "parameter_patterns":
self._extract_parameter_patterns(processed_traffic)
            },
            "monitoring_metadata": {
                "suggested_monitoring_rules":
self._generate_monitoring_rules(structured_result),
                "risk_level": self._assess_risk_level(structured_result),
                "update_strategy": "auto" if confidence > 0.7 else
"manual_review"
            }
        }
```

致　谢

写在深圳的一个炎热的夜晚，当最后一个字落下，也代表着我研究生生活也接近尾声。学习、工作和生活是这三年的主旋律，感谢这段日子让我重温了大学校园和生活，在忙碌的工作中留得半刻悠闲，或许以后都不会再有这样的日子了。

首先，务必要感谢我的导师周贤伟教授，您总是能用一些有趣而又真实的例子，为我的论文选题、构思指明方向。同时您专业、严谨的研究思维和工作作风，也深深的感染影响着我，让我终身受益。

其次，还要感谢数理学院的各位授课老师，您们传授的专业知识，让我夯实基础、打开思维，让我在工作中更能游刃有余。

同时，特别感谢我的女友，谢谢她的加油鼓劲和陪伴。

作者简历及在学研究成果

一、主要教育经历/工作经历（从大学起，到硕士入学止）

起止年月	学习或工作单位	备注
2015 年 09 月至 2019 年 07 月	在四川工业科技学院电子信息工程专业攻读学士学位	
2021 年 10 月至 2024 年 12 月	在普华永道单位从事信息安全管理咨询工作	
2023 年 09 月至今	在北京科技大学软件工程（非全日制）专业攻读硕士学位	

二、在学期间从事的科研工作

三、在学期间所获的科研奖励

四、在学期间发表的论文

独创性说明

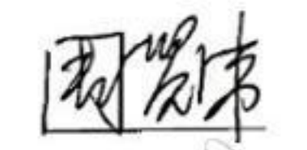
本人声明所呈交的论文是我个人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京科技大学或其它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作 者 签 名 ：胡程钰 日期：2025 年 12 月 25 日

关于论文使用授权的说明

本人完全了解北京科技大学有关保留、使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

（保密的论文在解密后应遵循此规定）

签名：  导师签名：  日期：2025 年 12 月 25 日

学位论文数据集

关键词*	密级*	中图分类号*	UDC	论文资助
大语言模型；数据安全监控；流量分析；信息泄露；企业内部风险	公开	TP311	004	
学位授予单位名称*		学位授予单位代码*	学位类别*	学位级别*
北京科技大学		10008	工程硕士	硕士
论文题名*		并列题名		论文语种*
AI 大语言模型应用数据安全风险监控平台设计与实现				汉语
作者姓名*	胡程钰		学号*	M202340082
培养单位名称*		培养单位代码*	培养单位地址	邮编
北京科技大学		10008	北京市海淀区学院路 30 号	100083
学科专业*		研究方向*	学制*	学位授予年*
软件工程		软件应用	2.5 年	2026 年
论文提交日期*	2025 年 12 月 25 日			
导师姓名*	周贤伟		职称*	教授
评阅人	答辩委员会主席*		答辩委员会成员	
	张志超			
电子版论文提交格式　文本（ ）　图像（ ）　视频（ ）　音频（ ）　多媒体（ ）　其他（ ） 推荐格式：application/msword; application/pdf				
电子版论文出版（发布）者		电子版论文出版（发布）地		权限声明
论文总页数*	80			
共 33 项，其中带*为必填数据，为 22 项。				